

PDS - RUST

a.a. 2023/2024 - Rebaudengo

*Questi appunti sono stati scritti rielaborando i contenuti delle lezioni e delle slide del corso. Potrebbero esserci delle imprecisioni ed errori di battitura. Per ulteriori informazioni potete scrivermi una mail o contattarmi su Telegram :)
angelo.iannielli@studenti.polito.it*

INTRODUZIONE

La programmazione di sistema permette di mettere le basi al funzionamento della programmazione applicativa, che necessita di usare la cpu, la memoria, le periferiche e in generale necessita del sistema operativo. Questo tipo di programmazione viene utilizzata per ottenere un funzionamento di tipo concorrente, sfruttando così la collaborazione fra più thread che operano in parallelo.

I linguaggi di programmazione non hanno limiti di spazio e tempo: questi però esistono e sono intrinseci nei componenti hardware. Per esempio, le cpu hanno uno spazio di indirizzamento (32 o 64 bit) che rappresenta il numero di parole che un programma può essere capace di indirizzare.

La scelta di Rust come linguaggio di programmazione è data dall'efficienza di cui è caratterizzato. Le risorse sono limitate, per cui è molto importante, in certe applicazioni, tenerle ben in considerazione. Inoltre, per scrivere codice ad alte prestazioni è necessario scegliere un linguaggio che permetta allo sviluppatore di controllare le risorse (riducendo le opzioni a C, C++, Rust, Swift).

Rust combina un alto livello di controllo con un sistema in grado di identificare una lunga serie di errori, dati dall'utilizzo scorretto di puntatori e interferenze fra thread. I pointer, infatti, possono causare crash puntando a zone di memoria interdette o errate. Inoltre, le zone di memoria occupate dinamicamente devono essere liberate in maniera opportuna.

INFO: Il **garbage collector** è un meccanismo di gestione automatica della memoria: individua le risorse non più utilizzate dal programma e le libera a runtime così che il programmatore non deve fare delle deallocazioni manuali. Riduciamo in questo modo tantissimi errori umani legati alla gestione della memoria ma aumenta il costo e overhead a runtime: le op. del garbage collector possono richiedere molto tempo e causare rallentamenti temporanei e sospensioni nell'esecuzione del programma (no good per sistemi real-time). Rust viene introdotto perché offre un controllo molto elevato sulla memoria senza usare garbage collector. Non si affida ad un sistema automatico di pulizia ma usa regole di OWNERSHIP, BORROWING e LIFETIMES per garantire che già in fase di compilazione la memoria venga gestita in modo sicuro.

INFO: I thread sono sequenze di esecuzione di codice di uno stesso processo che devono scambiarsi informazioni.

PROGRAMMI ED ESECUZIONE

Un eseguibile è formato da istruzioni macchina, dati e valori di configurazione. L'esecuzione di una istruzione prevede l'alternarsi di tre fasi:

- **fetch** = l'istruzione viene trasferita dalla memoria all'interno del processore,
- **decode** = viene trasformata in comandi da eseguire,
- **execute** = i comandi sono eseguiti.

Ogni processo utilizza uno **spazio di indirizzamento virtuale**, cioè una memoria logica separata da quella fisica della RAM. La traduzione da **indirizzo virtuale** a **indirizzo fisico** viene eseguita dalla **MMU (Memory Management Unit)** tramite le **tabelle delle pagine**, gestite dal sistema operativo.

La memoria è organizzata in **pagine**, alle quali possono essere assegnati permessi di **lettura, scrittura ed esecuzione**. Questo meccanismo garantisce isolamento tra processi e permette di eseguire più programmi contemporaneamente senza interferenze.

Con **I**, solo le pagine necessarie vengono caricate in RAM, mentre le altre restano sul disco. Se un processo richiede una pagina non presente in RAM, si verifica un **page fault** e il sistema operativo la recupera dal disco.

INFO: Il **principio di località** dei programmi (o dei riferimenti) stabilisce che se è appena stato fatto un accesso ad un indirizzo, allora è molto probabile che subito dopo ne venga fatto uno ad un indirizzo vicino.

I sistemi non elementari utilizzano delle cache, poste tra il processore e la memoria principale, che vengono usate perché molto più rapide della RAM. Quando si verifica un **miss** nella cache (non è presente il dato richiesto), viene caricata dalla memoria principale una **linea di cache**, ovvero un insieme di dati vicini (oltre a quello cercato) che saranno probabilmente utilizzati successivamente.

Gli aggiornamenti vengono effettuati subito sulla cache, ma poi vengono propagati anche sulla memoria principale. Quando più thread operano sugli stessi dati ma core diversi, occorre gestire correttamente la condivisione dei dati. In questi casi esistono due approcci che possono essere applicati: nel primo la modifica deve essere propagata immediatamente fino alla memoria principale (e non restare confinata alla cache), nel secondo viene invalidata tutta la cache che punta all'indirizzo del dato alterato.

Un compilatore si occupa di tradurre il codice sorgente in codice eseguibile, che non è necessariamente quello che viene lanciato dalla macchina. Con Java, per esempio, viene prodotto un file intermedio, che poi viene tradotto nuovamente dalla Java Virtual Machine. Oltre al codice del programma ci sono anche le librerie di esecuzione.

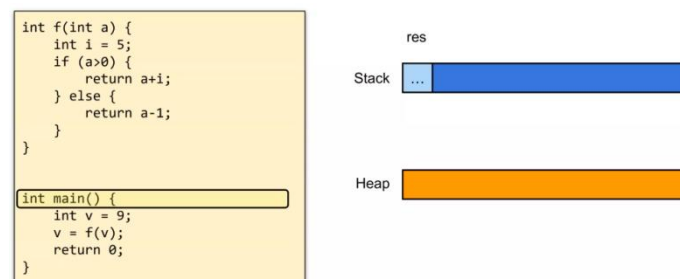
Flusso di esecuzione

Lo **stack** è utilizzato per memorizzare variabili locali, parametri passati alle funzioni e indirizzi di ritorno. Lo **heap** è usato per la gestione dinamica della memoria.

In un programma, ancora prima del main, vengono allocate le variabili globali e vengono eseguite dal sistema operativo alcune operazioni in preparazione dell'esecuzione.

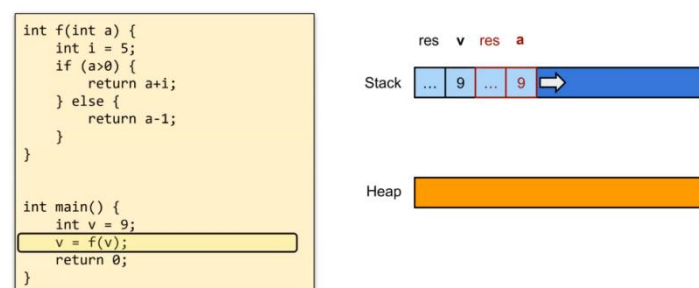
Lo stack ha una dimensione finita (spesso da 1-8 MB). Non può essere usato, per via della sua forte limitazione, per ospitare variabili grandi. Il registro **stack pointer** punta all'ultima variabile salvata nello stack.

INFO: Prima di iniziare il main viene già allocato, nello stack, un blocco di memoria che conterrà il valore di ritorno del programma.

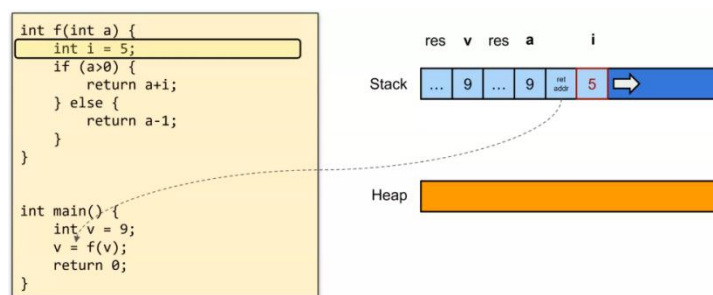


L'accesso alle variabili viene fatto a partire da un riferimento, che può essere lo stack pointer o il base pointer.

Ogni chiamata di funzione predispose lo stack ad un nuovo valore di ritorno.



Prima dell'operazione di call (per chiamare effettivamente la funzione), viene inserito l'indirizzo di ritorno.



Il return dealloca (funzione di pop) la memoria dedicata alla funzione conclusa. Nella pratica, tuttavia, questa memoria non viene realmente eliminata, ma viene solo aggiornato lo stack pointer, tornando indietro.

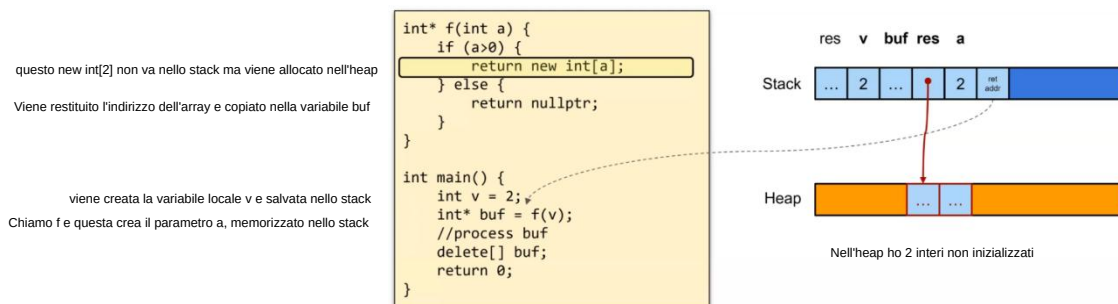
N.B. Non è possibile memorizzare nello stack un dato che duri più a lungo della funzione in cui è stato creato.

Lo **heap** permette di allocare dati che persistano al ciclo di vita della funzione che li hanno allocati. Viene utilizzato per memorizzare dati di grandi dimensioni ed anche per l'allocazione dinamica (ovvero per i dati di cui non si conosce a priori la dimensione). L'accesso a queste variabili viene fatto solo attraverso i puntatori.

La funzione di allocazione dinamica è la **malloc**, a cui è associata la **free** che permette di liberarla. L'utilizzo di queste funzioni crea dei buchi nella memoria, a cui sono associati dei tempi più lunghi di esecuzione (perché il sistema operativo deve trovare lo spazio che soddisfa la richiesta).

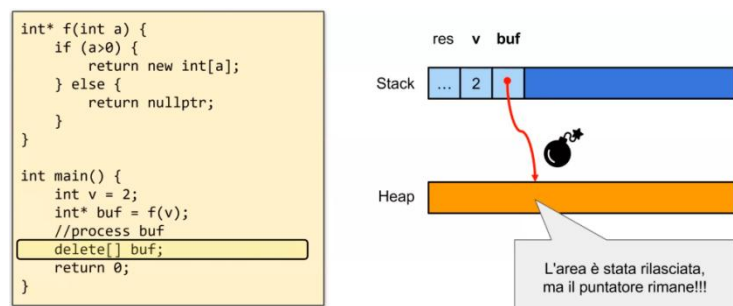
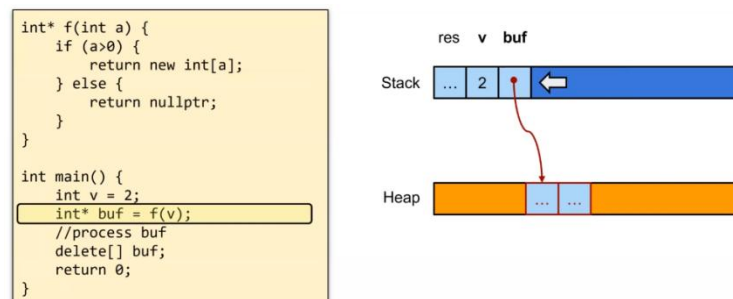
N.B. Non eseguire la free causa un **memory leak**, cioè della memoria inutilizzata resta occupata. Se lo spazio finisce il processo viene distrutto.

INFO: Chiamare due volte la free sullo stesso indirizzo può causare la perdita di dati indesiderata. Questo spazio, appartenente sempre a medesimo processo, potrebbe infatti essere stato occupato da altri dati.



Quando la funzione f termina, a scompare dallo stack e l'array continua ad esistere

Adesso possiamo scrivere:
buf[0]=10;
buf[1]=20;



Ciclo di vita delle variabili

Le variabili **globali** hanno un indirizzo fisso determinato dal compilatore e dal linker, vengono inizializzate all'avvio del programma e deallocate alla fine. Le variabili **locali** hanno un indirizzo relativo alla cima dello stack e hanno un ciclo di vita coincidente con quella del blocco in cui sono dichiarate. Le variabili **dinamiche** sono accessibili solo con un puntatore e hanno un indirizzo assoluto determinato a runtime. La loro durata è controllata dal programmatore, per cui è fondamentale utilizzarle in modo opportuno.

INFO: In C le funzioni di allocazione dinamica sono **malloc**, che prende in input una dimensione e restituisce un puntatore, **calloc**, che prende in input anche un intero n, da moltiplicare alla dimensione (è utilizzato per strutture array), **realloc**, che serve a ingrandire o rimpicciolire lo spazio allocato per un elemento dinamico.

In C++ è possibile avere un riferimento ad un tipo. Inoltre, dopo il primo passaggio di allocazione c'è un passaggio di costruzione della struttura. Il costruttore può inizializzare i dati.

Puntatori

I puntatori servono a manipolare dei dati attraverso il loro indirizzo in memoria. In C possono essere invalidati, attraverso il valore 0 o null.

In C e C++ è difficile sapere se un puntatore contiene un indirizzo valido, perché non sono offerti livelli di garanzia. Inoltre, non contiene informazioni sulla dimensione allocata, per cui non è possibile sapere quanto è grande il blocco di memoria a cui punta.

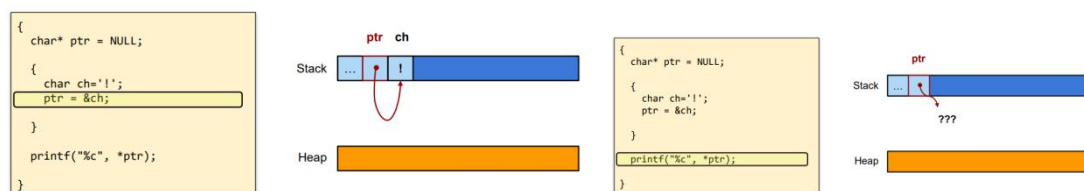
I puntatori, in generale, vengono utilizzati come spazio di appoggio per i risultati delle funzioni e per accedere ad informazioni senza la necessità che vengano copiati.

INFO: Gli smart pointer in C++ causano il rilascio automatico della memoria non appena finito il blocco in cui è stata allocata la memoria dinamica. In particolare i pointer di tipo **unique** non possono avere più copie allo stesso indirizzo, gli **shared** possono essere copiati (tenendo traccia del numero di copie attive).

UNDEFINED BEHAVIOUR

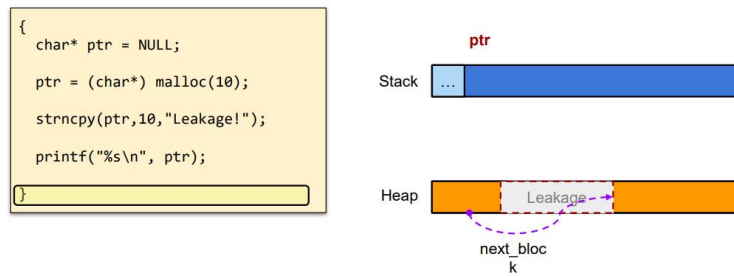
Si definisce **wild pointer** un puntatore con indirizzo non corretto, il cui utilizzo può corrompere aree di memoria in utilizzo da altre parti del programma.

Nel **dangling pointer** si prova ad accedere ad un puntatore il cui ciclo di vita, però, è terminato. La zona di memoria è stata quindi rilasciata, per cui si accede ad un'area non più di possesso del puntatore.

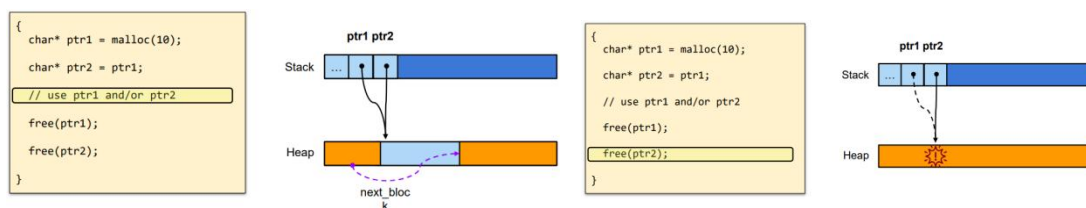


N.B. Facendo la printf non viene visualizzata la stringa perché, essendo appunto stata lanciata una funzione, tutto lo stack successivo viene sovrascritto con indirizzi, valori di ritorno, ... Non può, quindi, essere possibile che la cella precedentemente scritta sia rimasta inalterata.

Nel **memory leakage** viene allocato dello spazio senza poi liberarlo (e perdendo anche il puntatore). In questo modo la memoria rimane occupata benché non utilizzata.



Con il **double free** viene eseguita due volte la funzione di free su un blocco di memoria. Questo può avvenire, per esempio, quando ci sono più puntatori alla stessa cella.



N.B. Il **possessore** di un blocco di memoria è colui che lo alloca e ha la responsabilità di rilasciarlo. In Rust è il compilatore che se ne occupa, evitando una serie di errori ma impedendo allo sviluppatore di essere del tutto libero.

RUST

Rust è un linguaggio che ha lo scopo di assicurare **correttezza**, **velocità** e permettere la programmazione **concorrente**. Rust, tradotto, significa “ruggine” per rimandare all’idea di essere il “più vicino possibile al metallo”. Rust può essere utilizzato per il kernel di un sistema operativo (una parte di Linux è scritta così), ma anche applicazioni server.

Rust è fortemente **tipato** e le variabili mantengono il tipo associato staticamente (quindi non possono cambiarlo). In alcuni casi, attraverso un motore di inferenza, il linguaggio può dedurre il tipo di dato assegnato ad una variabile, per cui non è sempre necessario dichiararlo in modo esplicito.

Si propone di offrire astrazioni a costo nullo, per cui non si deve aggiungere nulla se non richiesto, garantendo la generazione del miglior assembler possibile indipendentemente dallo stile di scrittura di un algoritmo (iterazione, ricorsione, ecc).

Rust è un linguaggio compilato e, a differenza di Java, non può essere compilato una volta e essere eseguito dappertutto. Usa un paradigma imperativo, ma con aspetti funzionali e non utilizza il garbage collector.

N.B. Di default, in Rust le variabili sono **immutabili**.

Il compilatore ottimizza molto le dimensioni e la velocità del codice, facendo in modo di favorire l’uso delle cache e garantendo una buona gestione delle dipendenze.

Viene introdotto il concetto di possesso di un valore, il quale appartiene sempre ad una sola variabile. Questo possesso può essere trasferito se necessario.

In Rust non c’è un meccanismo di gestione delle eccezioni, che in altri linguaggi generano il cosiddetto “stato nascosto”, ovvero una condizione del programma data dal fatto che più funzioni, in catena, si interrompono quando si verifica una eccezione (e quindi danno luogo ad uno stato incognito di esecuzione). Con Rust esiste un tipo di dato detto **Result**, che permette di differenziare il risultato di una funzione tra correttamente eseguito (e quindi conterrà il valore risultante) o “Error”.

N.B. Anche se è disponibile il compilatore di Rust **rustc**, si consiglia l’uso di **cargo**.

INFO: Il progetto, in Rust, viene detto package. Per crearne uno da terminale è possibile lanciare:

```
cargo init rust0
```

Per creare una libreria si può usare:

```
cargo init --lib lib0
```

Ogni pacchetto possiede un file cargo.toml che è la descrizione del progetto stesso e delle dipendenze incluse.

N.B. Il codice prodotto da Rust è completo, quindi include tutte le librerie necessarie.

Si dice **macro** una funzione in grado di generare un’altra funzione. Nel caso di println! viene generata la versione di codice più adatta ad eseguire la print, in base ai parametri che sono stati passati.

CODE: Con le seguenti linee è possibile compilare e lanciare il codice.

```
cargo build  
cargo run
```

IL LINGUAGGIO

Rust favorisce l'immutabilità delle variabili, per cui ognuna di essa è di default non modificabile. Per tutta la durata del programma, inoltre, una variabile non cambia mai il proprio tipo. La parola chiave per dichiarare una variabile è **let**, mentre per dichiarare una mutabile è **let mut**.

N.B. Ogni variabile deve essere sempre inizializzata prima di essere letta.

```
let v: i32 = 123; // v è immutabile e ha tipo i32 (intero a 32 bit con segno)
// v = -5;      // ERRORE: Non è possibile riassegnare il valore

let mut w = v;    // w può essere riassegnata, ha lo stesso tipo di v (i32)
w = -5;          // OK. Ora w vale -5

let x = 1.3278;   // x è immutabile di tipo f64 (floating point a 64 bit)

let y = 1.3278f32; // y è immutabile di tipo f32 (floating point a 32 bit)

let one_million = 1_000_000 // si possono usare '_' per separare le cifre
```

La duplicazione di una variabile con lo stesso nome blocca automaticamente quella precedente. Questa tecnica è detta **shadow**. Le due variabili restano allocate contemporaneamente.

```
fn main() {
    let mut x = 5;

    x = x + 1;

    {
        let x = x * 2;
        println!("The value of x in the inner scope is: {x}");
    }

    println!("The value of x is: {x}");
}
```

INFO: La coppia di parentesi **()** rappresenta la tupla vuota, ovvero **void**.

Quando una espressione non ha il **;** alla fine, viene intesa come ritorno del blocco.

```
fn main() {
    let y = {
        let x = 3;
        x + 1 // espressione
    };

    println!("The value of y is: {y}");
}
```

```
fn main() {
    let condition = true;
    let number = if condition { 5 } else { 6 };

    println!("The value of number is: {number}");
}
```

TIPI

N.B. Non c'è ereditarietà tra tipi, quindi ogni tipo è indipendente.

Gli interi con segno sono i8, ..., i128, isize mentre quelli senza segno sono u8, ..., u128, usize.

Il singolo carattere, char, è identificato dal singolo apice. Per le stringhe, invece, va usato il doppio apice. In C, invece, le stringhe possono essere racchiuse da entrambi i simboli.

INFO: I caratteri delle stringhe seguono la codifica UTF-8.

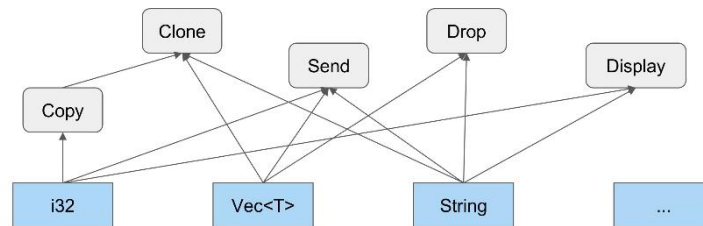
Il tipo void è detto **Unit** ed è rappresentato dalle parentesi ().

TRATTI

Ogni tipo può avere 0 o più tratti, ovvero delle proprietà o comportamenti che questi implementano. I tratti possono essere visti come delle interfacce, la cui implementazione è specifica del tipo che li possiede.

Il tipo int, per esempio, gode del tratto Copy: questo viene applicato quando ad una variabile se ne assegna il valore di un'altra. Il tratto Eq, invece, permette di confrontare l'uguaglianza tra due variabili attraverso ==. Ord, invece, permette il confronto d'ordine (>, <).

A differenza dei tipi, i tratti possono avere delle relazioni di "parentela" tra di loro. In alcuni casi i tratti possono essere mutuamente esclusivi.



INFO: Il tratto **Drop** determina la deallocazione automatica della memoria.

TUPLE

Le tuple sono insiemi di dati eterogenei e sono indicate come (T1, T2, ..., Tn). L'accesso al contenuto di una tupla può essere fatto usando la notazione .0, .1, ...

```
let t: (i32, bool) = (123, false); // t è una tupla formata da un intero
                                   // e da un booleano

let mut u = (3.14, 2.71);          // u è una tupla riassegnabile formata
                                   // da due double
let i = t.0;                       // i contiene il valore 123
u.1 = 0.0;                         // adesso u contiene (3.14, 0.0)
```

N.B. Un array invece è omogeneo e il puntatore si muove con uno spostamento fisso (perché gli elementi hanno grandezza uguale).

PUNTATORI

Il **possesso** coincide con la responsabilità di rilasciare una zona di memoria. Data una cella, in Rust esiste sempre solo un proprietario. Questo, però, non vuol dire che al dato può accedere solo un elemento del programma. I riferimenti, infatti, permettono di accedere a delle informazioni, senza detenerne il possesso.

Il **tempo di vita** di una variabile non è calcolato runtime: è un processo effettuato dal compilatore, che capisce qual è l'ultima riga di codice che utilizza una determinata variabile. Il concetto è più ampio di quello di scope, che rappresenta il blocco in cui una variabile è stata creata.

INFO: I blocchi **unsafe** permettono di lasciare allo sviluppatore il compito di liberare la memoria. Questo esclude tutti i controlli classici di Rust e permette di usare delle librerie non scritte in Rust. In altri casi si vogliono rilassare le restrizioni del linguaggio, nel caso in cui il programmatore fosse sicuro che le operazioni da lui scritte sono corrette.

RIFERIMENTI

Il primo tipo di puntatore è chiamato riferimento (ref, ref mut). Viene introdotto da **&** e permette di puntare ad una certa variabile o ad una espressione.

```
let r1 = &v;
```

Il ref può essere fatto solamente in lettura, la dereferenziazione avviene attraverso il simbolo *****. Un riferimento può essere copiato un numero indiscriminato di volte. In questo caso si dice che la variabile puntata viene "presa in prestito".

N.B. Finché esiste almeno un riferimento in uso, il valore originale a cui punta non si può modificare.

Il riferimento mutabile, al contrario, prende in prestito la variabile in maniera esclusiva, potendo anche modificarla. Questo viene indicato con **&mut**. In questo caso il riferimento non può essere copiato e nessuno può accedere al dato. Attraverso la dereferenziazione è possibile modificare il dato.

```
let r2 = &mut v;  
  
*r2 = ...;
```

INFO: La dereferenziazione viene applicata automaticamente quando, a sinistra del punto nel lancio di una funzione, si trova un puntatore. Questo comportamento funziona in modo annidato.

```
fn main() {  
    let v = 32;  
    let p = &v;  
    let pp = &p;  
    let ppp = &pp;  
  
    let str = ppp.to_string();  
    println!("{str}");  
}
```

Le regole vengono controllate dal **borrow checker**, che si assicura che i puntatori vengano usati correttamente.

```
fn main() {
    let mut i = 32;

    let r = &i;
    println!("{}", *r);

    i = i+1;    // Problematico!
    println!("{}", *r);
}
```

```
error[E0506]: cannot assign to `i` because it is
borrowed
--> src/main.rs:11:3
8 |     let r = &i;
  |           -- borrow of `i` occurs here
...
11 |     i = i+1;
   |     ^^^^^^ assignment to borrowed `i` occurs here
12 |     println!("{}", *r);
   |                   - borrow later used here
```

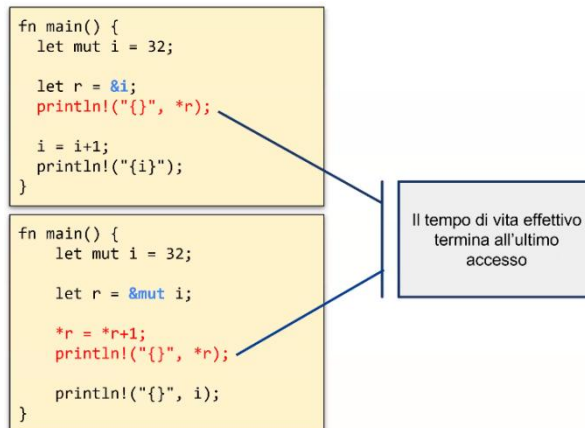
```
fn main() {
    let mut i = 32;

    let r = &mut i;
    println!("{}", i); // Problema!

    *r = *r+1;
    println!("{}", *r);
}
```

```
error[E0502]: cannot borrow `i` as immutable because it
is also borrowed as mutable
--> src/main.rs:9:18
8 |     let r = &mut i;
  |           ----- mutable borrow occurs here
9 |     println!("{}", i);
  |                  ^ immutable borrow occurs here
10
11 |     *r = *r+1;
  |           -- mutable borrow later used here
```

Se il riferimento non viene utilizzato successivamente (e quindi il suo tempo di vita finisce) è possibile accedere in lettura o scrittura anche se precedentemente dato in prestito.



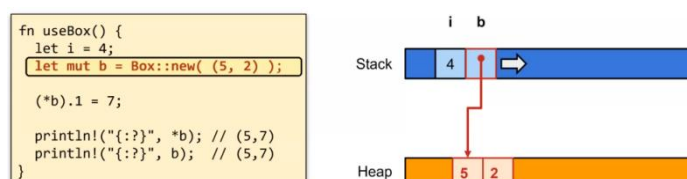
BOX

Si tratta di una struttura dati capace di incorporare qualsiasi tipo di dato. La caratteristica principale risiede nel fatto che la variabile allocata è sempre inserita nello heap. Il Box implementa il tratto Drop, per cui quando sparisce la memoria viene liberata automaticamente.

let b = Box::new(v);

Dentro la variabile b è contenuto il puntatore a v, per cui si può leggere il suo valore nel momento in cui si fa il dereferenziamento.

N.B. Il Box è un puntatore con possesso. Quando il blocco nel quale è contenuto viene concluso, la memoria viene rilasciata (a meno che il possesso non è stato mosso a qualcun altro).



Tratti di stampa

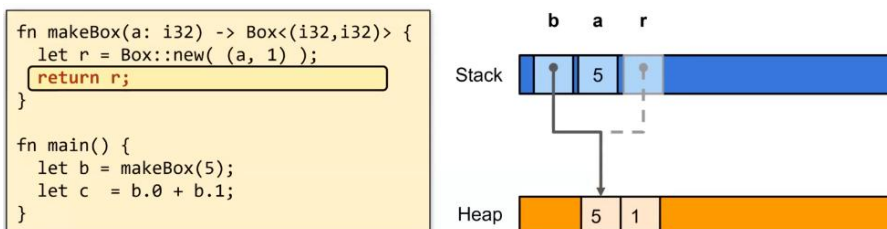
Il tipi semplici godono del tratto **Display**, che permette di stamparne il contenuto riferendosi ad essi con i simboli {}. Il tratto **Debug** ha due opzioni e serve a stampare altri tipi di dato. In particolare **{:?}** stampa le informazioni utili per lo sviluppatore, in modo piuttosto “grezzo”. Con **{:#?}** invece viene utilizzato un formato più leggibile. Il tratto **Pointer {p}** serve a stampare l’indirizzo dei puntatori.

```
fn main () {  
    let x = 842;  
  
    println!("{x:p}");  
}
```

INFO: Anche i dati semplici dispongono di un indirizzo, accessibile attraverso il tratto **{p}**.

Movimento

L’assegnazione di un puntatore, che sposta il possesso di un dato tra due variabili, è detto **movimento**.



INFO: Con una dichiarazione differente è possibile cambiare la mutabilità di una variabile, quando viene attuato uno spostamento.

```
fn makeBox(a: i32) -> Box<(i32,i32)> {  
    let mut r = Box::new( (a, 1) );  
    return r;  
}  
  
fn main() {  
    let mut b = makeBox(5);  
    b.0 = b.0 * 2;  
    let c = b.0 + b.1;  
  
    println!("{c}");  
}
```

Copy e Clone

Le variabili dei tipi che godono del tratto **Copy** possono essere duplicate, creando delle vere e proprie nuove variabili a sè stanti. Se un tipo gode solo del tratto **Clone**, come ad esempio i vettori, quello che accade eseguendo una assegnazione di una variabile ad una nuova è un movimento. Questo implica che il possesso viene spostato alla nuova entità, mentre quella precedente diventa inutilizzabile.

```
fn main() {  
  
    let x: u8 = 123; // u8 implements Copy  
    let y = x;      // x can still be used  
  
    println!("x={}, y={}", x, y);  
  
    let v: Vec<u8> = vec![1, 2, 3]; // Vec<u8> implements Clone, but not Copy  
  
    let w = v;      // This would move the value, rendering v unusable  
  
    println!("w={:?}", w);  
}
```



N.B. La chiamata esplicita del metodo `.clone()` permette invece di effettuare una deep copy: in questo caso viene creata una nuova variabile indipendente e non viene trasferito il possesso di quella precedente.

```
fn main() {
    let mut v: Vec<u8> = vec![1, 2, 3]; // Vec<u8> implements Clone, but not Copy
    let mut w = v.clone(); // This would move the value, rendering v unusable

    v[0] = 10;
    w[1] = 50;

    println!("v={:?} w={:?}", v, w);
}
```



Drop

Il tratto Drop gestisce la liberazione della memoria di una struttura dati. Come per ogni tratto, si possono aggiungere delle funzionalità, come in questo esempio dove è stata aggiunta una stampa.

```
struct S(i32); // struct che contiene un unico campo di tipo i32
               // alla struct associo dei metodi
impl S {
    fn display (&self) { // metodo chiamato su un oggetto di S
                        // ricevendolo come riferimento non mutabile
        println!("Sono S e contengo {} @ {:p}", self.0, self);
                        // mi dice che cosa contiene S e dove è memorizzato
    }
}
impl Drop for S { // metodo utilizzato per rilasciare la memoria
    fn drop(&mut self) // ha la possibilità di manipolare il dato
    {
        println!("Dropping S{} @ {:p}", self.0, self);
    }
}

fn main() {
    let s1 = S(1);
    let s2 = Box::new(S(2));

    s1.display();
    s2.display();
}
```



```
Sono S e contengo 1 @ 0x42f65af844
Sono S e contengo 2 @ 0x23b2c13ab10
Dropping S(2) @ 0x23b2c13ab10
Dropping S(1) @ 0x42f65af844
```

Movimento

Nel momento in cui si definisce una Struct, una variabile non può essere copiata direttamente, perché non possiede il tratto Copy.

```
struct S(i32); // struct che contiene un unico campo di tipo i32
               // alla struct associo dei metodi
impl S {
    fn display (&self) { // metodo chiamato su un oggetto di S
                        // ricevendolo come riferimento non mutabile
        println!("Sono S e contengo {} @ {:p}", self.0, self);
                        // mi dice che cosa contiene S e dove è memorizzato
    }
}
impl Drop for S { // metodo utilizzato per rilasciare la memoria
    fn drop(&mut self) // ha la possibilità di manipolare il dato
    {
        println!("Dropping S{} @ {:p}", self.0, self);
    }
}

fn main() {
    let s1 = S(1);
    let s2 = s1;
    s1.display();
    s2.display();
}
```



```
21 | let s1 = S(1);
    | -- move occurs because `s1` has type `S`,
    |    which does not implement the `Copy` trait
22 | let s2 = s1;
    |    -- value moved here
23 |
24 | s1.display();
    |    ^^ value borrowed here after move
```

È possibile, invece, sfruttare il tratto Clone, attraverso una leggera modifica nell'assegnazione. Aggiungendo il mut si verifica una copia profonda.

```
struct S(i32); // struct che contiene un unico campo di tipo i32
impl S {
    fn display (&self) { // metodo chiamato su un oggetto di S
                        // ricevendolo come riferimento non mutabile
        println!("Sono S e contengo {} @ {:p}", self.0, self);
                        // mi dice che cosa contiene S e dove è memorizzato
    }
}
impl Drop for S {
    fn drop(&mut self) // ha la possibilità di manipolare il dato
    {
        println!("Dropping S({}) @ {:p}", self.0, self);
    }
}

fn main() {
    let s1 = S(1);
    s1.display();
    let mut s2 = s1;

    s2.display();

    s2.0 = 7;
    s2.display();
}
```

drop3.

```
Sono S e contengo 1 @ 0x5353b2f8ec
Sono S e contengo 1 @ 0x5353b2f8f0
Sono S e contengo 7 @ 0x5353b2f8f0
Dropping S(7) @ 0x5353b2f8f0
```

N.B. In questo caso funziona perché avviene un movimento e la vecchia variabile non può essere più acceduta.

N.B. La struct ha una dimensione fissa, viene allocata nello stack.

Puntatori nativi

Si tratta degli equivalenti in C, il cui rilascio della memoria allocata non è gestita dal linguaggio. Si definiscono con ***const T** e ***mut T**, si possono dereferenziare nei blocchi **unsafe {...}**.

Array

Gli array sono sequenze di dati omogenei allocati nello stack. La loro dimensione è fissa e definita nel momento della creazione.

```
let a: [i32; 5] = [1, 2, 3, 4, 5]; // a è un array di 5 interi

let b = [0; 5]; // b è un array di 5 interi inizializzati a 0

let l = b.len(); // NOTARE il ; per distinguere le notazioni // metodo len() calcola la lunghezza // l vale 5

let e = a[3]; // e vale 4
```

Slice

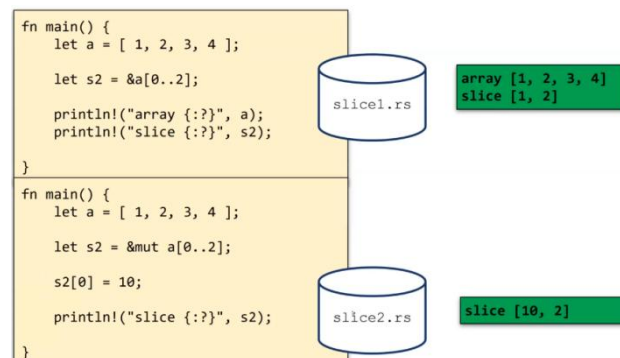
Permette di fare riferimento ad una sequenza di valori consecutivi di lunghezza non nota durante la compilazione. Questa struttura viene detta **fat pointer**, perché contiene un puntatore e il numero di elementi della sequenza. Di base una slice è immutabile, ma il contenuto può essere modificato aggiungendo la parola chiave mut.

```
let a = [ 1, 2, 3, 4 ];
let s1: &[i32] = &a; //s1 contiene i valori 1, 2, 3, 4
let s2 = &a[0..2]; // s2 contiene i valori 1, 2
let s3 = &a[2..]; // s3 contiene i valori 3, 4

let ms = &mut a[..];
```

N.B. La notazione `&a[0..2]` indica che la slice contiene i valori dell'array `a` dall'indice 0 all'indice 2 escluso.

Lo slice di un array permette di accedere ai valori utilizzando la classica notazione degli array con le `[]`.

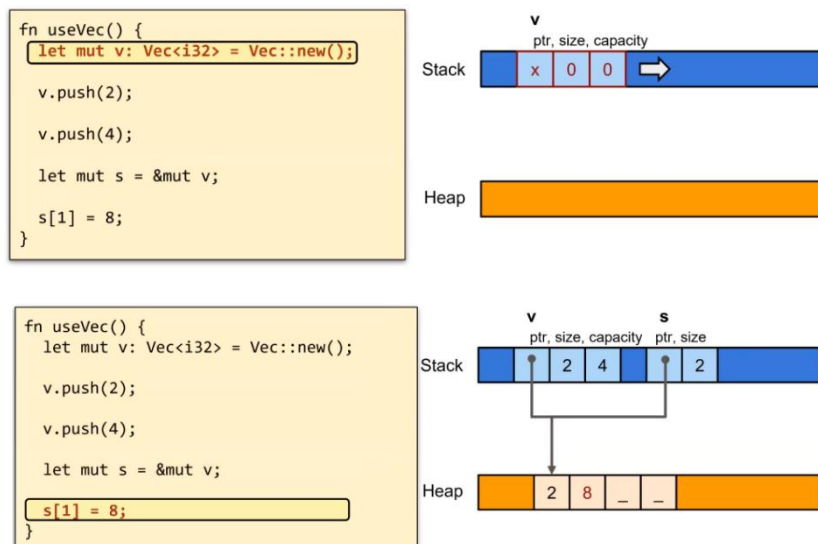


N.B. Finché lo slice mutabile viene utilizzato, l'array iniziale non può essere utilizzato.

Vec<T>

Si tratta di una sequenza ridimensionabile di elementi del tipo `T`, allocata nello heap. La variabile è una tupla di tre valori: un puntatore al buffer nello heap, la dimensione del buffer e il numero di elementi valorizzati.

Se il vettore è pieno ma viene chiesto di aggiungere un elemento, viene allocato un nuovo buffer di dimensioni maggiori e al suo interno viene spostato tutto il contenuto.



INFO: Il metodo **with_capacity()** di `Vec` permette di inizializzare un vettore già con un certo numero di elementi.

Il metodo **pop()** elimina l'ultimo elemento del vettore, **shrink_to_fit()** ridimensiona il buffer con la dimensione che serve realmente.

```

struct Test (i32);

impl Drop for Test {
    fn drop (& mut self) {
        println!("Destroying Test ({} at address {:p}", self.0, self);
    }
}

fn main() {
    let mut v =Vec::<Test>::with_capacity(4);
    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());

    for i in 0..4
    { v.push(Test(i)); }
    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());

    v.push(Test(10));
    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());

    for i in 20..25
    { v.push(Test(i)); }

    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());

    v.pop();

    v.shrink_to_fit();
    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());
}

```



vec1.rs

```

ptr: 0x25f2fa90c30 4 0
ptr: 0x25f2fa90c30 4 4
ptr: 0x25f2fa8e1d0 8 5
ptr: 0x25f2fa8ec40 16 10
Destroying Test (24) at address 0xaa4c0ff664
ptr: 0x25f2fa8e1d0 9 9
Destroying Test (0) at address 0x25f2fa8e1d0
Destroying Test (1) at address 0x25f2fa8e1d4
Destroying Test (2) at address 0x25f2fa8e1d8
Destroying Test (3) at address 0x25f2fa8e1dc
Destroying Test (10) at address 0x25f2fa8e1e0
Destroying Test (20) at address 0x25f2fa8e1e4
Destroying Test (21) at address 0x25f2fa8e1e8
Destroying Test (22) at address 0x25f2fa8e1ec
Destroying Test (23) at address 0x25f2fa8e1f0

```

N.B. Con la pop, il dato viene messo temporaneamente nello stack. L'indirizzo a cui fa riferimento il Drop, infatti, appartiene allo stack e non allo heap.

Il metodo **remove()** permette di eliminare un elemento all'interno del vettore. Se assegnato ad una variabile, il valore di ritorno contiene lo stesso dato. Poiché il valore viene inserito in una variabile che continua ad esistere, non viene visualizzato nessun messaggio di distruzione nel momento della remove.

```

#[derive(Debug)]
struct Test (i32);
impl Drop for Test {
    fn drop (& mut self) {
        println!("Destroying Test ({} at address {:p}", self.0, self);
    }
}

fn main() {
    let mut v =Vec::<Test>::new();
    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());

    v.push(Test(0));

    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());
    println!("&v[0]: {:p}", &v[0]);

    v.push(Test(1));
    v.push(Test(2));
    v.push(Test(3));
    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());

    v.push(Test(4));

    println!("ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());

    v.shrink_to_fit();
    println!("Shrinking ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());

    let a = v.remove(1);
    println!("Removing ptr: {:p} {} {}", v.as_ptr(), v.capacity(), v.len());
    println!("Removed {:?} ", a);

    println!("Terminating");
}

```



vec2.rs

```

ptr: 0x4 0 0
ptr: 0x1e4f5cddf60 4 1
&v[0]: 0x1e4f5cddf60
ptr: 0x1e4f5cddf60 4 4
ptr: 0x1e4f5cdec0 8 5
Shrinking ptr: 0x1e4f5cde080 5 5
Removing ptr: 0x1e4f5cde080 5 4
Removed Test(1)
Terminating
Destroying Test (1) at address 0x9b774ff61c
Destroying Test (0) at address 0x1e4f5cde080
Destroying Test (2) at address 0x1e4f5cde084
Destroying Test (3) at address 0x1e4f5cde088
Destroying Test (4) at address 0x1e4f5cde08c

```

Stringhe

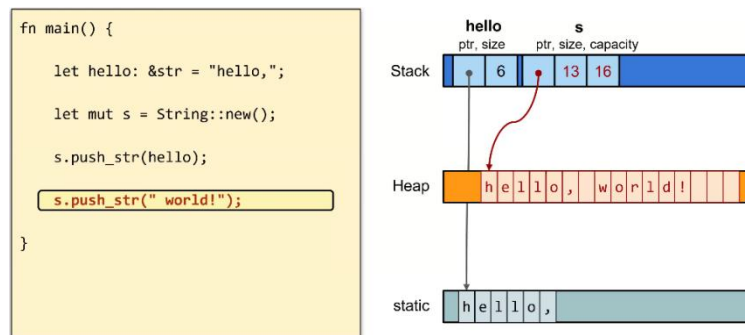
Le stringhe in Rust hanno due modi di essere rappresentate. La prima, immutabile, è dichiarata attraverso la keyword **str**, mentre la seconda, mutabile, con **String**.

I caratteri dei **char** sono codificati usando **Unicode** (che prevede, per ognuno di essi, 32 bit). le stringhe, però, usano caratteri **UTF-8**, che usano un numero di byte diverso in base alla “popolarità” dell’elemento da rappresentare (caratteri frequenti avranno pochi byte, caratteri più rari ne avranno un numero superiore).

Le stringhe costanti sono memorizzate in un’area statica della memoria e durano per tutto il programma.



Le String (mutabili) usano lo stesso approccio dei Vec, per cui esiste un oggetto che contiene un puntatore al buffer dinamico, la lunghezza e la capacità.



La funzione **push_str()** permette di aggiungere una stringa in coda alla variabile.

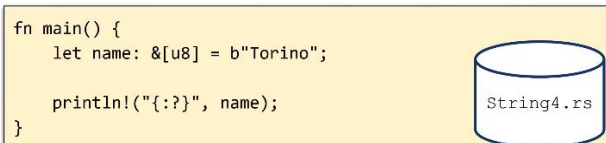
INFO: La capacità è sempre una potenza di 2.

Le String possono essere create con il metodo **new()** o con il **from()**. Viceversa si può trasformare una stringa dinamica in una statica attraverso **as_string()**.

```
let s0 = String::new(); //crea una stringa vuota  
let s1 = String::from("some text"); //crea una stringa inizializzata  
let s2 = "some text".to_string(); //equivalente al precedente  
s2.as_str();
```

Altri metodi utili sono **clear()**, **remove(indice_carattere)**, **to_uppercase()**, **replace("some", "more")**, **trim()**. Questi creano tutti un'altra stringa.

INFO: Quando una stringa è preceduta dalla b, indica al compilatore che deve essere considerata come sequenza di byte.

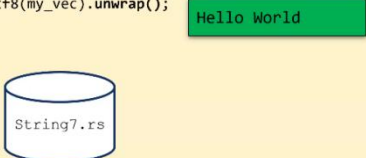


Il metodo **from_utf8(&bytes_array)** stampa come caratteri una sequenza di byte. Il risultato è nella forma `Ok("stringa")`, perché il ritorno è di tipo `Result`. Per visualizzare il contenuto pulito, occorre utilizzare **unwrap()**.

```
fn main() {
    let my_vec: Vec<u8> = vec![72, 101, 108, 108, 111, 32, 87, 111, 114, 108, 100];
    // "Hello World" in ASCII

    let vec_to_string = String::from_utf8(my_vec).unwrap();
    // Converting to string

    println!("{}", vec_to_string);
}
```



N.B. Se il `Result` è un `Error`, l'`unwrap` genera uno stato di **Panic** (per cui il programma viene arrestato).

INFO: Su Linux i caratteri sono in UTF-8 (quindi c'è completa compatibilità con Rust) mentre su Windows la codifica è su 16 bit. Quando si interagisce col sistema operativo, quindi, si utilizzano i tipi **OsStr** e **OsString**.

Similmente anche i separatori dei path sono diversi. In Linux si usa lo slash `"/"`, mentre su Windows il back slash `"\"`. La conversione, a partire dalla stringa, è possibile con i tipi **Path** e **PathBuf** (mutabile).

INFO: Siccome C utilizza le stringhe con un terminatore, per passarle da Rust si utilizzano **Cstr** e **Cstring**.

INFO: Il tipo corretto per inizializzare una stringa costante è **&'static str**.

Loop

Il **loop** genera una serie di iterazioni infinita. L'istruzione per uscire è **break**.

```
fn main() {
    let mut counter = 0;
    let mut sum = 0;

    let result = loop {
        counter += 1;
        sum += counter;

        if counter == 10 {
            break sum * 2;
        }
    };

    println!("The result is {result}");
}
```



Nei loop annidati è possibile specificare da quale ciclo si deve uscire. La parola chiave **break** consente di uscire da ciclo interno (quello dove si trova), il **continue** da quello più esterno. Con delle etichette è possibile dare un'indicazione specifica del ciclo da cui si vuole uscire.

```
fn main() {
    'outer: loop {
        println!("Entrato nel ciclo esterno");

        'inner: loop {
            println!("Entrato nel ciclo interno");

            // La prossima istruzione interromperebbe il ciclo interno
            //break;

            // Così si interrompe il ciclo esterno
            break 'outer;
        }
        //Il programma non raggiunge mai questa posizione
    }
    println!("Terminato il ciclo esterno");
}
```

Funzioni

Permettono di definire un comportamento, a partire da una lista di argomenti. Se non ritornano il valore `Unit ()`, il tipo ritornato deve essere indicato con la signature, preceduto dal simbolo `->`.

L'espressione non seguita dal `;` permette di definire un valore di ritorno.

```
fn print_number(x: i32) /* -> () */ {  
    println!("x is: {}", x);  
}
```

```
fn add_numbers(x: i32, y: i32) -> i32 {  
    x + y // NON c'è il ; finale  
}
```

INFO: In Rust non esiste l'istruzione `++` per incrementare un contatore. Al suo posto si usa `+= 1`.

For

Il `for` permette di iterare su un contenitore, per esempio su un vettore o una stringa. È possibile anche farlo iterare su una sequenza di valori.

La scansione può essere rappresentata con diverse notazioni:

- `..` indica tutti i valori possibili per un dato dominio
- `a..` indica tutti i valori a partire da **a** (incluso)
- `..b` indica tutti i valori fino a **b** (escluso)
- `..=c` indica tutti i valori fino a **c** (incluso)
- `d..e` indica tutti i valori tra **d** (incluso) ed **e** (escluso)
- `f..=g` indica tutti i valori tra **f** e **g** (inclusi)

```
fn main() {  
    let a = [10, 20, 30, 40, 50];  
    for element in a {  
        println!("the value is: {element}");  
    }  
  
    for n in 1..10 {  
        println!("{}", n);  
    }  
  
    let names = ["Bob", "Frank", "Ferris"];  
    for name in names.iter() {  
        println!("{}", name);  
    }  
  
    for name in &names[ ..=1 ] {  
        println!("{}", name);  
    }  
  
    for number in (1..=10).rev() {  
        println!("{}", number);  
    }  
  
    for (i,n) in names.iter().enumerate() {  
        println!("names[{}]: {}", i, n);  
    }  
}
```



Match

È l'equivalente dello `switch`, ma, invece di fare un confronto con alcune costanti, permette di utilizzare dei pattern **esaustivi** del dominio (anche con l'utilizzo di variabili).

```
let s = match item {  
    0 => "zero",  
    10 ..= 20 => "tra dieci e venti",  
    40 | 80 => "quaranta o ottanta",  
    _ => "altro",  
}
```

// valore singolo
// intervallo inclusivo
// alternativa
// qualunque cosa

Riga di comando

La libreria `std::env::args` permette di accedere ai parametri della linea di comando. Il metodo `collect()` permette di aggiungere tutti i parametri ad un vettore di stringhe. Poiché il primo parametro è il nome del programma, viene saltato.

```
use std::env::args;
fn main() {
    let args: Vec<String> = args().skip(1).collect();
    if args.len() > 0 { // we have args!
        println!("{}", args[0]);
    }
}
```



La libreria `clap` serve a creare un ambiente con un menu. La serie di caratteri `///` permette di definire una spiegazione da far stampare sul terminale, quando si richiama l'help del programma (la libreria implementa anch'esso).

```
use clap::Parser;

/// Simple program to greet a person
#[derive(Parser, Debug)]
#[command(version, long_about = None)]
struct Args {
    /// Name of the person to greet
    #[arg(short, long)]
    name: String,
    /// Number of times to greet
    #[arg(short, long, default_value_t = 1)]
    count: u8,
}

fn main() {
    let args = Args::parse();
    for _ in 0..args.count {
        println!("Hello {}!", args.name)
    }
}
```

```
$ demo --help
Simple program to greet a person

Usage: demo[EXE] [OPTIONS] --name <NAME>

Options:
  -n, --name <NAME>  Name of the
                      person to greet
  -c, --count <COUNT>  Number of times
                      to greet [default: 1]
  -h, --help           Print help
  -V, --version         Print version

$ demo --name Me
Hello Me!
```



La struttura `Args` permette di indicare le opzioni del programma. Queste si possono richiamare con il loro nome, anticipato dai segni `--`, oppure con la sola iniziale preceduta dal singolo `-`.

I/O da console

Il pacchetto `std::io` permette di eseguire operazioni di input e output. Restituiscono oggetti `Result` e per garantire la correttezza del programma si può verificare il contenuto attraverso il metodo `is_ok()`.

La funzione `read_line()` permette di leggere un input da tastiera.

```
use std::io;

fn main() {
    let mut s = String::new();

    if io::stdin().read_line(&mut s).is_ok() {
        println!("Got {}", s.trim());
    } else {
        println!("Failed to read line!");
    }

    //alternativamnte
    io::stdin().read_line(&mut s).unwrap();
    println!("Got {}", s.trim());
}
```



POSSESSO

Ogni valore, in Rust, è posseduto da una e una sola variabile, la quale è responsabile di rilasciare il dato. Le risorse vengono rilasciate quando si esce lo scope (nel caso dei puntatori viene liberato anche lo spazio usato sullo heap).

Quando ad una variabile dinamica (per esempio un Box) viene assegnato un nuovo valore, quello precedente viene liberato automaticamente (sempre alla fine dello scope, nel momento dell'assegnazione viene "oscurato").

```
fn main() {  
    let mut my_box = Box::new(1);  
    my_box = Box::new(2);  
    println!("{:?}", my_box);  
}
```

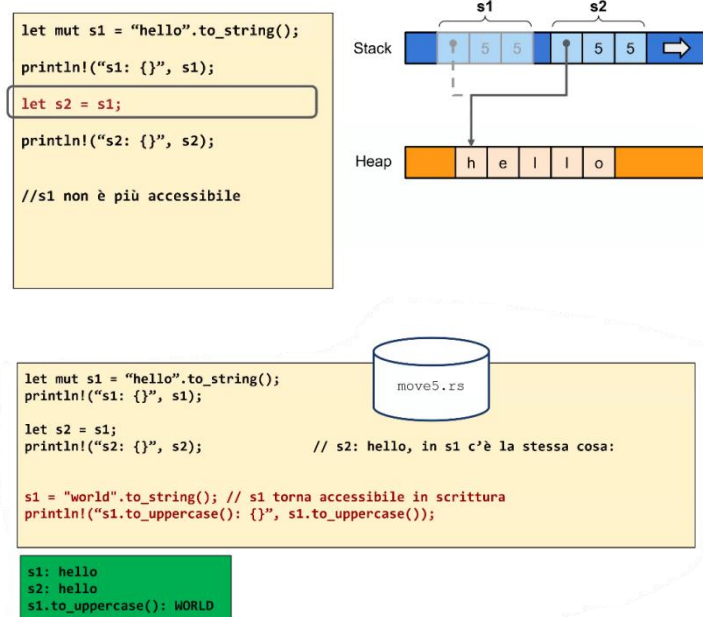


Allo stesso modo, anche con le stringhe la gestione della memoria è tale da evitare il leakage.

```
fn main() {  
    let s1 = String::from("hello");  
    println!("{s1}");  
    let mut s1 = String::from("ciao");  
    s1.push_str(" Mamma");  
    println!("{s1}");  
}
```



L'operazione del **movimento** sposta il possesso da una variabile di partenza a un'altra. Quella iniziale, dopo la nuova assegnazione, non può essere più utilizzata, a meno di fornirle una nuova inizializzazione.



N.B. Il movimento si ottiene anche quando vengono passate delle variabili alle **funzioni**. Il chiamante, infatti, perde il possesso delle variabili, che diventano di proprietà della funzione lanciata.

```
fn main() {
    let s = String::from("hello");
    takes_ownership(s);
    println!("{}", s);
}

fn takes_ownership(some_string: String) {
    let s = some_string.to_uppercase();
    println!("{}", s);
}
```

consumo2.rs

```
7 | fn takes_ownership(some_string: String) {
  | ----- ^^^^^ this parameter takes ownership of the value
  |         |
  |         in this function
= note: this error originates in the macro `crate::format_args_nl` which comes from
the expansion of the macro `println` (in Nightly builds, run with -Z macro-backtrace for
more info)
help: consider cloning the value if the performance cost is acceptable
3 |     takes_ownership(s.clone());
  |                     +++++++
```

Alcuni tipi elementari implementano **Copy** e per questo sono detti copiabili. Quando un valore viene assegnato ad un'altra variabile (o passato a una funzione), infatti, resta accessibile in lettura anche dalla variabile iniziale.

INFO: Tra i tipi elementari che implementano questo tratto, ci sono i numerici e in generale le loro combinazioni (tuple, array). I riferimenti ai valori mutabili, tuttavia, non possono essere copiati.

~ move (for types that do not implement Copy)

```
let s = String::from("hello");
let b = s + " world";
```

(cannot use s anymore)

copy (for types that do implement Copy)

```
let i = 42;
let j = i + 1;
```

(can still use i and j)

N.B. Se, ad esempio, un box viene letto per essere messo all'interno di una tupla, questo ne provoca il movimento, per cui la variabile iniziale smette di accedere al dato iniziale.

```
let x1 = 42;
let y1 = Box::new(84);
{
    let z = (x1, y1);
    println!("z: {:?}", z);
}
let x2 = x1;
println!("x2: {}", x2);
let y2 = y1;
println!("y2: {}", y2);
```

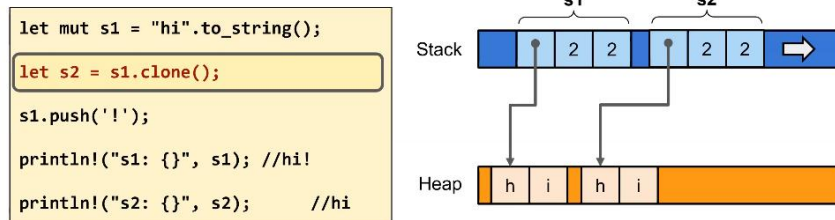
move6.rs

```
4 | let y1 = Box::new(84);
  | -- move occurs because `y1` has type
  | `Box<i32>`, which does not implement the `Copy` trait
...
7 | let z = (x1, y1);
  | -- value moved here
...
13 | let y2 = y1;
   | ^^ value used here after move
help: consider cloning the value if the performance cost
acceptable
7 | let z = (x1, y1.clone());
```

La **clonazione** si ottiene col tratto **Clone** e permette di creare una copia identica all'originale. In questo caso vengono copiati tutti i dati e non solo dei puntatori.

N.B. La clone deve essere usata in maniera esplicita.

INFO: La Copy è in mutua esclusione con il tratto Drop.



In Rust il **borrow checker** è un blocco di codice che verifica se una variabile è ancora accessibile.

RIFERIMENTI

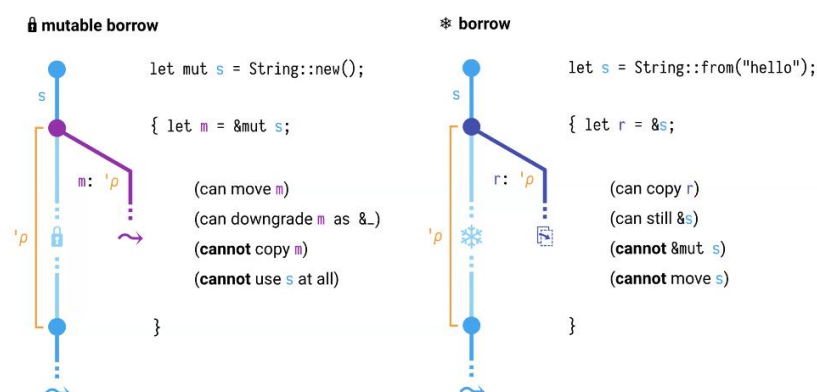
I **riferimenti** permettono di accedere ai dati (in lettura), senza però entrarne in possesso.

```
let point = (1.0, 0.0); //point possiede il valore

let reference = &point; //reference PUÒ accedere al valore in lettura
                        //finché esiste point
println!("{}", reference.0, reference.1);
```

Un riferimento prende in prestito (borrow) l'indirizzo di memoria in cui si trova il valore che si vuole usare. I riferimenti possono essere copiati, ma finché almeno uno di essi è accessibile, il valore non può essere modificato dalla variabile che ne detiene la proprietà.

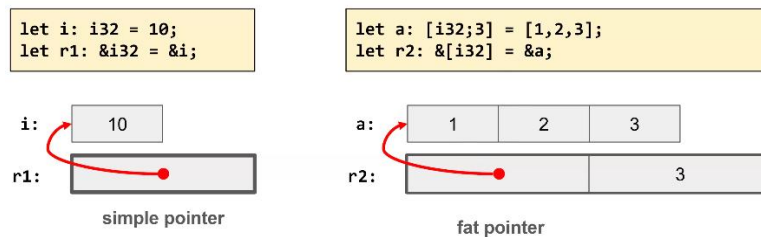
I **riferimenti mutabili** permettono la modifica del valore di una variabile. Sono mutuamente esclusivi, per cui può essercene solo uno per volta collegato a ciascuna variabile. In questo caso, il possessore originale non può accedere neanche in lettura al proprio dato, almeno finché il riferimento non smette di essere utilizzato.



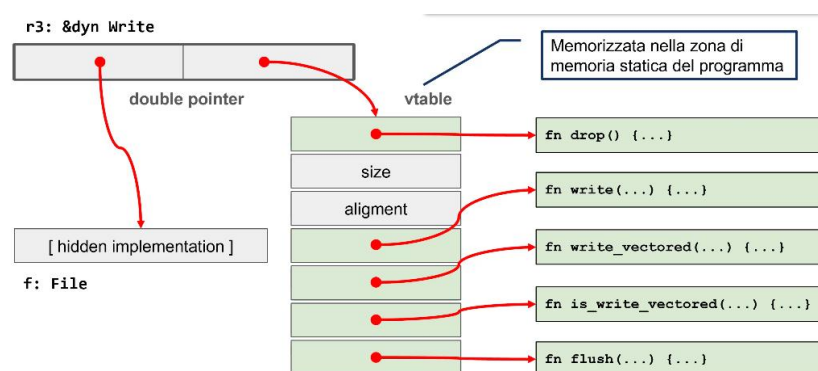
INFO: Attraverso l'apice ' è possibile assegnare una label al tempo di vita delle variabili.

Puntatori in memoria

Ci sono 3 diversi tipi di puntatori. Il **puntatore semplice** possiede solo un indirizzo ad una cella di memoria, mentre un **fat pointer** possiede anche la dimensione della struttura a cui sta puntando.



Un **puntatore doppio** viene utilizzato per indicare un dato noto solo per l'insieme dei tratti che implementa.



INFO: Le **vtable** non sono duplicate per ogni dato, ma ne esiste una sola copia per tipo, a cui poi fanno riferimento tutte le istanze.

Tempo di vita di un riferimento

Il tempo di vita corrisponde sempre ad un insieme di righe di codice, non allo scope della variabile. Questo, infatti, non rappresenta il tempo in cui una variabile resta allocata, bensì al tempo in cui è accessibile.

Questo permette al borrow checker di essere molto preciso durante le sue verifiche. Garantisce, infatti, che tutti gli accessi avvengano solo in un intervallo di tempo (righe) compreso in quelle in cui il dato esiste e serve a impedire il fenomeno dei **dangling pointer** (accesso ad aree rilasciate).

INFO: L'insieme di righe in cui si accede al riferimento viene detto **lifetime**.

Il tempo di vita di una variabile può essere specificato attraverso l'assegnazione di una etichetta, nella firma del tipo. La notazione è la seguente:

&'a NomeTipo

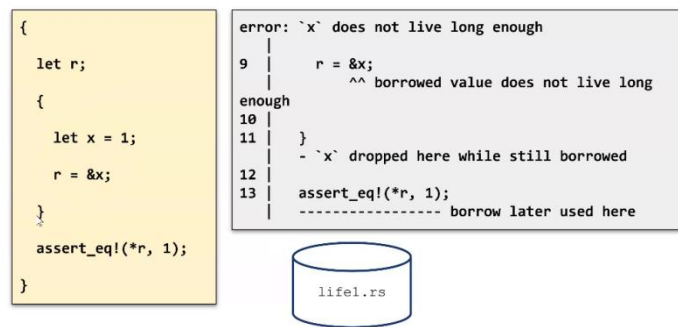
Dove **a** è l'etichetta e **'** (tick) serve a qualificarlo come lifetime.

Se invece un riferimento è valido per tutto il programma viene indicato con:

&'static NomeTipo

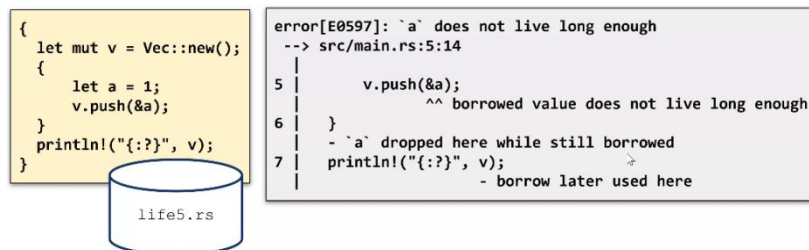
INFO: Le stringhe espresse in formato letterale (direttamente con `""`) hanno il tipo `&'static str`.

N.B. Il tempo di vita di un puntatore deve essere contenuto in quello della variabile a cui punta.



INFO: La macro `assert_eq` esegue un confronto, se non è verificato allora lancia un panic.

N.B. I riferimenti presenti in una struttura devono avere un tempo di vita maggiore della stessa struttura, altrimenti questa verrebbe invalidata.



Le slice permettono di creare dei riferimenti per prendere in prestito parti in una struttura, ad esempio un vec o una box.

INFO: La macro `vec!` permette di creare e inizializzare un vettore.

```
fn main() {
  let vec = vec![1, 2, 3, 4, 5];

  // Ottenere una slice che include tutto il vettore
  let slice1: &[i32] = &vec[..];

  // Ottenere una slice che include solo una parte del vettore
  let slice2: &[i32] = &vec[1..3]; // Dal secondo all'indice 3 (escluso)

  println!("{:?}", slice1); // Stampa: [1, 2, 3, 4, 5]
  println!("{:?}", slice2); // Stampa: [2, 3]
}
```

```
fn main() {
  let boxed_slice: Box<[i32]> = Box::new([1, 2, 3, 4, 5]);

  // Ottenere una slice da un Box<[i32]>
  let slice: &[i32] = &*boxed_slice;

  println!("{:?}", slice); // Stampa: [1, 2, 3, 4, 5]
}
```

```
fn main() {
  let boxed_slice: Box<[i32]> = Box::new([1, 2, 3, 4, 5]);

  // Creare una slice che considera solo una parte del vettore contenuto nel Box
  let slice: &[i32] = &boxed_slice[1..3]; // Da 2 a 3 (escluso 3)

  println!("{:?}", slice); // Stampa: [2, 3]
}
```

TIPI COMPOSTI

INFO: In C/C++ la **enum** è un tipo che permette di definire un elenco di valori attribuibili (e a ognuno dei quali viene attribuito un valore numerico). La **union** permette di far assumere alla variabile valori di tipo diverso (in modo esclusivo). Non esiste un modo per capire che tipo di valore è stato inserito, per cui è compito del programmatore sapere cosa è stato fatto.

```
union sign
/* A definition and a declaration */ C/C++
{
    int svar;
    unsigned uvar;
} number;
```

```
enum DAY          /* Defines an enumeration type */
{
    sunday = 0,    /* Names day and declares a */
    monday,        /* variable named workday with */
    tuesday,       /* that type */
    wednesday,     /* wednesday is associated with 3 */
    thursday,
    friday,
    saturday       /* saturday is associated with 6 */
} workday;
```

Una **struct** è una struttura dati che mette insieme tipi eterogenei in diversi campi. Permette, quindi, di definire un nuovo tipo di dato, che si compone di più attributi.

INFO: Per convenzione, le struct sono indicate in CamelCase, mentre variabili e funzioni usano lo snake_case.

```
let mut s = Player { name: "Mario".to_string(), health: 25, level: 1 };
let p = Player { name, health, level }; // {name: name, health: health, level:
level}
```

L'accesso ai campi segue la notazione **var.field**.

INFO: L'inizializzazione può essere fatta prendendo i valori da un'altra istanza della struct.

```
#[derive(Debug)]
struct Player {
    name: String, // nickname
    health: i32,  // stato di salute (in punti vita)
    level: u8,    // livello corrente
}

fn main() {
    let p1 = Player {name: "Mario".to_string(), health: 25, level: 1 };
    let p2 = Player {name: "Giovanni".to_string(), .. p1};

    println!("{:?} \n{:?}", p1, p2);
}
```



INFO: Esiste una particolare struct che non assegna dei nomi ai campi. In questo caso la dichiarazione è fatta usando le parentesi tonde. Di fatto è una sorta di tupla.

```
struct Playground ( String, u32, u32 );
struct Empty;    // non viene allocata memoria per questo tipo di valore

let mut f = Playground( "football".to_string(), 90, 45 );
let e = Empty;
```

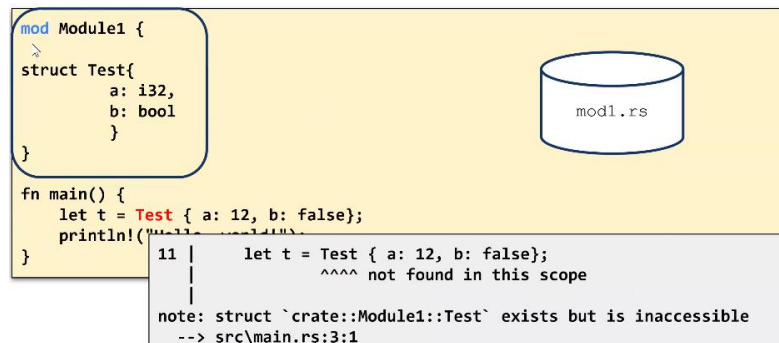
INFO: Il compilatore ottimizza la disposizione dei campi per compattare la memoria. Inoltre, per garantire l'allineamento, può utilizzare un padding. Questo implica che l'ordinamento dei campi in memoria non è necessariamente uguale a quello dichiarato (ma la logica di accesso lo è, quindi un accesso .1, .2, ... è consentito).

Per avere una rappresentazione coerente col C, bisogna anteporre **#[repr(C)]**.

Moduli e costruttori

Rust parte dall'assunzione che tutto è privato. La **privacy** è limitata al modulo in cui la Struct è definita, ovvero alla porzione del file sorgente.

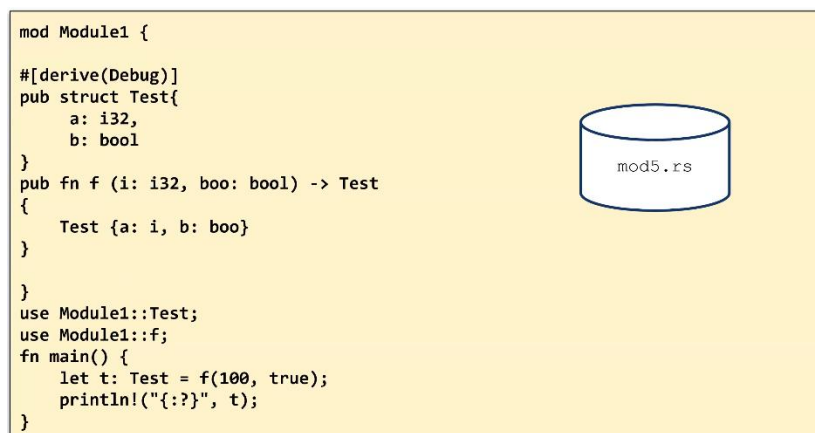
Un modulo si introduce attraverso la parola **mod**. Se una struttura viene dichiarata all'interno di un modulo, questa può essere usata solo al suo interno, oppure attraverso una chiamata.



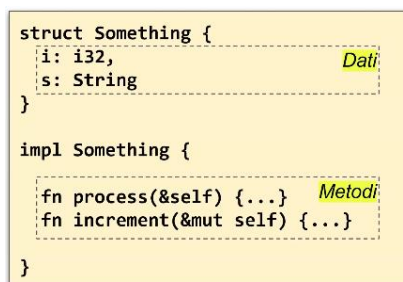
Per richiamare una struttura di un modulo si usa la notazione:

use ModuleName::StructName

N.B. La struttura, per essere utilizzata fuori dal modulo in cui è scritta, deve essere resa pubblica attraverso la keyword **pub**. La parola chiave va posta sia prima della kw struct, che prima dei campi a cui si vuole accedere direttamente dall'esterno. È possibile, infatti, accedere ai singoli campi utilizzando, però, dei metodi (pubblici).



N.B. In Rust non ci sono le classi. I metodi vanno definiti in modo sparato delle struct, in un blocco **impl**.



INFO: Il **ricevitore** è la variabile su cui viene applicata la funzione. Questo viene identificato da **self**.

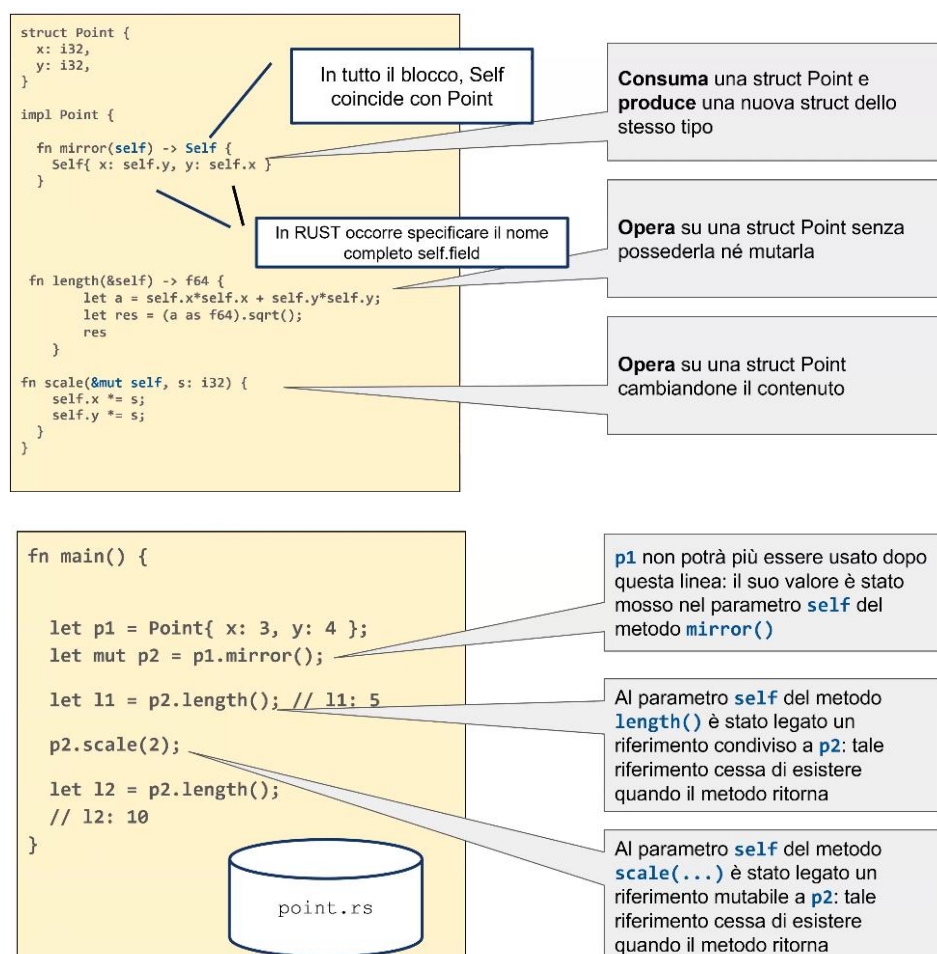
Ad una funzione può essere passato il ricevitore in tre forme: **self** (per **movimento**), **&self** (per riferimento **condiviso**) e **&mut self** (per riferimento **esclusivo**).

N.B. Il passaggio di una variabile ad una funzione ne causa il suo consumo: il possesso, infatti, viene trasferito al metodo e la funzione chiamante non può più utilizzarla. Per questo è importante passare un riferimento, e non la variabile in sé, quando si prevede di continuare ad utilizzare il dato nel blocco chiamante.

Le funzioni che non hanno come parametro **self** sono dette **funzioni associate** e svolgono il ruolo di costruttori o da metodi statici.

INFO: I blocchi **impl** di una struct possono essere molteplici.

INFO: Il tipo **Self** va scritto con la iniziale maiuscola, **self** (minuscolo) indica invece l'istanza della struct.



In Rust non esiste il concetto di costruttore, ma è possibile definire delle funzioni che permettano di inizializzare le istanze. In generale, per convenzione, un metodo di questo tipo viene denominato con `new()` se non prende alcun parametro, altrimenti con `with_details(...)` se prende parametri.

```
pub fn new() -> Self { ... }
with_details( ... ) -> Self { ... }
```

N.B. In Rust non c'è l'overloading delle funzioni, per cui funzioni che svolgono le stesse operazioni ma prendono parametri di tipo diverso devono avere anche nomi diversi.

```
#[derive(Debug)]
struct Player {
    name: String, // nickname
    health: i32,  // stato di salute (in punti vita)
    level: u8,    // livello corrente
}

impl Player {
    fn with_name(s: String) -> Self {
        Self {name: s, health: 25, level: 1}
    }
}

fn main() {
    let p1 = Player::with_name("Mario".to_string());

    println!("{:?}", p1);
}
```



INFO: Il C++ segue il paradigma **RAII** per cui chi acquisisce una risorsa è responsabile del suo rilascio. In particolare, il costruttore ha il compito di acquisire le risorse e il distruttore di rilasciarle (questo avviene non appena si esce fuori dallo scope della variabile).

In Rust il **distruttore** è un tratto, detto **Drop**. Questo tratto implementa il metodo `drop(&mut self)` ed è in mutua esclusione con `Copy`.

```
pub struct Shape {

    pub position: (f64, f64),
    pub size: (f64, f64),
    pub type: String

}
```

```
impl Drop for Shape {

    fn drop(&mut self) {
        println!("Dropping shape!");
    }

}
```

ENUM

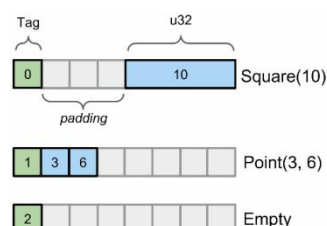
Un enum viene definito attraverso il dominio dei valori che può assumere. A differenza di altri linguaggi, può possedere anche dei metodi e ciascuna variante può essere di tipo diverso.

```
enum HttpResponse {
    Ok = 200,
    NotFound = 404,
    InternalError = 500
}
```

```
enum HttpResponse {
    Ok,
    NotFound(String),
    InternalError {
        desc: String,
        data: Vec<u8> },
}
```

INFO: I tipi algebrici possono essere di tipo somma o prodotto. Nel secondo si ha il prodotto dei domini dei parametri ed è il caso delle struct. Il primo ha come dominio l'unione dei valori delle singole alternative ed è il caso delle enum.

In memoria gli enum possiedono un byte dedicato al tag (che indica la variante assunta). Il resto dello spazio dipende dalla variante più grande (+ padding).



```
enum Shape {
    Square(u32),
    Point(u8, u8),
    Empty,
}
```

INFO: La direttiva `#![allow(dead_code)]` serve a far ignorare al compilatore la presenza di codice inutilizzato.

```
#![allow(dead_code)]

#[derive(Debug)]
enum Color {
    Red,
    Green = 50,
    Yellow
}

fn main() {
    println!("Colore Rosso {:?} {:?}", Color::Red, Color::Red as i32);
    println!("Colore Giallo {:?} {:?}", Color::Yellow, Color::Yellow as i32);
}
```



```
Colore Rosso Red 0
Colore Giallo Yellow 51
```

Gli enum vengono molto usati con **match**, che permette di analizzare tutti i casi possibili.

```
enum Operazioni {
    Add,
    Subtract,
}

impl Operazioni {
    fn run(&self, x: i32, y: i32) -> i32 {
        match self {
            Self::Add => x + y,
            Self::Subtract => x - y,
        }
    }
}

fn main() {
    let op1 = Operazioni::Add;
    let op2 = Operazioni::Subtract;

    let res = op1.run(5,7);
    println!("{}", res);
    let res = op2.run(5,7);
    println!("{}", res);
}
```



```
enum Shape {
    Square { s: f64 },
    Circle { r: f64 },
    Rectangle { w: f64, h: f64 }
}
```

```
fn compute_area(shape: Shape) -> f64 {
    match shape {
        Shape::Square { s } => s*s,
        Shape::Circle { r } => r*r*3.1415926,
        Shape::Rectangle {w, h} => w*h,
    }
}
```

Quando non è necessario analizzare tutti i casi possibili è possibile utilizzare il pattern matching e la destrutturazione attraverso le righe:

oppure

```
if let <pattern> = <value> ...

while let <pattern> = <value>
```

```
fn process(shape: Shape) {
    // stampa solo se shape è Square...
    if let Square { s } = shape {
        println!("Square side {}", s);
    }
}
```

INFO: La destrutturazione permette di spaccettare un dato complesso, estraendone il contenuto.

La enum **Option** permette di stabilire due casi: uno in cui è presente un valore, l'altro invece no. In particolare le due varianti sono **Some(T)** e **None**.

```
fn divide(numerator: f64, denominator: f64) -> Option<f64> {
    if denominator == 0.0 {
        None
    } else {
        Some(numerator / denominator)
    }
}

fn main() {
    let result = divide(820.0, 15.2);

    match result {
        Some(value) => println!("La divisione è {}.", value),
        None => println!("Non è possibile eseguire la divisione."),
    }
}
```



La enum **Result** permette di gestire gli errori. Le varianti sono **Ok(T)** e **Err(E)**. L'Ok permette di restituire un valore, mentre l'errore tipicamente ritorna una stringa.

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {
    if y == 0.0 {
        // Returning an error if division by zero is attempted
        Err("Division by zero!")
    } else {
        // Returning the result of the division
        Ok(x / y)
    }
}

fn main() {
    // Attempting division
    let dividend = 10.0;
    let divisor = 2.0;

    // Handling the result
    match divide(dividend, divisor) {
        Ok(result) => println!("Result: {}", result),
        Err(err) => println!("Error: {}", err),
    }
}
```



```
fn string_to_int(s: &str) -> Result<i32, &'static str> {
    match s.parse::<i32>() {
        Ok(n) => Ok(n),
        Err(_) => Err("Impossibile convertire la stringa in un numero intero."),
    }
}

fn main() {
    match string_to_int("mamma") {
        Ok(num) => println!("Il numero convertito è: {}", num),
        Err(message) => println!("{}", message),
    }
}
```



POLIMORFISMO

Il polimorfismo può essere realizzato attraverso due strumenti: i tratti e la programmazione generica. Questi strumenti non sono alternativi, ma complementari. Il polimorfismo permette di associare comportamenti comuni ad oggetti (tipi) diversi tra loro.

INFO: In C++ viene realizzato attraverso l'ereditarietà. È possibile definire dei metodi virtuali, la cui implementazione è diversa per ciascun tipo associato. Per accedere alle reali implementazioni occorre fare un doppio passaggio in memoria, passando dalla vtable che ne contiene il riferimento. Questo meccanismo può rallentare l'esecuzione del codice.

In Rust non esiste l'ereditarietà. Al suo posto ci sono i **tratti**, che hanno una analogia con le interfacce in Java. Questi, infatti, forniscono solo una dichiarazione dei comportamenti, che dovranno essere poi implementati dai singoli tipi. Un tratto, dunque, è una collezione di metodi che permette di definire un comportamento che un tipo può implementare.

INFO: L'uso dei tratti non comporta overhead, nella maggior parte dei casi. Un costo aggiuntivo si verifica col double pointer, quando si fa riferimento ad un tipo generico che implementa un certo tratto (&dyn SomeTrait).

Un tratto si definisce dichiarando la firma dei suoi metodi:

```
trait SomeTrait { fn someOperation(&mut self) -> SomeResult; ... }
```

Il tratto può già fornire una implementazione: in questo caso sarebbe quella di default. Invece, per implementare un tratto su un certo tipo si utilizza impl.

```
impl SomeTrait for SomeType { ... }
```

INFO: L'implementazione di un tratto può essere messa dove è dichiarato il tipo o il tratto, non in un terzo file. Il tratto, inoltre, deve essere importato col costrutto

```
use SomeNameSpace::SomeTrait;
```

A meno che non faccia parte del preludio della libreria standard, il quale viene importato automaticamente (per esempio Clone e Iter).

```
trait T1 {  
    fn returns_num() -> i32;    //ritorna un numero  
    fn returns_self() -> Self;  //restituisce un'istanza del tipo che lo implementa  
}
```

```
struct SomeType;  
impl T1 for SomeType {  
    fn returns_num() -> i32 { 1 }  
    fn returns_self() -> Self {SomeType}  
}
```

```
struct OtherType;  
impl T1 for OtherType {  
    fn returns_num() -> i32 { 2 }  
    fn returns_self() -> Self {OtherType}  
}
```

N.B. La parola chiave Self si riferisce al tipo che lo implementerà.

```
trait T2 {  
    fn takes_self(self);  
    fn takes_immut_self(&self);  
    fn takes_mut_self(&mut self);  
}
```

equivalenti

```
trait T2 {  
    fn takes_self(self: Self);  
    fn takes_immut_self(self: &Self);  
    fn takes_mut_self(self: &mut Self);  
}
```

```

trait HasArea {
    fn get_area(&self) -> f64;
}

struct Circle { radius: f64 }
impl HasArea for Circle {
    fn get_area(&self) -> f64 {
        std::f64::consts::PI * (self.radius * self.radius)
    }
}

struct Rectangle { width: f64, height: f64, }
impl HasArea for Rectangle {
    fn get_area(&self) -> f64 {
        {
            self.width * self.height
        }
    }
}

fn main() {
    let circle = Circle { radius: 3.0 };
    let rectangle = Rectangle { width: 4.0, height: 5.0 };
    println!("L'area del cerchio è: {}", circle.get_area());
    println!("L'area del rettangolo è: {}", rectangle.get_area());
}

```



INFO: Alcuni metodi non sono legati all'istanza del tipo, per cui non la prendono come parametro. Il tratto **Default**, per esempio, serve a generare una nuova istanza, assegnandole un certo valore iniziale.

```

trait Default {
    fn default() -> Self
}

```

```

fn main() {
    let zero: i32 = Default::default();
    let zero_again = i32::default();
}

```

N.B. Il linguaggio fornisce delle implementazioni standard per una serie di tratti. Per richiamarla è possibile utilizzare **#[derive(Default)]** sulla propria struttura.

```

#[derive(Default)]
struct Person {
    name: String,
    age: u32,
}

fn main() {
    // Utilizzando il valore predefinito fornito dal tratto Default
    let default_person: Person = Default::default();
    println!("Name: {}", default_person.name); // Stampa: Name:
    println!("Age: {}", default_person.age); // Stampa: Age: 0
}

```



Quando si implementa un tratto la signature deve essere la stessa, per cui, se ritorna una istanza del tipo, dovrà essere scritto "**-> Self**", per poi, invece, specificarlo all'interno del blocco del codice.

```

trait Inizializzabile {
    fn inizializza() -> Self;
}

struct Punto {
    x: i32,
    y: i32,
}

impl Inizializzabile for Punto {
    fn inizializza() -> Self {
        Punto { x: 0, y: 0 }
    }
}

fn main() {
    let punto = Punto::inizializza();
    println!("Punto inizializzato: ({}, {})", punto.x, punto.y);
}

```



INFO: Il tratto **Debug**, aggiunto attraverso **#[derive(Debug)]** permette di dettagliare il contenuto di una struttura dati ai fini della stampa.

Nell'implementazione di un tratto è possibile utilizzare dei parametri con tipi diversi, in base al tipo che lo sta implementando. Questi tipi vengono dichiarati con la keyword `type` e permettono di personalizzare ancora di più il comportamento del tratto.

```
trait T3 {
    type AssociatedType;
    fn f(arg: Self::AssociatedType);
}
```

```
struct SomeType;
impl T3 for SomeType {
    type AssociatedType = i32;
    fn f(arg: Self::AssociatedType) {}
}
```

```
struct OtherType;
impl T3 for OtherType {
    type AssociatedType = &str;
    fn f(arg: Self::AssociatedType) {}
}
```

```
fn main() {
    SomeType::f(1234);
    OtherType::f("Hello, Rust!");
}
```

```
trait Multiplicabile {
    fn moltiplica(&self, factor: i32) -> i32;
    fn moltiplica_per_due(&self) -> i32 {
        self.moltiplica(2)
    }
}
struct Numero { valore: i32 }

impl Multiplicabile for Numero {
    fn moltiplica(&self, factor: i32) -> i32 {
        self.valore * factor
    }

    fn moltiplica_per_due(&self) -> i32 {
        println!("Moltiplicazione per 2");
        self.moltiplica(2) // Ridefinizione del metodo con un comportamento diverso
    }
}

fn main() {
    let numero = Numero { valore: 5 };
    println!("Risultato: {}", numero.moltiplica_per_due());
}
```



I tratti possono essere organizzati in gerarchie di **sotto-tratti** e **super-tratti**. In particolare, la notazione

`trait Subtrait: Supertrait {...}`

Indica che il sottotratto deve implementare anche i metodi presenti nel supertratto.

```
trait Animali { fn fanno_verso(&self); }

trait Mammiferi: Animali { fn si_cibano(&self); }
trait Uccelli: Animali { fn volano(&self); }

struct Cani;
struct Crow;

impl Animali for Cani {
    fn fanno_verso(&self) { println!("Bau Bau"); }
}
impl Mammiferi for Cani {
    fn si_cibano(&self) { println!("I cuccioli di cani sono allattati."); }
}

impl Animali for Crow {
    fn fanno_verso(&self) { println!("Cra Cra"); }
}
impl Uccelli for Crow {
    fn volano(&self) { println!("I corvi volano."); }
}
```



```
fn main() {
    let dog = Cani;
    let corvo = Crow;
    dog.fanno_verso();
    dog.si_cibano();
    corvo.fanno_verso();
    corvo.volano();
}
```

N.B. I metodi del sottotratto vengono aggiunti a quelli del supertratto, per cui funzioni con lo stesso nome non vengono sostituite. Per evitare chiamate ambigue, occorre specificare il tratto di riferimento.

```
trait Supertrait {
  fn f(&self) {println!("In super");}
  fn g(&self) {}
}

trait Subtrait: Supertrait {
  fn f(&self) {println!("In sub");}
  fn h(&self) {}
}
```

```
struct SomeType;
impl Supertrait for SomeType {}
impl Subtrait for SomeType {}

fn main() {
  let s = SomeType;
  s.f(); //Errore: chiamata ambigua
  <SomeType as Supertrait>::f(&s);
  <SomeType as Subtrait>::f(&s);
}
```

L'invocazione dei metodi di un tratto può avvenire in modalità **statica**, se il tipo del valore è noto e quindi il compilatore identifica direttamente la funzione da richiamare, oppure **dinamica**, se dispone di un puntatore ad un valore di tipo non ancora noto. Nel secondo caso, il compilatore può sapere solo che il tipo implementa un certo tratto, ma per accedere alla sua implementazione dovrà usare la vtable. Questo meccanismo viene utilizzato quando si ha bisogno di lasciare più libertà all'interno del codice.

Una funzione, quindi, può prendere come parametro un riferimento a un tratto, mantenendo un certo grado di incertezza, attraverso **&dyn**.

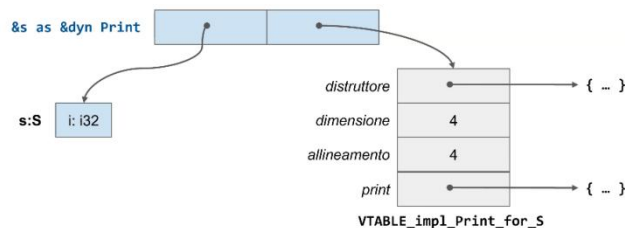
```
trait Print {
  fn print(&self);
}

struct S { i: i32 }
impl Print for S {
  fn print(&self){
    println!("S {}", self.i); }
}
```

```
fn process(v: &dyn Print){
  v.print();
}

fn main() {
  process(&S{i: 0});
}
```

Gli **oggetti-tratto** sono strutture dati che puntano a un dato e a una vtable. Sono dei riferimenti ai tipi tratto e vengono implementati attraverso un fat pointer.



```
trait Shape { fn area(&self) -> f64; }

struct Circle { radius: f64 }
impl Shape for Circle {
  fn area(&self) -> f64 { std::f64::consts::PI * self.radius * self.radius }
}

struct Rectangle {
  width: f64,
  height: f64
}
impl Shape for Rectangle {
  fn area(&self) -> f64 { self.width * self.height }
}

fn main() {
  let circle = Circle { radius: 5.0 };
  let rectangle = Rectangle { width: 3.0, height: 4.0 };

  let mut shapes: Vec<&dyn Shape> = Vec::new();
  shapes.push(&circle);
  shapes.push(&rectangle);
  for shape in shapes {
    println!("Area: {}", shape.area());
  }
}
```



Tratti standard

Nella libreria standard sono messi a disposizione alcuni tratti. Rust permette di utilizzare alcuni operatori convenzionali (per esempio `==`, `+`, `-`) e di associarli ad alcuni tratti.

```
use std::ops::Add;

struct Point {
    x: i32,
    y: i32,
}

impl Add for Point {
    type Output = Self;

    fn add(self, other: Self) -> Self::Output {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}

fn main() {
    let p1 = Point { x: 1, y: 0 };
    let p2 = Point { x: 2, y: 3 };
    let p3 = p1 + p2; // Usa l'implementazione di Add
    println!("p3.x: {}, p3.y: {}", p3.x, p3.y);
}
```



L'operatore di uguaglianza richiede che gli operandi soddisfino le proprietà di simmetria ($a=b \Rightarrow b=a$), transitività ($a=b, b=c \Rightarrow a=c$), riflessione ($a=a$).

N.B. I numeri reali (floating point) non godono della proprietà riflessiva.

Rust introduce **PartialEq**, un sottotratto di **Eq**, e si applica anche a tipi che non godono della proprietà riflessiva. **PartialEq** ha due metodi, **Eq** non ne ha nessuno (per questo è un **marker**).

I parametri presi dai tratti di uguaglianza sono **Self** e **RHS** (Right Hand Side). Normalmente coincidono, ma possono essere diversi.

N.B. Il metodo **eq** va definito per esplicitare il concetto di uguaglianza tra due valori, il metodo **ne** viene implementato automaticamente a partire da **eq**, essendone la negazione.

```
#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

impl PartialEq for Point {
    fn eq(&self, other: &Self) -> bool {
        self.x == other.x && self.y == other.y
    }
}

fn main() {
    let point1 = Point { x: 1, y: 2 };
    let point2 = Point { x: 1, y: 2 };
    let point3 = Point { x: 3, y: 4 };

    println!("point1 == point2: {}", point1 == point2); // true
    println!("point1 == point3: {}", point1 == point3); // false
}
```



INFO: I metodi di **PartialEq** possono essere derivati automaticamente, se l'uguaglianza è definita dalla corrispondenza dei campi semplici di cui è formata la variabile. Per farlo si utilizza **#[derive(partialEq)]**.

```

struct FloatingPointNumber {
    value: f64,
}

impl PartialEq for FloatingPointNumber {
    fn eq(&self, other: &Self) -> bool {
        let epsilon = 0.00001;
        let difference = (self.value - other.value).abs();
        difference <= epsilon
    }
}

fn main() {
    let a = FloatingPointNumber { value: 1.0 };
    let b = FloatingPointNumber { value: 1.01 };
    let c = FloatingPointNumber { value: 1.000000001 };

    println!("a == b: {}", a == b);
    println!("a == c: {}", a == c);
}

```



È possibile eseguire dei confronti con tipi diversi di dato. In tal caso, occorre specificarlo nella signature del metodo.

```

#[derive(Debug)]
struct Point {
    x: i32,
    y: i32,
}

impl PartialEq<i32> for Point {
    fn eq(&self, other: &i32) -> bool {
        self.x == *other && self.y == *other
    }
}

fn main() {
    let point = Point { x: 5, y: 5 };

    // Confronto tra un punto e un valore i32
    println!("point == 5: {}", point == 5); // true
}

```



Similmente, esistono anche i tratti **PartialOrd** e **Ord**.

```

enum Ordering {
    Less,
    Equal,
    Greater,
}

trait PartialOrd<Rhs = Self>: PartialEq<Rhs>
where Rhs: ?Sized, {
    fn partial_cmp(&self, other: &Rhs) ->
        Option<Ordering>;

    // metodi con implementazione di default
    fn lt(&self, other: &Rhs) -> bool;
    fn le(&self, other: &Rhs) -> bool;
    fn gt(&self, other: &Rhs) -> bool;
    fn ge(&self, other: &Rhs) -> bool;
}

```

Tipo Rhs deve essere di dimensione nota a livello di compilazione

```

trait Ord: Eq + PartialOrd<Self> {
    fn cmp(&self, other: &Self)
        -> Ordering;

    // implementazione di default
    fn max(self, other: Self) -> Self;
    fn min(self, other: Self) -> Self;
    fn clamp(self, min: Self, max: Self)
        -> Self;
}

```

N.B. Il metodo `partial_cmp` restituisce un `Option`. In questo modo è permesso non conoscere la risposta, ritornando `None`. I risultati sono forniti attraverso un `enum`, che possiede le opzioni `Less`, `Equal`, `Greater`.

```
#[derive(Debug, PartialEq)]
struct Point {
    x: f64,
    y: f64,
}
impl PartialOrd for Point {
    fn partial_cmp(&self, other: &Self) -> Option<std::cmp::Ordering> {
        if self.x < other.x && self.y < other.y {
            Some(std::cmp::Ordering::Less)
        } else if self.x > other.x && self.y > other.y {
            Some(std::cmp::Ordering::Greater)
        } else if self.x == other.x && self.y == other.y {
            Some(std::cmp::Ordering::Equal)
        } else {
            None // In caso di confronto non definito
        }
    }
}
fn main() {
    let point1 = Point { x: 1.0, y: 4.0 };
    let point2 = Point { x: 3.0, y: 2.0 };

    match point1.partial_cmp(&point2) {
        Some(ordering) => { println!("point1 è {:?} di point2", ordering); }
        None => { println!("Confronto non definito tra point1 e point2"); }
    }
}
```

partialord1.rs

Il tratto **Ord** viene usato se i due operandi sono pienamente confrontabili. Questo permette di definire i metodi max e min. In questo caso il secondo parametro deve essere di tipo Self.

```
#[derive(Debug, Eq, PartialEq, PartialOrd)]
struct Point {
    x: i32,
    y: i32,
}
impl Ord for Point {
    fn cmp(&self, other: &Self) -> std::cmp::Ordering {
        if self.x != other.x {
            self.x.cmp(&other.x)
        } else {
            self.y.cmp(&other.y)
        }
    }
}
fn main() {
    let point1 = Point { x: 3, y: 5 };
    let point2 = Point { x: 3, y: 8 };

    let max_point = point1.max(point2);
    println!("Il punto con coordinate massime è {:?}", max_point);
}
```

ord1.rs

```
use std::cmp::Ordering;

#[derive(Debug, Eq, PartialEq, PartialOrd)]
struct Person {
    name: String,
    age: i32,
}
impl Ord for Person {
    fn cmp(&self, other: &Self) -> Ordering {
        other.age.cmp(&self.age)
    }
}
fn main() {
    let john = Person { name: String::from("John"), age: 25 };
    let emma = Person { name: String::from("Emma"), age: 23 };
    let peter = Person { name: String::from("Peter"), age: 25 };

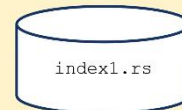
    println!("John is {:?}, compared to Emma.", john.cmp(&emma));
    // Stampa: John is Greater, compared to Emma.
    println!("John is {:?}, compared to Peter.", john.cmp(&peter));
    // Stampa: John is Equal, compared to Peter.
}
```

ord2.rs

Il metodo **drop(&mut self)** del tratto **Drop** può essere richiamato, arbitrariamente, in un certo punto del codice. Permette di liberare la memoria in anticipo. È mutuamente esclusivo con Copy per impedire il doppio rilascio della memoria.

Una struttura dati, inoltre, può essere utilizzata a mo' di array, implementando i tratti **Index** e **IndexMut**. Questi permettono di usare la notazione `t[i]` per accedere ai campi del dato, attraverso una trasformazione (da parte del compilatore) nelle espressioni `*t.index(i)` e `*t.index_mut(i)`.

```
use std::ops::Index;
struct MyVector<T> {
    data: Vec<T>,
    matricola: i32
}
impl<T> MyVector<T> {
    fn new(matr: i32) -> MyVector<T> {
        MyVector { data: Vec::new(),
                    matricola: matr }
    }
    fn push(&mut self, item: T) {self.data.push(item);}
}
impl<T> Index<usize> for MyVector<T> {
    type Output = T;
    fn index(&self, index: usize) -> &Self::Output {
        println!("Accesso at index {} => ", index);
        &self.data[index]
    }
}
fn main() {
    let mut my_vector = MyVector::new(250_000);
    my_vector.push(23);
    my_vector.push(25);
    println!("{}", my_vector[0]);
    println!("{}", my_vector[1]);
}
```



Dereferenziare un valore

Deferenziare vuol dire passare dal puntatore al dato. Questo comportamento è definito dai tratti **Deref** e **DerefMut**. Questo funzionamento, però, può essere applicato ad un'altra struttura dati che non sia un puntatore, sempre con l'obiettivo di accedere a qualcosa che si trovi al suo interno.

```
trait Deref {
    type Target: ?Sized;
    fn deref(&self) -> &Self::Target;
}

trait DerefMut: Deref {
    fn deref_mut(&mut self) -> &mut Self::Target;
}
```

L'operazione che permette di arrivare ad un valore target è detta **deref coercion**.

```
use std::ops::Deref;
#[derive(Debug)]
struct CustomStruct {
    number: i32,
    boxed_value: Box<i32>,
}

impl Deref for CustomStruct {
    type Target = i32;

    fn deref(&self) -> &Self::Target {
        &self.number
    }
}

fn main() {
    let boxed_value = Box::new(42);
    let custom_struct = CustomStruct { number: 10, boxed_value };

    let mut custom_struct_ref = &custom_struct;

    println!("Dereferencing custom_struct: {:?}", *custom_struct_ref);
    // Prints: Dereferencing custom_struct: 10
    println!("Dereferencing boxed_value: {:?}", *custom_struct_ref.boxed_value);
    // Prints: Dereferencing boxed_value: 42
}
```



INFO: Il Box applica in modo nativo la dereferenziazione.

```
fn main() {  
    let b:Box<i32> = Box::new(10);  
    println!("{}", b);  
}
```

Il **DerefMut** permette di usare l'operatore ***** per eseguire una modifica sul dato.

Conversione tra tipi

I tratti **From** e **Into** servono a effettuare una conversione di tipo. Il metodo **from** prende un parametro di tipo **T** e lo converte in **Self**, quindi è implementato sulla destinazione. Il metodo **into**, invece, viene implementato sul dato di partenza.

```
trait From<T>: Sized {  
    fn from(other: T) -> Self;  
}  
  
trait Into<T>: Sized {  
    fn into(self) -> T;  
}
```

```
struct Point {  
    x: i32,  
    y: i32,  
}  
  
impl From<(i32, i32)> for Point {  
    fn from((x, y): (i32, i32)) -> Self {  
        Point { x, y }  
    }  
}  
  
impl From<[i32; 2]> for Point {  
    fn from([x, y]: [i32; 2]) -> Self {  
        Point { x, y }  
    }  
}
```

```
fn main() {  
    // from  
    let p1 = Point::from((3, 1));  
    let p2 = Point::from([5, 2]);  
  
    // into  
    let p3: Point = (1, 3).into();  
    let p4: Point = [4, 0].into();  
}
```

from1.rs

N.B. Siccome **into** prende un **self**, il dato viene consumato.

TIPI GENERICI

Quando serve essere più generici si possono usare delle metavariable che permettono di generalizzare il tipo di una variabile. Le metavariable si indicano con le parentesi angolari, scrivendo **<T>**. Il tipo vero e proprio viene specificato dal chiamante della funzione che lo usa.

La clausola **where** permette di imporre un vincolo sul tipo utilizzato, per esempio per indicare un tratto che deve soddisfare.

```
fn max<T>(  
    t1: T,  
    t2: T) -> T where T: Ord {  
    return  
        if t1 < t2 { t2 }  
        else { t1 }  
}
```

Rust

Rust applica la regola della monomorfizzazione: ogni implementazione genera un eseguibile per uno specifico tipo. Il codice macchina risulta quindi replicato, ma l'implementazione è ottimizzata.

I tipi generici possono trovarsi anche nelle struct e nei loro metodi. Le metavariables possono essere indicate con una qualsiasi lettera.

```
struct MyStruct<T> where T: SomeTrait {  
    foo: T,  
    //...  
}
```

```
impl<T> struct MyStruct<T>  
    where T: SomeTrait {  
    fn process(&self) { ... }  
}
```

Il vincolo sui tratti prende il nome di **Trait Bound** e può essere espresso in una versione compatta (direttamente nelle parentesi angolari) o estesa (col where).

```
fn run_query<M: Mapper + Serialize, R: Reducer + Serialize>(  
    data: &DataSet, map: M, reduce: R) -> Results  
{ ... }
```

```
fn run_query<M,R>(< data: &DataSet, map: M, reduce: R) -> Results  
    where M: Mapper + Serialize,  
           R: Reducer + Serialize  
{ ... }
```

Tratti e tipi generici

Per ottenere il polimorfismo si possono usare sia i tratti che i tipi generici. L'utilizzo degli oggetti-tratto (&dyn) permette di definire il tratto di una variabile, comportando la scrittura, da parte del compilatore, di una sola funzione. I tipi generici, invece, generano tante copie di codice quante sono le combinazioni di tipi impiegati.

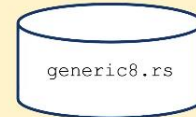
Lo svantaggio degli oggetti-tratto, però, è l'impiego del doppio puntatore, che non si verifica nel caso dell'uso dei tipi generici. Questi, inoltre, permettono di ottenere un codice ottimizzato sui tipi usati.

```
trait Suona {  
    fn suona(&self);  
}  
struct Chitarra;  
struct Pianoforte;  
impl Suona for Chitarra {  
    fn suona(&self) {  
        println!("Chitarra: Strum!");  
    }  
}  
impl Suona for Pianoforte {  
    fn suona(&self) {  
        println!("Pianoforte: Ding!");  
    }  
}  
fn esegui_melodia(strumento: &dyn Suona) { strumento.suona(); }  
fn main() {  
    let chitarra = Chitarra;  
    let pianoforte = Pianoforte;  
    esegui_melodia(&chitarra);  
    esegui_melodia(&pianoforte);  
}
```



N.B. L'oggetto-tratto, generalmente, è più svantaggioso rispetto all'uso dei tipi generici. Può essere usato per i contesti in cui si ha a disposizione una memoria ridotta.

```
trait Suona { fn suona(&self); }
struct Chitarra;
struct Pianoforte;
impl Suona for Chitarra {
    fn suona(&self) {
        println!("Chitarra: Strum!");
    }
}
impl Suona for Pianoforte {
    fn suona(&self) {
        println!("Pianoforte: Ding!");
    }
}
fn esegui_melodia<T: Suona>(strumento: & T) { strumento.suona(); }
fn main() {
    let chitarra = Chitarra;
    let pianoforte = Pianoforte;
    esegui_melodia(&chitarra);
    esegui_melodia(&pianoforte);
}
```



LIFETIME

Ci sono delle situazioni in cui il compilatore non riesce a ricavare in modo chiaro il tempo di vita di un riferimento, per esempio quando viene dato in uscita da una funzione. In questi casi viene chiesto al programmatore di esplicitare il lifetime della variabile, che permetta di chiarire il legame tra input e output.

Uno dei casi più frequenti è legato alle funzioni che prendono in ingresso dei riferimenti e restituiscono un nuovo riferimento, estratto da una delle strutture ricevute. Il legame fra i riferimenti va espresso attraverso le etichette (indicate con ') al fine di eliminare ogni ambiguità.

N.B. Se il tempo di vita del risultato dipende da più parametri in ingresso, tutti questi vengono marcati con la stessa label. Questo non vuol dire che hanno tutti lo stesso lifetime, ma che deve essere presa l'intersezione dei tempi (quindi il caso pessimistico, il più breve).

```
fn f<'a>(p1: &'a i32, p2:&'a i32) -> &'a i32 { ... }
fn f<'a, 'b>(p1: &'a i32, p2:&'b i32) -> &'a i32 { ... }
```

Questo problema si verifica ogni volta che una funzione prende in ingresso più di un riferimento e ne restituisce uno nuovo: occorre quindi ricordarsi di definire le label per garantire disambiguità.

```
fn confronta(str1: &str, str2: &str) -> &str {
    if str1.len() > str2.len() { str1 } else { str2 }
}
fn main() {
    let s1 = String::from("hello");
    let s2 = String::from("world");
    let risultato;
    risultato = confronta(&s1, &s2);

    println!("La stringa più lunga è: {}", risultato);
}
```



```
1 | fn confronta(str1: &str, str2: &str) -> &str {
  |                                     ^ expected named lifetime parameter
= help: this function's return type contains a borrowed value, but the
signature does not say whether it is borrowed from `str1` or `str2`
help: consider introducing a named lifetime parameter
1 | fn confronta<'a>(str1: &'a str, str2: &'a str) -> &'a str {
  |             ++++      ++          ++          ++
```

Se l'output dipende da un solo parametro, allora va specificata solamente l'etichetta a cui è legato.

```
fn stampa<'a> (str1: &'a str, str2: &str) -> &'a str {
    println!("{}",str2);
    str1
}
fn main() {
    let s1 = String::from("Viva i lifetimes");
    let s2 = String::from("Questa è una stringa di benvenuto");

    let risultato;
    risultato = stampa(&s1, &s2);

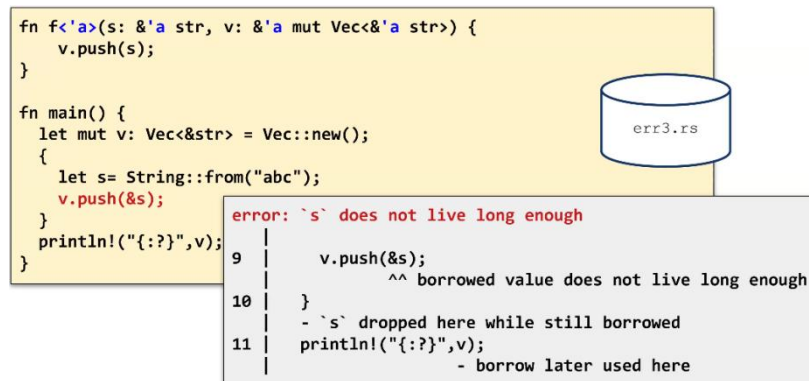
    println!("{}", risultato);
}
```



INFO: Se una signature contiene sia etichette che metavariables, vanno messe le prime e poi le seconde, separate da una virgola.

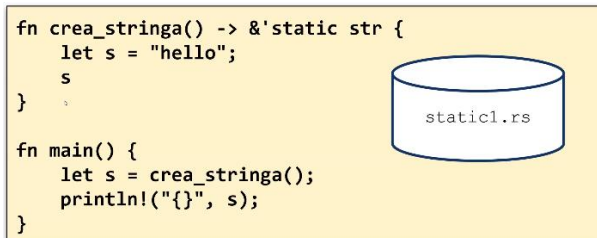
< 'a , T >

Un altro problema che può verificarsi riguarda i riferimenti contenuti in una struttura. Il cui tempo di vita della struct, infatti, deve essere incluso in quello del riferimento.



Nell'esempio, poiché va considerato il caso peggiore, il compilatore restituisce un errore. Questo accade perché il campo contenuto nel vettore cessa di esistere prima dello stesso vettore.

INFO: Tutte le stringhe statiche restituite da una funzione vanno esplicitate con 'static.



Elisione dei tempi di vita

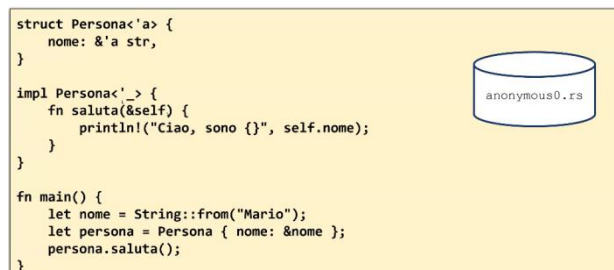
Il compilatore applica tre regole per assegnare, ad un riferimento presente in una struttura o una funzione, un tempo di vita distinto. Questo permette, così, di evitare al programmatore di doverli definire esplicitamente.

- 1) Ogni parametro con riferimento ha un proprio tempo di vita;
- 2) Se c'è un solo parametro di tempo di vita in input, quello stesso tempo di vita è propagato a tutti gli output;
- 3) Se ci sono più parametri di lifetime in ingresso ma uno di loro è &self o &mut self, allora il parametro di lifetime di self è assegnato a tutti gli output.

Le ultime due permettono, quindi, di far avanzare delle ipotesi al compilatore, che è in grado così di prendere una decisione sui tempi di vita. Questo, però, non garantisce la correttezza della decisione.

Anonymus Lifetime

Oltre a quelli citati sopra, esiste anche un altro caso in cui non occorre specificare i tempi di vita, perché il compilatore sa dedurli da sé. Questo caso si verifica quando in una struttura dati c'è un riferimento ma non ci sono ambiguità sul tempo di vita di uscita dei suoi metodi. La notazione è <'>.



CHIUSURE

La programmazione funzionale introduce il concetto di funzioni di ordine superiore, dove una funzione può restituire, o prendere come parametro, un'altra funzione.

INFO: Anche negli altri linguaggi è possibile utilizzare dei puntatori a funzione. Non è quindi una caratteristica speciale di Rust.

I puntatori a funzione possono essere dichiarati e inizializzati.

```
fn f1(i: i32, d: f64) -> f64 { Rust
    return i as f64 * d;
}

let ptr: fn(i32, f64) -> f64;

ptr = f1; //assegno il puntatore

let num = ptr(2, 3.14);
// chiamo la funzione
```

```
fn add(a: i32, b: i32) -> i32 { a + b }
fn subtract(a: i32, b: i32) -> i32 { a - b }
fn multiply(a: i32, b: i32) -> i32 { a * b }

fn main() {
    let (x, y) = (10, 5);

    // Selezioniamo la funzione in base a una condizione.
    let operation_name = "add";

    // Assegniamo il puntatore alla funzione corrispondente.
    let operation: fn(i32, i32) -> i32 = match operation_name {
        "add" => add,
        "subtract" => subtract,
        "multiply" => multiply,
        _ => panic!("Operazione non supportata"),
    };

    let result = operation(x, y);

    println!("Il risultato dell'operazione '{}' è: {}", operation_name, result);
}
```



```
fn add_one(x: i32) -> i32 {
    x + 1
}

fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {
    f(arg) + f(arg)
}

fn main() {
    let answer = do_twice(add_one, 5);
    println!("The answer is: {}", answer);
}
```



FUNZIONI LAMBDA

Una funzione lambda è una funzione anonima costituita da un blocco di codice espresso in forma letterale. Queste funzioni non sono dichiarate attraverso una signature e non vengono introdotte dalla parola chiave fn. Possono inoltre essere assegnate a delle variabili e invocabili utilizzando queste ultime.

INFO: Le funzioni lambda e le chiusure possono essere considerate equivalenti.

Le funzioni lambda sono in grado di catturare delle variabili libere dall'ambiente circostante, sia per riferimento (di default) che per movimento (con la kw move). Proprio per questa capacità di conservare uno stato, la stessa funzione, chiamata due volte con gli stessi parametri, può dare risultati diversi.

```
fn main() {
    let add = |num1, num2| num1 + num2;
    println!("La somma è: {}", add(1, 2));
}
```

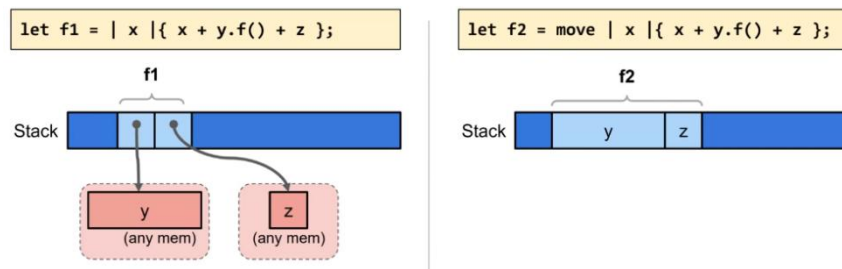
```
fn main() {
    let factor = 2;
    let multiply = |n| n * factor;
    println!("Il risultato è: {}", multiply(5));
}
```

Variabile libera
catturata

```
fn main() {
    let multiply = |x: i32, y: i32| -> i64 {
        (x * y).into() };
    println!("{}", multiply(3, 4)); // prints 12
}
```

Una variabile che fa riferimento ad una chiusura viene memorizzata come una struttura che contiene sia il codice che le variabili libere (riferimenti o cattura). Le variabili libere sono tutte quelle variabili che vengono utilizzate all'interno della funzione, ma sono state definite all'esterno e non sono state passate tra i parametri.

N.B. Se viene anteposta la parola **move** alla funzione lambda, le variabili libere utilizzate vengono prelevate e inserite al suo interno, nel blocco di memoria della variabile in cui è salvata la funzione.



La cessione di una variabile segue le regole solite di Rust, per cui se lascio un riferimento mutabile ad una lambda, non posso accedere al suo valore (all'esterno) fin quando non smetto di usare quella funzione.

```
fn main() {
    let mut count = 0;

    let mut increment_n = |n| {
        count += n; // Incrementiamo la variabile catturata
        println!("Il conteggio è: {}", count);
    };

    increment_n(10);
    println!("{}", count);
    increment_n(5);
    println!("{}", count);
}
```

```
increment_n(10);
increment_n(5);
println!("{}", count);
```

Il compilatore non compila!
Come mai?

Questo lo compila!

TRATTI FUNZIONALI

I tratti funzionali sono specifici per le chiusure e sono di tre tipi.

Il tratto **FnOnce** permette di usare la funzione una sola volta, perché consuma le variabili catturate. Il tratto **FnMut** permette di modificare i dati contenuti nelle variabili libere. Il tratto **Fn** è il più generico, permette di chiamare la funzione senza richiedere modifiche.

INFO: Il compilatore è in grado di capire quale tratto applicare alla chiusura, sulla base delle operazioni contenute all'interno.

Esempio di Fn (può essere richiamata più volte, non modifica alcun parametro libero):

```
fn main() {
    let greeting = "Ciao";

    // Definiamo una closure che non modifica variabili esterne
    let greet = || {
        println!("{}", greeting);
    };

    // Chiamiamo la closure
    greet();
    greet();
    greet();
}
```



In questo esempio la funzione può essere chiamata più volte perché la variabile non viene consumata, ma solo posseduta dalla lambda:

```
fn main() {
    let data = vec![1, 2, 3, 4, 5];

    // Definiamo una closure che può essere chiamata solo una volta
    let process_data = move || {
        println!("Elaborazione dei dati in corso...");
        let sum: i32 = data.iter().sum();
        println!("La somma dei dati è: {}", sum);
    };

    // Chiamiamo la closure
    process_data();

    process_data();
}
```



Esempio di FnMut (viene modificata una variabile esterna):

```
fn main() {
    let mut counter = 0;

    // Definiamo una closure che può essere chiamata più volte
    let mut increment_counter = || {
        counter += 1;
        println!("Il contatore è ora: {}", counter);
    };

    // Chiamiamo la closure più volte
    increment_counter();
    increment_counter();
    increment_counter();
}
```



Esempio di FnOnce (in questo esempio il vettore viene distrutto nel ciclo, perché i singoli elementi non vengono utilizzati con un riferimento):

```
fn main() {
    let vec = vec![1, 2, 3, 4, 5];

    // Definiamo una chiusura che prende il possesso del vettore vec
    let consume_vector = move || {
        // Consumiamo il vettore stampando tutti i suoi elementi
        for num in vec {
            println!("{}", num);
        }
    };

    // Chiamiamo la chiusura. Dopo questa chiamata, non possiamo più usarla.
    consume_vector();

    // Tentare di chiamare nuovamente la chiusura genererebbe un errore di
    // compilazione.
    // consume_vector();
}
```



Funzioni di ordine superiore

Le funzioni di ordine superiore accettano come parametro una lambda e tramite la programmazione generica è possibile richiedere in ingresso una funzione che implementi uno dei tratti funzionali.

```
fn consume_closure<F>(f: F)
    where
        F: FnOnce() -> String,
{
    println!("La closure dice: {}", f());
}

fn main() {
    let text = "Hello, world!".to_string();

    let printer = || text;

    consume_closure(printer);

    // La seguente linea sarebbe errore se decommentata,
    // perché printer non può essere usata più di una volta.
    // consume_closure(printer);
}
```



```
fn apply_function<F> (input: i32, function: F) -> i32
    where
        F: Fn(i32) -> i32,
{
    function(input)
}

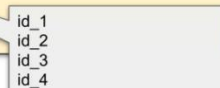
fn main() {
    let doubled = apply_function(10, |n| n * 2);
    println!("{}", doubled);
}
```



È possibile restituire una chiusura, in tal caso nella signature occorrerà indicare il tratto funzionale, preceduto da impl, e il tipo restituito.

```
fn generator(prefix: &str) -> impl FnMut() -> String {
    let mut i = 0;
    let b = prefix.to_string();
    return move || {i+=1; format!("{}",b,i)}
}

fn main() {
    let mut f = generator("id_");
    for _ in 1..5 {
        println!("{}",f());
    }
}
```



INFO: Si può imporre `FnOnce` nel caso in cui si volesse permettere una volta sola l'esecuzione della funzione di ritorno.

```
fn crea_messaggio(s: String) -> impl FnOnce() -> String {  
    move || s  
}  
  
fn main() {  
    let messaggio = crea_messaggio("Ciao Rust!".to_string());  
    println!("{}", messaggio());  
}
```

higherorderfunction7.rs

GESTIONE DEGLI ERRORI

La gestione degli errori, in Rust, è permessa attraverso il tipo algebrico **Result**.

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

Per discriminare il risultato di una funzione, si utilizza il `match`. In caso di errore, si esegue un `return`. Nella signature è possibile esplicitare il tipo di errore che può essere inserito nella `Result`, come ad esempio `io::Error`.

```
fn read_file(name: &str) -> Result<String, io::Error> {  
    let r1 = File::open(name);  
    let mut file = match r1 {  
        Err(why) => return Err(why),  
        Ok(file) => file,  
    };  
    let mut s = String::new();  
    let r2 = file.read_to_string(&mut s);  
    match (r2) {  
        Err(why) => Err(why),  
        Ok(_) => Ok(s),  
    }  
}
```

readfile.rs

Oltre al `match`, esistono alcuni metodi specifici che permettono di analizzare il contenuto di una `Result`. In particolare, `is_ok(&self)` e `is_err(&self)` permettono di determinare il successo o insuccesso di un processo. I metodi `ok(self)` e `err(self)` consumano il risultato e lo trasformano in un oggetto di tipo `Option<T>` o `Option<E>` (rispettivamente possiedono il dato/errore oppure un `None`).

Il metodo `map(self, op: F) -> Result<U, E>` permette di applicare una funzione sul risultato, oppure di far passare invariato l'errore.

Il metodo `unwrap(self) -> T` restituisce il contenuto ma lancia un `panic!` se contiene un errore.

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {  
    if y == 0.0 {  
        Err("Impossibile dividere per zero")  
    } else {  
        Ok(x / y)  
    }  
}
```

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {
    . . .
}

fn main() {
    let dividend = 10.0;
    let divisor = 0.0;

    let result = divide(dividend, divisor);

    if result.is_ok() {
        println!("Il risultato della divisione è: {}", result.unwrap());
    } else {
        println!("Errore: {}", result.unwrap_err());
    }
}
```



```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {
    . . .
}

fn main() {
    let dividend = 10.0;
    let divisor_success = 2.0;
    let divisor_error = 0.0;

    let result_success = divide(dividend, divisor_success);
    let result_error = divide(dividend, divisor_error);
    println!("10 / 2");
    match result_success.ok() {
        Some(value) => println!("Il risultato della divisione è: {}", value),
        None => println!("Nessun risultato presente"),
    }
    match result_success.err() {
        Some(message) => println!("Errore: {}", message),
        None => println!("Nessun errore presente"),
    }
    println!("10 / 0");
    match result_error.ok() {
        Some(value) => println!("Il risultato della divisione è: {}", value),
        None => println!("Nessun risultato presente"),
    }
    match result_error.err() {
        Some(error_message) => println!("Errore: {}", error_message),
        None => println!("Nessun errore presente"),
    }
}
```



INFO: La macro **panic!** si occupa di liberare tutte le variabili allocate. Può anche essere richiamato esplicitamente, passandogli una stringa con un messaggio.

Il metodo **expect()** si “aspetta” un risultato corretto, in caso contrario lancia un panic. A differenza dell’**unwrap** permette di aggiungere un messaggio.

```
fn divide(x: f64, y: f64) -> Result<f64, &'static str> {
    . . .
}

fn main() {
    let dividend = 10.0;
    let divisor = 0.0;

    let result = divide(dividend, divisor);

    // Utilizzando expect() per ottenere il valore
    // o stampare un messaggio di errore personalizzato

    let quotient = result.expect("ATTENZIONE, ERRORE NEI DATI");
    println!("Il risultato della divisione è: {}", quotient);
}
```



Per semplificare la sintassi della gestione degli errori, Rust fornisce l'operatore `?` che può essere utilizzato sulle espressioni che generano un tipo `Result`. Questo permette di avere una replica del valore di ritorno, con la condizione che il tipo dell'errore sia lo stesso. Si tratta di una **syntax sugar**, cioè permette di gestire più facilmente il valore senza alterare il funzionamento del codice e delegando al compilatore la scrittura dell'`if/match`.

N.B. In caso di errore, dunque, questo viene semplicemente propagato oltre. Per questo è necessaria una corrispondenza tra i tipi dell'errore nella funzione chiamata e in quella chiamante.

```
fn read_file_contents(filename: &str) -> Result<String, io::Error> {
    let mut file = File::open(filename)? ;
    // Apre il file e gestisce gli errori con l'operatore ?
    let mut contents = String::new();
    file.read_to_string(&mut contents)? ;
    // Legge il contenuto del file e gestisce gli errori con l'operatore ?
    Ok(contents) // Restituisce il contenuto del file se tutto è andato bene
}

fn main() {
    match read_file_contents("prova.txt") {
        Ok(contents) => println!("Contenuto del file: {}", contents),
        Err(err) => println!("Errore: {}", err),
    }
}
```



INFO: L'operatore funziona anche con il tipo `Option`. Se viene invocata una funzione che ritorna un `Result`, il risultato NON viene convertito automaticamente in `Option`.

```
fn get_first_element(numbers: Vec<i32>) -> Option<i32> {
    let first = numbers.first()? ;
    Some(*first)
}
```

Poiché dichiarare il tipo di errore è un vincolo particolarmente limitante, è possibile ampliare il dominio di casi possibili includendo un tipo di errore generico. Questo è possibile attraverso **dyn Error**. Siccome può essere di tipi diversi, l'errore viene messo in un `Box`, per poterlo inserire nello heap.

```
use std::error::Error;

fn divide(x: f64, y: f64) -> Result<f64, String> {
    if y == 0.0 {
        return Err(String::from("Divisione per zero"));
    }
    Ok(x / y)
}

fn main() -> Result<(), Box<dyn Error>> {
    let result = divide(10.0, 5.0)? ;
    println!("Il risultato della divisione è: {}", result);

    let result2 = divide(10.0, 0.0)? ;
    println!("Il risultato della divisione è: {}", result2);

    Ok(())
}
```

Lo si può leggere come:
"ogni tipo di possibile errore"



In questo caso si deve usare un oggetto-tratto che implementa il tratto **Error**, che deve essere condiviso tra tutti.

Il chiamante può discriminare tra i vari tipi attraverso il metodo **downcast_ref::<TipoErrore>()**

```
fn sum_file(path: &Path) -> Result<i32, Box<dyn error::Error>> {
    let mut file = File::open(path) ? ; // io::Error -> Box<dyn error::Error>
    let mut contents = String::new();
    file.read_to_string(&mut contents) ? ; // io::Error -> Box<dyn error::Error>
    let mut sum = 0;
    for line in contents.lines() {
        sum += line.parse::<i32>() ? ; // ParseIntError -> Box<dyn error::Error>
    }
    Ok(sum)
}

fn handle_sum_file_errors(path: &Path) {
    match sum_file(path) {
        Ok(sum) => println!("sum is {}", sum),
        Err(err) => {
            if let Some(e) = err.downcast_ref::<io::Error>()
            {
                //tratto io::Error
                println!("Errore di I/O nella gestione del file: {}", e);
            }
            else if let Some(e) = err.downcast_ref::<ParseIntError>()
            {
                //tratto ParseIntError
                println!("Errore nell'elaborazione del file: {}", e);
            }
            else { unreachable!(); } //non può capitare
        }
    }
}
```



Per propagare errori eterogenei è possibile implementare dei tipi custom. Per esempio è possibile definire un enum che comprende più tipi di errore.

```
#[derive(Debug)]
enum SumFileError {
    Io(io::Error),
    Parse(ParseIntError),
}
```

INFO: Questo è possibile ma richiede alcune operazioni laboriose, per cui ogni variante deve implementare i tratti Error, Debug, Display e From

INFO: Esiste un crate, denominato **thiserror**, che fornisce supporto per la creazione di tipi che implementano il tratto Error.

```
[dependencies]
thiserror = "1.0"
```

```
#[derive(Error, Debug)]
enum SumFileError {

    #[error("IO error {0}")]
    Io(#[from] io::Error),

    #[error("Parse error {0}")]
    Parse(#[from] ParseIntError),
}
```

```
#[derive(Error, Debug)]
pub enum SumFileError {
    #[error("IO Error {0}")]
    Io(#[from] io::Error),
    #[error("Parse Error {0}")]
    Parse(#[from] ParseIntError),
}

fn sum_file(path: &Path) -> Result<i32, SumFileError> {
    let mut file = File::open(path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let mut sum = 0;
    for line in contents.lines() { sum += line.parse::<i32>()?; }
    Ok(sum)
}
```



ITERATORI

Gli iteratori sono degli strumenti di scansione di sequenze di valori, contenuti all'interno di contenitori e accessibili, ad esempio, tramite un riferimento. Un iteratore ha, tipicamente, un modo per verificare se sono presenti ulteriori valori da generare (`hasNext()->bool`) ed un modo per accedere all'effettivo valore seguente (`next()->E`). In Rust queste due cose sono combinate sotto un solo metodo, **`next()`**, che restituisce un `Option`.

L'uso degli iteratori si sovrappone a quello dei cicli `for/whil`, ma offre svariati vantaggi in termini di compattezza e leggibilità del codice.

```
let v1 = vec![1,2,3,4,5,6];
let mut v2 = Vec::<String>::new();

for i in 0..v1.len() {
    if v1[i] %2 != 0 { continue; }
    v2.push(format!("{}",v1[i]))
}
println!("{:?}", v2); // ["a2","a4","a6"]
```

```
let v1 = vec![1,2,3,4,5,6];
let mut v2: Vec<String>;

v2 = v1.iter()
    .filter(|val|{ *val %2 == 0})
    .map(|val| format!("{}",val))
    .collect();

println!("{:?}", v2); // ["a2","a4","a6"]
```

Gli iteratori operano in modalità **lazy**: questo vuol dire che si predispongono ad acquisire/generare i dati, ma lo fanno solo nel momento in cui c'è l'effettiva richiesta del valore successivo. È quindi il consumatore finale che invoca il `next`, instaurando il meccanismo per il quale l'iteratore fornisce, via via, i dati a chi li ha richiesti.

Un iteratore è una qualsiasi struttura dati che implementa il tratto **Iterator**. Questo fornisce alcuni metodi, ma il più importante è `next()`, che è pensato per restituire un `Option` contenente il dato successivo.

```
trait Iterator{
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    ...// molti altri metodi con implementazione di default
}
```

A partire da una struttura dati (contenitore) che implementa il tratto `Iterator`, è possibile ricavare un iteratore sui suoi dati. Questo è possibile tramite i metodi **`iter()`**, che non consuma il contenuto del contenitore, **`iter_mut()`**, che permette di modificare i valori, **`into_iter()`**, che prende il possesso della struttura e la consuma, estraendo e restituendo i valori contenuti.

N.B. L'iter, di per sè, non fa nulla, mette solo a disposizione i dati. Per poterli processare bisogna usare, per esempio, un ciclo `for` oppure combinarli con altre funzioni.

Le due opzioni di questo esempio sono equivalenti:

```
fn main() {
    let numbers = [1, 2, 3, 4, 5];

    // Itera attraverso ciascun elemento dell'array e stampalo

    for num in numbers.iter() {
        println!("{}", num);
    }

    for num in &numbers {
        println!("{}", num);
    }
}
```



La funzione di iterazione `into_iter()` consuma i dati che estrae, allo stesso modo di un ciclo `for` che accede ai valori di un vettore senza usare dei riferimenti.

```
fn main() {

    let v = vec![10.0, 30.0, 50.0, 90.0];

    let mut sum=f64::default();

    // into_iter consuma il contenitore
    for num in v.into_iter() {
        sum += num;
    }
    println!("{}", sum);

    let v2 = vec![1, 3, 5, 9];

    let mut sum2=i32::default();

    for num2 in v2 {
        sum2 += num2;
    }

    println!("{}", sum2);
}
```



Si può definire una propria idea di iterazione su una struttura, attraverso la scrittura del metodo `next` nel tratto `Iterator`. Nell'esempio, si crea una sorta di range, in cui il `next` ritorna un numero partendo da `count` e arrivando a `max` (nella struttura `Contatore`).

```
// Definiamo una struttura che implementerà il tratto Iterator
struct Contatore {
    count: usize,
    max: usize,
}
// Implementiamo il tratto Iterator
impl Iterator for Contatore {
    type Item = usize;
    fn next(&mut self) -> Option<Self::Item> {
        if self.count < self.max {
            let val = self.count;
            self.count += 1;
            Some(val)
        } else {
            None
        }
    }
}

fn main() {
    let mut contatore = Contatore { count: 0, max: 10 };

    // Usiamo l'iteratore per stampare i valori
    while let Some(i) = contatore.next() {
        println!("{}", i);
    }
}
```

```
trait Iterator{
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    ...
}
```



INFO: Il ciclo `for` su una struttura che implementa il tratto `Iterator`, usa implicitamente la funzione `next`.

```
struct ContatoreFibonacci {
    a: usize,
    b: usize,
    max: usize,
}
impl Iterator for ContatoreFibonacci {
    type Item = usize;
    // Definiamo il metodo next per ottenere il prossimo elemento
    fn next(&mut self) -> Option<Self::Item> {
        if self.a < self.max {
            let valore = self.a;
            let nuovo_valore = self.a + self.b;
            self.a = self.b;
            self.b = nuovo_valore;
            Some(valore)
        } else {
            None
        }
    }
}

fn main() {
    let fibonacci_iteratore = ContatoreFibonacci { a: 0, b: 1, max:1000 };
    for numero in fibonacci_iteratore {
        println!("Fibonacci: {}", numero);
    }
}
```



Un tipo è iterabile se può essere scandito con un iteratore. Per far ciò deve implementare il tratto **IntoIterator**, che necessita della definizione del metodo `into_iter`.

```
trait IntoIterator where Self::IntoIter: Iterator<Item=Self::Item> {
    type Item;
    type IntoIter: Iterator;
    fn into_iter(self) -> Self::IntoIter;
}
```

In questo caso la struttura di partenza viene trasformata per essere iterabile, per esempio i campi di una struct possono essere convertiti in un array.

```
struct Pixel {
    r: i8,
    g: i8,
    b: i8,
}
impl IntoIterator for Pixel {
    type Item = i8;
    type IntoIter = std::array::IntoIter<i8, 3>;
    fn into_iter(self) -> Self::IntoIter {
        std::array::IntoIter::new([self.r, self.g, self.b])
    }
}

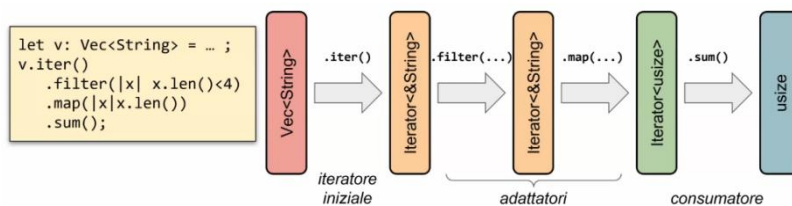
fn main() {
    let pixel = Pixel { r: 54, g: 23, b: 74 };
    let mut iter = pixel.into_iter();
    // Chiamare next restituirà il prossimo elemento dell'iteratore, se presente
    if let Some(component) = iter.next() {
        println!("Il primo componente è: {}", component);
    }
    // Puoi continuare a chiamare next per ottenere i successivi elementi
    if let Some(component) = iter.next() {
        println!("Il secondo componente è: {}", component);
    }
    if let Some(component) = iter.next() {
        println!("Il terzo componente è: {}", component);
    }
    // Se chiami next dopo che tutti gli elementi sono stati consumati, otterrai None
    if let Some(component) = iter.next() {
        println!("Questo non verrà stampato perché non ci sono più elementi.");
    } else {
        println!("Non ci sono più componenti.");
    }
}
```

```
trait IntoIterator where Self::IntoIter: Iterator<Item=Self::Item> {
    type Item;
    type IntoIter: Iterator;
    fn into_iter(self) -> Self::IntoIter;
}
```

intoiterator1.rs

Adapter

Gli iteratori vengono utilizzati da due tipologie di funzioni: gli **adapter**, che ricevono gli items e generano un nuovo iteratore, e i **consumer**, che consumano un iteratore per produrre il loro risultato.



La funzione **map** permette, attraverso una closure, di eseguire una certa operazione su ogni elemento di un iteratore. Ritorna, di volta in volta, il nuovo elemento, andando a generare un nuovo iteratore.

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5];

    // Creiamo un iteratore dal vettore
    // applichiamo il metodo map per raddoppiare ogni elemento

    let doubled_iter = numbers.iter().map(|&x| x * 2);

    // Stampiamo i risultati
    for doubled in doubled_iter {
        println!("{}", doubled);
    }
}
```

map1.rs

Il metodo **filter()** include esclusivamente gli elementi che soddisfano una condizione, fornita attraverso una chiusura.

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5, 6, 7, 8, 9];

    // Utilizzo di filtri per selezionare solo i numeri pari
    let even_numbers = numbers.iter().filter(|&x| x % 2 == 0);

    // Stampiamo i numeri pari
    for n in even_numbers {
        println!("Even number: {}", n);
    }
}
```

filter1.rs

Il metodo **filter_map()** ritorna gli elementi processati, sono se non valgono None.

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5];

    // Utilizzo di filter_map per filtrare e mappare i numeri pari
    let even_numbers = numbers
        .iter()
        .filter_map(|&x| {
            if x % 2 == 0 {
                Some(x)
            } else {
                None
            }
        });

    for n in even_numbers {
        println!("{}", n);
    } // Stampa: 2, 4
}
```

filtermap1.rs

La funzione **take(n)** permette di prendere i primi n elementi e di inserirli in un nuovo iteratore.

```
fn main() {
    let numbers = vec![10, 20, 30, 40, 50];

    let first_two_over_twenty = numbers.iter()
        .take(3)
        .filter(|&num| *num > 10);

    for n in first_two_over_twenty {
        println!("Numeri maggiori di 10 tra i primi 3: {:?}", n);
    }
}
```

take3.rs

La funzione **take_while()** estrae valori fin quando non si verifica una certa condizione.

```
fn main() {
    let numbers = vec![5, 10, 15, 20, 22, 30];

    // Prende numeri finché sono multipli di 5
    let multiples_of_five = numbers.iter()
        .take_while(|&num| *num % 5 == 0);

    for n in multiples_of_five {
        println!("Multipli di 5: {:?}", n);
    }
}
```

takewhile.rs

N.B. Quando si prende il possesso di un contenitore (che sia attraverso `into_iter()` oppure con un ciclo che non usa dei riferimenti), questo viene consumato anche se non si è acceduto a tutti gli elementi. Il ciclo `for` in questo esempio consuma il vettore anche se non lo ha letto tutto, per cui viene impedita la stampa (commentata) anche dei valori che non sono stati processati.

```
fn main() {
    let numbers = vec![2, 3, 0, 5, 8, 10, 9, 7];

    let mut count = 0;
    for num in numbers.iter() {
        println!("{}", num);
        if *num > 5 {break};
    }
    println!("{:?}", numbers);

    for num in numbers {
        println!("{}", num);
        if num > 5 {break};
    }
    // println!("{:?}", numbers);
    // for num in numbers.into_iter()
    // {
    //     println!("{}", num);
    //     if num > 5 {break};
    // }

    // equivalente a
    // let n = numbers.into_iter()
    //     .take_while(|num| *num > 5);
}
```



Il metodo **enumerate()** fornisce una sequenza di tuple contenenti indice e valore per ogni elemento contenuto nell'iteratore di partenza.

```
fn main() {
    let my_vec = vec!["a", "b", "c"];

    // Otteniamo un iteratore che produce coppie (indice, valore)
    let mut iter = my_vec.iter().enumerate();

    // Iteriamo attraverso ogni coppia (indice, valore)
    while let Some((index, value)) = iter.next() {
        println!("Indice: {}, Valore: {}", index, value);
    }
}
```

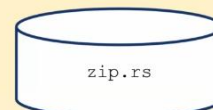


La funzione **zip()** combina due iteratori producendo una sequenza di tuple, contenenti un valore a testa. La funzione **unzip()** esegue il meccanismo inverso.

```
fn main() {
    let numbers = vec![1, 2, 3];
    let letters = vec!['a', 'b', 'c'];

    // Otteniamo un iteratore che produce tuple (numero, lettera)
    let mut iter = numbers.iter().zip(&letters);

    // Iteriamo attraverso ogni coppia (numero, lettera)
    while let Some((number, letter)) = iter.next() {
        println!("Numero: {}, Lettera: {}", number, letter);
    }
}
```



La funzione **cloned()** prende un iteratore che produce riferimenti e restituisce un iteratore con i valori clonati da questi riferimenti.

```
fn main() {
    let words = vec!["hello", "world", "rust"];

    // Otteniamo un iteratore che produce copie delle stringhe
    let cloned_iter = words.iter().cloned();

    // Stampiamo ogni parola nel nuovo iteratore
    for word in cloned_iter {
        println!("Parola: {}", word);
    }
    for word in words {
        println!("Parola: {}", word);
    }
}
```



La funzione **copied()** genera un iteratore che produce una copia dei valori originali.

```
fn main() {
    let tuple_vec = vec![(1, 'a'), (2, 'b'), (3, 'c')];

    // Otteniamo un iteratore che produce copie delle tuple originali
    let copied_iter = tuple_vec.iter().copied();

    // Stampiamo ogni tupla nel nuovo iteratore
    for tuple in copied_iter {
        println!("Tuple: {:?}", tuple);
    }
    for tuple in tuple_vec {
        println!("Tuple: {:?}", tuple);
    }
}
```



Consumer

Il metodo **collect()** produce un contenitore a partire da un iteratore.

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5];

    // Filtriamo solo i numeri pari e li raccogliamo in un nuovo vettore
    let even_numbers: Vec<_> = numbers.iter()
        .filter(|&x| x % 2 == 0)
        .collect();

    println!("Numeri pari: {:?}", even_numbers);
}
```



La funzione **for_each()** applica una closure su ciascun elemento (come se fosse un ciclo).

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5];

    // Stampiamo ogni numero del vettore
    numbers.iter().for_each(|&num| {
        println!("Numero: {}", num);
    });
}
```



Il metodo **all()** ritorna un booleano in base ad una condizione (che deve essere verificata da tutti gli elementi).

```
fn main() {
    let numbers = vec![2, 4, 6, 8, 10];

    // Verifichiamo se tutti gli elementi sono pari
    let all_even = numbers.iter().all(|&x| x % 2 == 0);

    if all_even {
        println!("Tutti gli elementi sono pari.");
    } else {
        println!("Almeno un elemento non è pari.");
    }
}
```

all.rs

La funzione **any()** restituisce un booleano, verificando se almeno un elemento soddisfa la condizione che gli è stata passata.

```
fn main() {
    let words = vec!["hello", "world", "rust", "programming"];

    // Verifichiamo se almeno una stringa ha una lunghezza maggiore di 5 caratteri
    let any_long_word = words.iter().any(|&word| word.len() > 5);

    if any_long_word {
        println!("Almeno una parola ha una lunghezza maggiore di 5 caratteri.");
    } else {
        println!("Nessuna parola ha una lunghezza maggiore di 5 caratteri.");
    }
}
```

any.rs

La funzione **find()** cerca il primo elemento che soddisfa una determinata condizione. Ritorna un Option.

```
fn main() {
    let numbers = vec![2, 4, 6, 7, 8, 9];

    // Trova il primo numero dispari nell'iteratore
    let first_odd = numbers.iter().find(|&x| *x % 2 != 0);

    match first_odd {
        Some(odd) => println!("Il primo numero dispari è: {}", odd),
        None => println!("Nessun numero dispari trovato."),
    }
}
```

find.rs

La funzione **fold()** permette di definire un valore (accumulatore) che viene portato avanti in tutte le iterazioni. Ogni elemento viene processato modificando il valore dell'accumulatore (che viene inizializzato attraverso il primo parametro).

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5];

    // Somma tutti gli elementi della collezione
    let sum = numbers.iter().fold(0, |accumulatore, &x| accumulatore + x);

    println!("La somma di tutti gli elementi è: {}", sum);
}
```

fold1.rs

COLLEZIONI DI DATI

Le collezioni sono un insieme di strutture utilizzate per collezionare dati omogenei. I linguaggi, tra cui anche Rust, offrono una serie di strutture standard in grado di gestire alcuni problemi comuni. I tre macro-gruppi sono le liste ordinate, gli insiemi di elementi univoci (tipicamente non ordinato) e le mappe (di valori, identificati da una chiave).

Descrizione	Rust	C++	Java	Python
Array dinamico	<code>std::Vec<T></code>	<code>std::vector<T></code>	<code>java.util. ArrayList<T></code>	<code>list</code>
Coda a doppia entrata	<code>std::VecDeque<T></code>	<code>std::deque<T></code>	<code>java.util. ArrayDeque<T></code>	<code>collections. deque</code>
Lista doppiamente collegata	<code>std::LinkedList<T></code>	<code>std::list<T>*</code> *esiste anche collegata solo in avanti (forward_list)	<code>java.util. LinkedList<T></code>	—
Coda a priorità	<code>std::BinaryHeap<T></code>	<code>std::priority_queue<T></code>	<code>java.util. PriorityQueue<T></code>	<code>heapq</code>
Tabella hash	<code>std::HashMap<K,V></code>	<code>std::unordered_map<K,V></code>	<code>java.util. HashMap<K,V></code>	<code>dict</code>
Mappa ordinata	<code>std::BTreeMap<K,V></code>	<code>std::map<K,V></code>	<code>java.util. TreeMap<K,V></code>	—
Insieme Hash	<code>std::HashSet<T></code>	<code>std::unordered_set<T></code>	<code>java.util. HashSet<T></code>	<code>set</code>
Insieme ordinato	<code>std::BTreeSet<T></code>	<code>std::set<T></code>	<code>java.util. TreeSet<T></code>	—

Tutte le collezioni hanno dei metodi comuni, come ad esempio **new()**, **len()**, **iter()**, **into_iter()**, **collect()**. Non tutte, però, possiedono il metodo **iter_mut()**.

Vec<T>

È una sequenza ridimensionabile di elementi di tipo T. La variabile possiede un puntatore allo heap e le informazioni di capacità e spazio occupato. Possiede il costruttore **new()** e la macro **vec![]**. Gli inserimenti possono essere fatti in fondo col metodo **push()**.

INFO: Se il buffer è pieno, viene trovata un'altra area di memoria dove spostare tutti gli elementi e aggiungerne nuovi.

Il riferimento al contenuto è dato dalla notazione **&v[i]** oppure con i metodi **get()** e **get_mut()**.

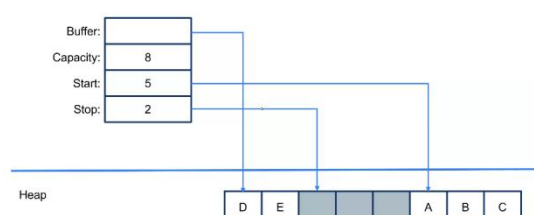
N.B. La notazione **&v[i]** genera un panic nel momento in cui si prova ad accedere ad un indice che non ricade nell'intervallo appropriato. Le **get**, invece, ritornano un **Option::None**.

I metodi **first()** e **last()** permettono di accedere facilmente al primo e ultimo elemento del vettore. Tra le altre funzioni utili ci sono **capacity()**, **pop()** (che rimuove l'ultimo elemento, ritornando **Option**), **get(range)**.

VecDeque<T>

Permette l'inserimento in testa, oltre che in coda. Le funzioni utili sono **push_back()**, **push_front()**, **pop_back()**, **pop_front()**.

INFO: Gli elementi non sono necessariamente continui fra loro in memoria.



LinkedList<T>

Si tratta di una lista doppiamente linkata, per cui ha un costo operativo costante nell'inserimento e cancellazione di elementi all'interno (perché occorre solo aggiornare i puntatori). Non c'è il fenomeno del buffer pieno e della riallocazione.

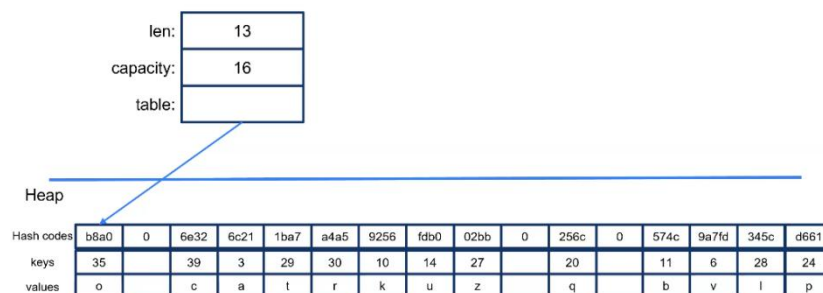
I metodi sono simili al VecDeque, si hanno infatti **push_back()**, **push_front()**, **pop_back()**, **pop_front()**.

Il metodo **split_off()** permette di dividere in due una lista: quella originale viene terminata nell'elemento precedente, mentre quello indicato viene restituito come riferimento alla nuova variabile. Il risultato della split, dunque, sarà una lista che parte dall'indice scelto, mentre quella precedente viene "tagliata".

INFO: Per ordinare una lista bisogna convertirla in vec, ordinarlo e ritornare alla lista.

HashMap<K, V>

È una collezione di coppie composte da una chiave e un valore. Viene utilizzata quando le chiavi non hanno un ordine. Queste sono univoche e il tipo K deve implementare i tratti **Eq** e **Hash**.



Così come i vec, se lo spazio finisce viene riallocata l'intera collezione in un blocco di memoria più grande.

```
fn main() {
    let mut scores = HashMap::new();

    // Inserimento di coppie chiave-valore
    scores.insert(String::from("Alice"), 100);
    scores.insert(String::from("Bob"), 85);
    scores.insert(String::from("Charlie"), 90);

    // Accesso ai valori tramite chiave
    println!("Punteggio di Alice:{}", scores["Alice"]);

    // Aggiornamento di un valore
    scores.insert(String::from("Bob"), 90);

    // Stampa di tutti i punteggi
    for (name, score) in &scores {
        println!("{}", name, score);
    }

    // Verifica se una chiave esiste nella HashMap
    if !scores.contains_key("David") {
        println!("David non ha un punteggio registrato");
    }

    // Rimozione di una coppia chiave-valore
    scores.remove("Bob");

    // Verifica se la chiave è presente dopo la rimozione
    if scores.contains_key("Bob") {
        println!("Bob ha ancora un punteggio registrato.");
    } else {
        println!("Bob non ha più un punteggio registrato.");
    }

    // Controllo della lunghezza della HashMap
    println!("Numero di punteggi: {}", scores.len());

    // Iterazione sui valori della HashMap
    for score in scores.values() {
        println!("Punteggio: {}", score);
    }

    // Iterazione sui riferimenti della HashMap
    for (name, score) in &scores {
        println!("{}", name, score);
    }

    // Rimozione di tutti gli elementi dalla HashMap
    scores.clear();

    // Verifica se la HashMap è vuota
    if scores.is_empty() {
        println!("La HashMap è vuota.");
    }
}
```

INFO: Il metodo **iter_mut()** permette di modificare il valore e non la chiave.

Il metodo **entry** permette di evitare la doppia lettura nella mappa, nel momento in cui si vuole modificare o aggiungere un elemento. Di norma, infatti, servirebbe una prima lettura per ricercarne l'eventuale esistenza ed una seconda per individuare il punto in cui poter aggiungere l'elemento, qualora non esistesse.

Il metodo ritorna un enum **Entry**, che contiene le informazioni del valore ricercato (se esiste) oppure la posizione in cui è possibile aggiungerlo.

```
entry(&mut self, key: K) -> Entry<'a, K, V>

pub enum Entry <'a, K, V> {
    Occupied(OccupiedEntry <'a, K, V>),
    Vacant(VacantEntry <'a, K, V>),
}
```

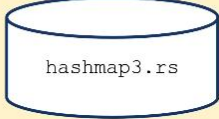
Sono fornite anche le funzioni **and_modify** (che prende una closure), **or_insert** e **or_insert_with**.

```
fn main() {
    let mut scores = HashMap::new();

    // Inserimento di una coppia chiave-valore utilizzando il metodo entry
    scores.entry("Alice").or_insert(100);
    scores.entry("Bob").or_insert(90);
    scores.entry("Charlie").or_insert(80);

    // Aggiornamento del punteggio di Alice usando il metodo entry
    let alice_entry = scores.entry("Alice");
    match alice_entry {
        Entry::Occupied(mut entry) => {
            *entry.get_mut() += 10; // Aggiunge 10 al punteggio di Alice
        }
        Entry::Vacant(_) => {
            println!("Alice non trovata"); // Alice non è presente nella mappa
        }
    }

    // Stampa della HashMap aggiornata
    println!("HashMap: {:?}", scores);
}
```



```
use std::collections::HashMap;

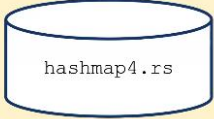
fn main() {
    let mut scores = HashMap::new();

    scores.insert("Team Blue", 10);
    scores.insert("Team Red", 20);

    // Incrementa il punteggio del team "Team Blue" se esiste,
    // altrimenti inserisci un nuovo punteggio
    scores.entry("Team Blue").and_modify(|score| *score += 5).or_insert(15);

    // Incrementa il punteggio del team "Team Green" se esiste,
    // altrimenti inserisci un nuovo punteggio
    scores.entry("Team Green").and_modify(|score| *score += 5).or_insert(15);

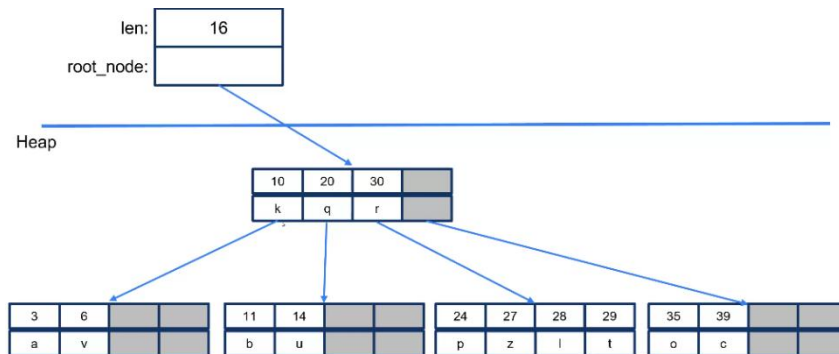
    println!("{:?}", scores);
    // Stampa: {"Team Blue": 15, "Team Red": 20, "Team Green": 15}
}
```



BTreeMap<K, V>

È una collezione di coppie composte da una chiave e un valore (quest'ultimo salvato nello heap come singolo albero). Viene utilizzata quando le chiavi hanno un ordine e permette di migliorare l'efficienza di accesso ai nodi. La chiave deve essere univoca e implementare il tratto **Ord**.

Questa mappa è suddivisa per livelli e il puntatore principale accede al nodo radice.



Anche in questo caso è possibile che ci sia dello spazio vuoto lasciato per ospitare nuovi elementi. Se dovesse riempirsi tutto, la struttura può essere riallocata.

INFO: Il contenuto di un nodo non è fisso: gli elementi possono essere ridistribuiti per garantire un bilanciamento dell'albero.

Anche la BTreeMap implementa il metodo `entry`, ma non sono forniti i metodi `and_modify()` e `or_insert()`. Per modificare o inserire un valore si usano **`get_mut()`** e **`insert()`**.

INFO: I tipi di dato per le chiavi e i valori nelle mappe sono inferiti: il compilatore è in grado di capirli con la prima assegnazione.

Insiemi

Applicano gli stessi criteri delle mappe. In particolare un **HashSet<T>** è un insieme di elementi univoci salvati nello heap con una singola hash table. Il **BTreeSet<T>** è un insieme di elementi univoci salvati nello heap come albero, dove ogni entry rappresenta un nodo.

BinaryHeap<T>

È una struttura dati i cui valori sono salvati nello heap. L'elemento più grande si trova nel fronte della struttura dati. Il tipo `T` deve implementare il tratto `Ord`.

Il metodo **`peek()`** permette di ritornare l'elemento più grande, modificabile con **`peek_mut()`**.

La funzione **`into_sorted_vec()`** permette di ottenere un vettore ordinato.

FILE I/O

In Rust, le struct **Path** (unsized e accessibile in lettura) e **PathBuf** (modificabile) permettono di nascondere le differenze tra diversi sistemi operativi nella definizione di un percorso file. I metodi forniti permettono di ricavare informazioni come proprietario, data di creazione e permessi di ciascun file.

La funzione **read_dir(dir: &Path)** ritorna un **Result** contenente un tipo **DirEntry**, che descrive gli elementi contenuti in una cartella (esplicitando nome, tipo, metadati e path). Le funzioni **create_dir** e **remove_dir** permettono, invece, di creare ed eliminare le cartelle.

```
use std::fs;

fn main() -> std::io::Result<> {
    // Definisci il percorso della nuova directory da creare
    let new_directory_path = "./mynewdir";

    // Crea la nuova directory
    fs::create_dir(new_directory_path)?;

    println!("Directory creata con successo!");

    Ok(())
}
```



Nell'esempio viene utilizzato il **?** perché tutti gli errori possibili sono dello stesso tipo (**std::io**), come indicato nella signature del **main**.

La manipolazione dei file è possibile attraverso i metodi **copy**, **rename** (può essere utilizzato anche per spostarlo) e **remove_file**.

L'accesso al blocco di byte di un file è permesso dal sistema operativo e per poterci accedere occorre "aprire" il file. Il sistema operativo restituisce, in tal caso, un riferimento opaco al file attraverso un **handle** o **file descriptor** (di fatto un numero intero). La struct **File** fornisce due metodi per aprire un file: **open** (accesso in lettura) e **create** (tronca il file se esiste o ne apre uno nuovo, con accesso in scrittura).

```
use std::fs::File;
use std::io::prelude::*;
use std::io::Error;

fn main() -> Result<>, Error> {
    // Definisci il percorso del file da aprire
    let file_path = "./myfile";

    // Apri il file in modalità di lettura
    let mut file = File::open(file_path)?;

    // Leggi il contenuto del file in una stringa
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;

    // Stampa il contenuto del file
    println!("Contenuto del file:");
    println!("{}", contents);

    Ok(())
}
```



N.B. La funzione **close** del file non esiste. L'operazione è implicita nella distruzione dell'handle del file.

La struct **OpenOption** permette di configurare più opzioni di apertura, fornendo una modalità specifica. In particolare, offre i metodi **read()**, **create()**, **write()**, **truncate()**, **append()** i quali prendono come parametro un booleano. Dopo aver impostato la configurazione, occorre usare il metodo **open** per procedere all'effettiva apertura.

```
use std::fs::OpenOptions;
use std::io::prelude::*;

fn main() -> std::io::Result<()> {
    let file_path = "./new_file.txt";

    // Aprire il file in modalità di scrittura, troncandolo se esiste già
    let mut file = OpenOptions::new()
        .write(true)
        .truncate(true)
        .open(file_path)?;

    let text = "Questo è un nuovo file creato in Rust!";
    file.write(text.as_bytes())?;

    Ok(())
}
```



INFO: Le funzioni **read_to_string** e **write** offrono un meccanismo compatto per leggere e scrivere il contenuto di un file di dimensioni non troppo grandi, quindi non devono essere usate se un file può avere dimensioni maggiori della memoria allocabile dal processo.

Rust gestisce le operazioni di I/O attraverso alcuni tratti che implementano le funzioni di lettura e scrittura, come **Read**, **BufRead**, **Write** e **Seek**. In caso di errore ritornano un enum **ErrorKind** che rappresenta le diverse categorie di errori che si possono verificare. Tra le varianti di errore ci sono: **NotFound**, **PermissionDenied**, **Interrupted**, **TimedOut**.

INFO: L'errore **Interrupted** indica un errore non fatale, per cui è possibile che l'operazione vada a buon fine ripetendola (magari aspettando un po' di tempo).

```
fn main() {
    // Tentativo di apertura di un file
    match File::open("testo.txt") {
        Ok(mut file) => {
            let mut contenuto = String::new();
            // Tentativo di lettura del contenuto del file
            match file.read_to_string(&mut contenuto) {
                Ok(_) => println!("Contenuto del file: {}", contenuto),
                Err(e) => match e.kind() {
                    // Gestione specifica per diversi tipi di errori
                    ErrorKind::NotFound => println!("Il file non è stato trovato."),
                    ErrorKind::PermissionDenied => println!("Permesso negato."),
                    _ => println!("Si è verificato un errore durante la lettura del file: {}", e),
                },
            }
        },
        Err(e) => match e.kind() {
            ErrorKind::NotFound => println!("Il file non è stato trovato."),
            ErrorKind::PermissionDenied => println!("Permesso negato."),
            _ => println!("Si è verificato un errore durante l'apertura del file: {}", e),
        },
    }
}
```



Il tratto **Read** è implementato dai tipi **File**, **Stdin** e **TcpStream** attraverso l'implementazione di **read()**. Permette la lettura del contenuto di un buffer e restituisce il numero di byte ancora da leggere attraverso un **Ok(n)**. La lettura, quindi, può essere fatta in più riprese finché **n** non vale 0.

Basati sul metodo **read**, ci sono altri metodi più specifici come **read_to_end**, **read_to_string**, **take** (limita il numero di byte da prendere, ha effetto sulla **read** seguente).

Il tratto **BufRead** ottimizza le operazioni in lettura. Quando viene richiamata una `read`, viene impiegato un buffer in memoria con l'obiettivo di ridurre il numero di chiamate alle system call. L'accesso al file system, quindi, avviene solo quando si vuole caricare il buffer.

Il tratto richiede i metodi **fill_buf()**, che riempie il buffer, e **consume()**, che consuma un numero specificato di byte. I metodi **read_line** e **lines** scandiscono il buffer.

```
use std::fs::File;
use std::io;
use std::io::{Write, BufReader, BufRead};

fn main() -> io::Result<()> {
    let path = "myfile";

    let mut output = File::create(path)?;
    write!(output, "Rust\n♥️\nFun"?);

    let input = File::open(path)?;
    let buffered = BufReader::new(input);

    for line in buffered.lines() {
        println!("{}", line?);
    }

    Ok(())
}
```



INFO: La macro **write!** stampa su un buffer.

Nell'esempio seguente viene creato e successivamente riempito un buffer da un file, per essere poi letto riga per riga in un ciclo. Attraverso il metodo `len()` si verifica se nel buffer sono presenti ancora dati da leggere.

```
fn main() -> Result<(), Error> {
    // Aprire il file in modalità di lettura
    let file = File::open("./prova.txt"?);

    // Creare un BufReader per il file
    let mut buf_reader = BufReader::new(file);

    // Buffer per memorizzare il contenuto letto
    let mut buffer = String::new();

    // Riempire il buffer interno di BufReader
    buf_reader.fill_buf()?;

    // Leggere il contenuto del buffer fino a quando non si raggiunge la fine del file
    while buf_reader.buffer().len() > 0 {
        // Leggere una riga dal buffer
        let bytes_read = buf_reader.read_line(&mut buffer)?;

        // Stampa la riga letta
        println!("{}", buffer);

        // Pulisci il buffer per prepararlo per la prossima lettura
        buffer.clear();
        // Continua a riempire il buffer interno di BufReader
        buf_reader.fill_buf()?;
    }

    Ok(())
}
```



Il tratto **Write** è implementato dalle struct **File**, **Stdout**, **StdErr** e **TcpStream** attraverso i metodi `write` e `flush`. Il metodo **write** permette di scrivere il contenuto del buffer ricevuto come argomento e restituisce il numero di byte scritti. La funzione **flush**, invece, finalizza l'output sul disco garantendo che tutti i buffer transitori siano stati svuotati. Con **write_all** viene richiamata ricorsivamente la `write` fin quando tutti i dati sono stati scritti (o si verifica un errore).

Il tratto **Seek** permette di muovere il cursore su file nel disco. La posizione può essere definita rispetto al punto corrente, iniziale o finale. I metodi forniti sono **seek**, che posiziona il cursore alla posizione in byte indicata dal parametro fornito, e **stream_position**, che restituisce la posizione corrente.

Serde

Si tratta di un crate che permette la conversione a partire da file tradizionali verso strutture dati Rust (**serializzazione**), e viceversa (**deserializzazione**). Questi formati sono solitamente di tipo testuale, per esempio JSON e YML.

Per utilizzarlo occorre aggiungere la dipendenza nel `cargo.toml`, indicando la versione dei formati che si intendono utilizzare.

```
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
```

La struttura dati che si vuole usare per la conversione deve essere marcata con un **derive**.

```
#[derive(Serialize, Deserialize, Debug)]
struct Data {
    name: String,
    data: Vec<u8>,
    attributes: HashMap<String, String>,
}
```

```
use serde::{Serialize, Deserialize};
use std::fs::File;
use std::io::Write;
// Definiamo una struttura dati per i nostri dati JSON.
#[derive(Debug, Serialize, Deserialize)]
struct Persona {
    nome: String,
    cognome: String,
    eta: u32,
}
fn main() -> Result<(), Box<dyn std::error::Error>> {
    let persona1 = Persona {
        nome: "Mario".to_string(),
        cognome: "Rossi".to_string(),
        eta: 30,
    };
    let persona2 = Persona {
        nome: "Luigi".to_string(),
        cognome: "Bianchi".to_string(),
        eta: 25,
    };
    let persone = vec![persona1, persona2];

    // Serializziamo il vettore in formato JSON.
    let json_data = serde_json::to_string(&persone)?;
    // let json_data = serde_json::to_string_pretty(&persone)?;
    // let mut file = File::create("persone.json"); // Scriviamo il JSON su un file.
    file.write_all(json_data.as_bytes())?;
    Ok(())
}
```



```
fn main() -> Result<(), Box<dyn Error>> {
    // Lettura del file JSON
    let file = File::open("people.json");
    let reader = BufReader::new(file);

    // Deserializzazione dei dati JSON in una struttura Rust
    let people: Vec<Person> = serde_json::from_reader(reader)?;

    // Stampa dei dati deserializzati
    println!("People:");
    for person in &people {
        println!("{:?}", person);
    }
}
Ok(())
```



SMART POINTER

Gli smart pointer si comportano come puntatori, permettendo di accedere ad un dato, ma presentano anche ulteriori caratteristiche, in grado di garantire un accesso efficiente e controllato alla memoria.

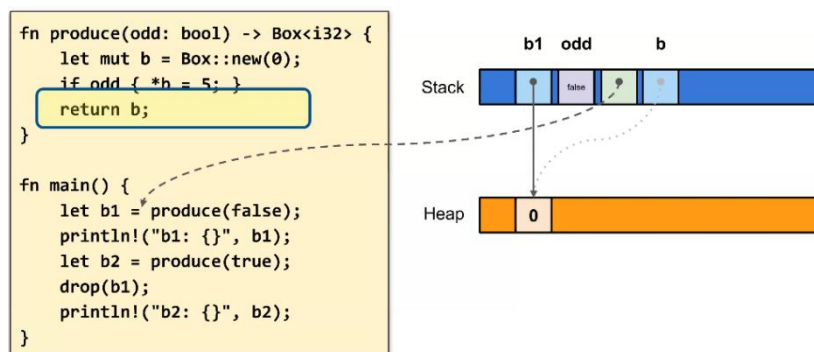
Rust fornisce diverse strutture di smart pointer, che implementano i tratti di **Deref** e **DerefMut**. Il compilatore, incontrando l'espressione `*ptr`, ne applica una trasformazione in `*ptr.deref()` o `*ptr.deref_mut()`.

```
trait Deref {
    type Target: ?Sized;
    fn deref(&self) -> &Self::Target;
}

trait DerefMut: Deref {
    fn deref_mut(&mut self) -> &mut Self::Target;
}
```

Box<T>

Permette di allocare dinamicamente un dato nello heap e ne detiene il possesso. La variabile memorizza un puntatore, che può essere semplice, fat o double. Se il tipo T implementa il tratto `Sized` il compilatore conosce a priori la dimensione del dato, in questo caso il puntatore sarà semplice. Se il compilatore non conosce la dimensione, è necessario utilizzare un fat pointer.



INFO: Il `drop` è implicito, ma si può richiamare esplicitamente all'interno dello scope.

N.B. Si può fare un `Box` solo di dati dei quali si può avere il possesso: è impossibile, per esempio, creare una `Box` di una `&str`.

Tipi ricorsivi

Esistono alcuni tipi particolari, come la Cons List, formati da una enum che può possedere un riferimento ad un altro valore dello stesso tipo. Per esempio, in questo caso, si hanno delle tuple in cui il primo elemento è un valore numerico e il secondo un riferimento ad una tupla dello stesso tipo.

```
enum List {
    Cons(i32, List),
    Nil,
}
```

`(1, (2, (3, Nil)))`

N.B. Rust non permette di allocare un tipo così fatto, perché non riesce a dare un limite dimensionale a questa struttura. L'unico modo per crearla è attraverso l'ausilio di un `Box`, che permette di dare una dimensione finita (intero + `usize` del `box`) al singolo elemento.

```
enum List {
    Cons(i32, Box<List>),
    Nil,
}
```

Cons

i32	Box usize
-----	---

```
use crate::List::{Cons, Nil};

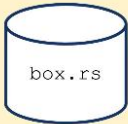
enum List {
    Cons(i32, Box<List>),
    Nil,
}

fn main() {
    let a = Cons(3, Box::new(Nil));
    let b = Cons(2, Box::new(a));
    let c = Cons(1, Box::new(b));
    let head = Cons(0, Box::new(c));

    // equivalente a
    // let list = Cons(0, Cons(1, Cons(2, Cons(3, Nil))));

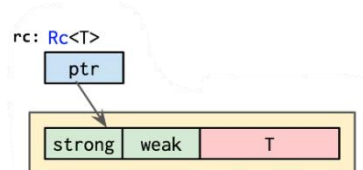
    let mut current_node = &head;

    while let Cons(value, next) = current_node {
        println!("Value: {}", value);
        current_node = next;
    }
}
```



Rc<T>

Il tipo **Rc** definisce un puntatore che rimanda ad una struttura nello heap che contiene, oltre al dato **T**, un contatore **strong** e un contatore **weak**. Viene utilizzato quando occorre avere più possessori di un dato immutabile*: il primo contatore, infatti, tiene traccia del numero di copie del puntatore, mentre il secondo del numero di riferimenti deboli esistenti. Il contatore viene decrementato ogni volta che un puntatore esce dal proprio scope, provocando la liberazione della memoria nel caso in cui tutti i riferimenti fossero stati eliminati.



L'accesso ad altre variabili è fornito attraverso l'operazione di **clone()**.

N.B. Questo smart pointer è utilizzabile solo in un ambiente **single-thread**. L'aggiornamento del contatore, infatti, è problematica nel contesto dei multi-thread e va gestita in maniera opportuna.

Il campo **weak** viene incrementato nel momento in cui viene creato un puntatore **weak** al dato.

N.B. Per evitare problemi di omonimia con i metodi del dato incapsulato, i metodi **Rc** vanno dichiarati in modo esteso. Non è possibile, per esempio, eseguire il comando `rc.clone()` ma occorre usare:

Rc:::<T>::strong_count(&a)

Dove **&a** è il riferimento alla struttura **rc**.

```

use std::rc::Rc;

fn main() {
    let mut valore = Rc::new(5);

    println!("Valore: {:?}", valore);

    let copia = Rc::clone(&valore);
    println!("Valore di copia: {:?}", copia);

    let rif_rc = &*copia;

    visualizza(rif_rc);
    println!("Leggo questi valori: {} e {}", valore, copia);
}

fn visualizza (par: &i32) {
    println!("Ho un riferimento a questo valore {}", par);
}

```



INFO: Con la notazione **&*rc** è possibile ottenere un riferimento al valore interno dell'Rc.

***N.B.** Il dato contenuto nell'Rc è mutabile solo se non esistono altri riferimenti che lo puntano. Il metodo da usare è **get_mut**, che ritorna una Option contenente un &mut al valore.

```

use std::rc::Rc;

fn main() {
    let mut valore = Rc::new(5);
    {
        println!("Valore: {:?}", valore);

        let copia = Rc::clone(&valore);
        println!("Valore di copia: {:?}", copia);

        match Rc::get_mut(&mut valore) {
            Some(v) => *v += 10, // Modifica il valore se è possibile.
            None => println!("Non è possibile ottenere un riferimento mutabile."),
        }
    }
    // Prova a ottenere un riferimento mutabile.
    match Rc::get_mut(&mut valore) {
        Some(v) => *v += 10, // Modifica il valore se è possibile.
        None => println!("Non è possibile ottenere un riferimento mutabile."),
    }

    println!("Valore: {:?}", valore);
}

```



Weak<T>

Permette di fare riferimento ad un dato in modalità "leggera". Questo tipo di dato permette di realizzare una sequenza circolare di puntatori, che altrimenti genererebbero una situazione tale per cui sarebbe impossibile deallocare le strutture create.

Lo smart pointer di tipo Weak si crea a partire da un Rc con il metodo **downgrade(&rc)**. Questo permette l'accesso al valore dell'Rc solo se questo esiste ancora e il numero di accessi attivi di questo tipo è memorizzato nel campo weak.

N.B. Questo puntatore non partecipa al processo di tenuta in vita dell'Rc di riferimento.

Se il valore originale è ancora in vita, il puntatore può essere trasformato in Rc col metodo **upgrade()**.

N.B. Il dato non è leggibile attraverso la dereferenziazione del puntatore Weak.

```

use std::rc::Rc;
fn main() {
    let five = Rc::new(5);
    let ten = Rc::clone(&five);

    let weak_five = Rc::downgrade(&five);

    println!("Conteggio referenze strong di five: {}", Rc::strong_count(&five));
    println!("Conteggio referenze weak di five: {}", Rc::weak_count(&five));

    let strong_five = weak_five.upgrade();
    println!("{:?}", strong_five);
    match strong_five {
        Some(strong_five) => println!("{:?}", strong_five),
        None => println!("Il dato non è disponibile "),
    }
}

```



INFO: La funzione upgrade restituisce una Option.

Cell<T>

Esistono delle strutture che permettono dei controlli runtime (e non in fase di compilazione) grazie alle quali è possibile bypassare le regole “solite” di Rust, come ad esempio quella che impone l’esistenza di un solo riferimento mutabile alla volta. Questo tipo, però, funzionano solo in contesti non concorrenti.

Le funzioni base sono **set**, **get**, **take**, **replace** e **into_inner** (che ritorna il dato distruggendo la cella).

N.B. Quello che si realizza è una internal mutability nel contesto di una external immutability: nonostante venga inserito all’interno di strutture non mutabili, il dato contenuto all’interno della Cell è modificabile.

```

use std::cell::Cell;
#[derive(Debug)]
struct SomeStruct {
    a: u8, // il campo a è soggetto a tutti i vincoli del borrow checker
    b: Cell<u8>, // il campo b è soggetto ad un rilassamento di questi vincoli
}
fn main() {
    let my_struct = SomeStruct {
        a: 0,
        b: Cell::new(1),
    };

    let copy_struct = &my_struct;
    my_struct.b.set(100);

    // OK: anche se esiste un Reference, `b` è una Cell e può essere modificata

    println!("{:?}", my_struct);
    println!("{:?}", copy_struct);
}

```



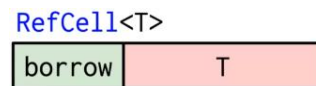
N.B. Anche se viene creato un riferimento alla struttura che contiene la Cell, quest’ultima può essere modificata. Lo smart pointer, infatti, permette di avere più flessibilità rispetto alle variabili comuni.

N.B. Questo smart pointer non permette di creare riferimenti diretti al dato contenuto al suo interno.

RefCell<T>

Questo puntatore permette di creare riferimenti al suo contenuto. La struttura contiene il campo `borrow` che tiene traccia del numero di prestiti correnti. Gli accessi sono regolati dal borrow checker, attraverso controlli runtime.

L'accesso è consentito in lettura con più riferimenti, ma solo se tutti quanti accedono solo in lettura. Se viene chiesto in lettura ma è già occupato in scrittura, genera un panic (così come se viene chiesto due volte in scrittura).



La funzione **borrow** restituisce uno smart pointer `RefCell` alla struttura e incrementa il contatore `borrow`. Il **borrow_mut**, invece, permette le operazioni in scrittura. Esistono due metodi equivalenti (**try_borrow** e **try_borrow_mut**) che, in caso di errore, non generano un panic, restituendo invece un `result`.

```
use std::cell::RefCell;
fn main() {
    let mut c = RefCell::new(5);
    println!("Il dato è {:?}", c);
    {
        *c.get_mut() += 5;
        println!("Il dato aggiornato è {:?}", c);
        let mut m = c.borrow_mut();
        if (c.try_borrow().is_err())
        {
            println!("Non posso fare un altro prestito"); // Si entra qua
        }
        *m = 6;
        println!("Il dato è {:?}", m);
    }
    if (c.try_borrow().is_ok())
    {
        println!("Posso fare un altro prestito");
        let m = c.borrow();
        println!("Il dato è {:?}", m);
        if (c.try_borrow_mut().is_ok())
        {
            println!("Posso fare un altro prestito"); // non si entra qua
            let mut m2 = c.borrow_mut();
        }
        drop(m);
        let val = c.into_inner();
        println!("Valore Finale dopo la distruzione del RefCell{:?}", val);
    }
}
```



INFO: Il metodo `into_inner()` ritorna il dato, distruggendo la variabile da cui lo prende.

Combinare i due smart pointer permette di avere più puntatori (grazie a **Rc**) ad elementi mutabili (grazie a **RefCell**).

```
#[derive(Debug)]
enum List {
    Cons(Rc<RefCell<i32>>, Rc<List>),
    Nil,
}

use List::{Cons, Nil};
use std::rc::Rc;
use std::cell::RefCell;

fn main() {
    let value = Rc::new(RefCell::new(5));

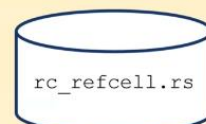
    let a = Rc::new(Cons(Rc::clone(&value), Rc::new(Nil)));

    let b = Cons(Rc::new(RefCell::new(6)), Rc::clone(&a));
    let c = Cons(Rc::new(RefCell::new(10)), Rc::clone(&a));

    println!("a before = {:?}", a);
    println!("b before = {:?}", b);
    println!("c before = {:?}", c);

    *value.borrow_mut() += 10;

    println!("a after = {:?}", a);
    println!("b after = {:?}", b);
    println!("c after = {:?}", c);
}
```



Cow<'a, B>

Implementa il meccanismo **clone on write**, per cui se si cerca di modificare un contenuto, il dato viene clonato, lasciando l'originale invariato.

N.B. Se si possiede il dato, questo può essere modificato direttamente, mentre se lo si ha in prestito viene eseguita una copia.

```
pub enum Cow<'a, B>
where B: 'a + ToOwned + ?Sized,
{
    Borrowed(&'a B), // Questo indica che il valore è preso in prestito
                    // e non può essere modificato
    Owned(<B as ToOwned>::Owned), // Questo indica che il valore è di proprietà
                                // e può essere modificato.
}
```

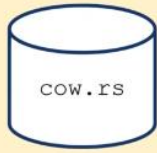
Il metodo **from()** crea un Cow e in base al tipo di dato fornito decide se collocarlo in un Borrowed o Owned. Il metodo **to_mut()** restituisce un riferimento mutabile ai dati, **into_owned()** restituisce i dati come proprietà, **as_ref()** e **as_mut()** ritornano dei riferimenti.

Nell'esempio seguente il Cow di input genera una variante **Borrowed**, perché il dato viene prestato (viene fornito un riferimento).

```
use std::borrow::Cow;
fn abs_all<'a>(input: &'a mut Cow<'a, [i32]>) -> Cow<'a, [i32]>{
    // Check if the input slice contains any negative values
    if input.iter().any(|&x| x < 0) {
        // If yes, create a new vector with the absolute values
        let mut output = Vec::new();
        for &x in input.iter(){
            output.push(x.abs());
        }
        // Return the vector as an owned slice
        println!("Owned => ");
        Cow::Owned(output)
    } else {
        // If no, return the input slice as a borrowed slice
        println!("Borrowed => ");
        Cow::Borrowed(input)
    }
}
fn main() {
    let slice = [0, 1, 2];
    let mut input = Cow::from(&slice[..]);
    let mut output = abs_all(&mut input); // No clone occurs because input doesn't need to be mutated.
    println!("{:?}", output);

    let slice = [-1, 0, 1];
    let mut input = Cow::from(&slice[..]); // Clone occurs because input needs to be mutated.
    output = abs_all(&mut input);
    println!("{:?}", output);

    let mut input = Cow::from(vec![-1, 0, 1]); // No clone occurs because input is already owned.
    output = abs_all(&mut input);
    println!("{:?}", output);
}
```



CONCORRENZA

Un programma è concorrente quando si svolge in più flussi paralleli. Questo è possibile grazie ai processori multi-core, che permettono di distribuire tali processi.

Due processi distinti sono isolati e vivono nel proprio spazio di indirizzamento. All'interno di un singolo processo è possibile avviare più flussi di esecuzione, i thread, che al contrario condividono lo spazio di indirizzamento.

INFO: La cooperazione tra processi diversi, che quindi non condividono lo spazio, ha la difficoltà aggiuntiva dello scambio di informazioni, che è più complicato da gestire.

Ogni processo in partenza ha un solo thread e può chiedere allo schedatore di avviarne altri. Un thread occupa del tempo di cpu, quindi coinvolge il sistema operativo. Si chiama **thread nativo** quel thread che è gestito direttamente dal sistema operativo, il quale si occupa di schedarlo in base alla disponibilità della cpu. Il **green thread**, invece, è schedato al livello utente e richiede una forma di cooperazione da parte del codice, che deve invocare esplicitamente lo schedatore.

Le operazioni da effettuare per utilizzare un thread sono la **creazione**, che ritorna un **handle** col quale fare riferimento al thread, l'**identificazione** del thread corrente (numero univoco al livello di sistema), l'**attesa** di terminazione del thread e l'accesso al suo stato finale (operazione di **join**).

N.B. Tra le funzioni non supportate c'è la cancellazione di un thread. Occorre quindi applicare una strategia per comunicare al thread di interrompersi.

L'utilizzo dei thread implica un aumento considerevole della complessità del codice, per cui occorre usarli solo quando si ha un reale beneficio. Possono essere utili, per esempio, quando c'è un forte utilizzo della cpu oppure quando c'è un forte utilizzo di operazioni di I/O (perché nei tempi di attesa si possono svolgere delle operazioni). Si utilizzano i thread, quindi, quando si devono fare "più cose contemporaneamente".

Se la maggior parte del tempo di un programma è, però, dedicato ad operazioni di I/O e si ha necessità di "aspettare insieme", allora è più opportuno ricorrere alla programmazione asincrona.

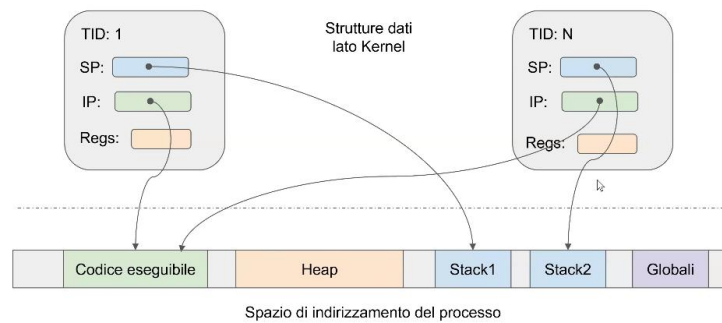
L'utilizzo di più thread è problematico perché ogni core accede ad una propria cache: in caso di scrittura è possibile che si verifichino delle inconsistenze sui dati e che quindi thread su core paralleli vedano valori diversi per le stesse variabili. Il problema è quindi di **visibilità**: le scritture dovrebbero arrivare fino alla memoria principale, in modo da essere visibili a tutti i thread attivi.

Un altro problema è dato dall'**atomicità** delle operazioni: nel caso si lavorasse con un solo thread e questo venisse interrotto, successivamente riprenderebbe dallo stesso punto, senza provocare particolari problemi. Nel caso di più thread, però, questo potrebbe portare a dei risultati intermedi non accettabili.

Gli aggiornamenti della memoria, infine, potrebbe propagarsi con tempi differenti, per cui gli effetti delle operazioni attuate dai thread potrebbero apparire ad altri thread con un altro ordine. Questo problema è detto di **ordinamento**.

INFO: I diversi modelli di processori hanno risposte diverse a questi problemi. Gli x86, per esempio, offrono operazioni di tipo fence, che forzano il completamento delle operazioni di scrittura bloccando temporaneamente gli altri core e invalidando la loro cache. Questo comportamento, però, ha effetti importanti sulle prestazioni. ARM, invece, crea delle memory barrier di lettura, scrittura e completa con comportamenti diversi a seconda delle esigenze.

Ogni creazione di un nuovo thread per un processo comporta la creazione di uno stack. Il codice eseguibile, lo heap e le variabili globali, invece, sono condivise.



La creazione di un thread avviene attraverso la funzione di libreria `spawn`, che accetta una chiusura contenente il codice da lanciare.

```
use std::thread;
use std::time::Duration;

fn run(msg: &str) {
    for i in 0..10 {
        println!("{}", msg, i);
        thread::sleep(Duration::from_nanos(1));
    }
}

fn main() {
    let t1 = thread::spawn(|| {run("aaaa"); });
    let t2 = thread::spawn(|| {run("bbbb"); });
    t1.join().unwrap();
    t2.join().unwrap();
}
```

Rust

spawn.rs

```
aaaa0
aaaa1
aaaa2
bbbb0
bbbb1
bbbb2
aaaa3
aaaa4
aaaa5
aaaa6
aaaa7
bbbb3
bbbb4
bbbb5
aaaa8
aaaa9
bbbb6
bbbb7
bbbb8
bbbb9
```

INFO: La funzione `run` stampa 10 volte la stringa fornita, seguita da un numero.

N.B. Le variabili restituite da `spawn` sono gli handle dei thread. Si può aspettare il loro termine attraverso la funzione `join()`, che ritorna un `Result`.

Il lancio di più thread provoca la perdita del determinismo: non è possibile sapere in che ordine e in che momento vengano eseguiti.

Se due thread cercano di accedere alla stessa variabile in lettura/scrittura si verifica una **corsa critica**, per cui in determinate condizioni non controllabili dal programmatore (dovute per esempio ad operazioni non atomiche) il dato memorizzato può assumere un valore che non è nè quello scritto dal primo thread, nè dal secondo. Occorre, quindi, fare in modo che l'accesso in memoria sia deterministico, sia attraverso soluzioni hardware che a livello di sistema operativo (attraverso le strutture `mutex` e `condition variable`).

Un **mutex** genera una barriera e garantisce che, entrando nel blocco di codice in questione, venga invalidata la linea di cache corrispondente alla variabile condivisa. All'uscita dal blocco viene poi eseguita la flush della cache, in modo che la variabile risulti aggiornata. Un mutex può essere libero o appartenere ad un solo thread.

Una **condition variable** fa attendere un thread fin quando non si verifica una condizione (a patto che questo glielo comunichi), senza consumare cpu. Garantisce una barriera di protezione e coerenza della memoria, se viene usato combinato ad un mutex.

Il linguaggio fornisce dei **tipi atomici**, che garantiscono operazioni Read-Modify-Write atomiche (ovvero non interrompibili e non osservabili negli stati intermedi). Queste operazioni sono limitate ai tipi semplici e sono esposte attraverso alcune particolari astrazioni.

N.B. Non c'è alcuna parentela tra thread diversi, a differenza dei processi, in cui c'è un padre e un figlio.

Attraverso la keyword `move` della chiusura fornita ad un thread è possibile catturare una variabile libera e darne la proprietà al thread.

```
use std::thread;

fn main() {
    let data = vec![1, 2, 3, 4];
    // provare con
    // let data = vec![];
    // e vedere che cosa succede

    // Creiamo un nuovo thread e trasferiamo il possesso del vettore "data"
    let handle = thread::spawn(move || {
        let len:usize= data.len();
        let somma: usize = data.iter().sum();
        somma/len
    });
    println!("Il thread principale aspetta");
    let average = handle.join();
    match average {
        Ok(res) => {println!("La media è {:?}", res);},
        Err(err) => {println!("Errore {:?}", err);}
    }
    println!("Il thread principale termina.");
}
```

Diagramma: Un rettangolo giallo contiene il codice Rust. Una linea blu punta dal testo "Cattura per movimento delle variabili che diventano di proprietà del thread" a una parola "move" sottolineata nel codice. Un cilindro blu con "join.rs" è collegato al metodo "join()" nel codice.

È possibile, in maniera opzionale, configurare un thread prima di lanciarlo, in modo da aggiungere alcune informazioni. La struct **Builder** permette di assegnare un nome al thread e di definire la dimensione dello stack. Il metodo `spawn` consuma l'oggetto così creato, restituendo un `JoinHandle`.

```
use std::thread;

let builder = thread::Builder::new()
    .name("t1".into())
    .stack_size(100_000);

let handler = builder.spawn(|| { /* codice */});

handler.unwrap().join().unwrap();
```

Tratti della concorrenza

Rust introduce due tratti marcatori (che non hanno metodi associati) al fine di fornire indicazioni sul comportamento di un certo tipo in ambito multi-thread. Il tratto **Send** è applicato su tutti i tipi il cui possesso può essere trasferito, in sicurezza, da un thread ad un altro. Questo è garantito dal fatto che non è possibile avere accessi contemporanei al loro contenuto. Il tratto **Sync** è applicato a quei tipi `T` che possono essere condivisi, per riferimento, tra thread differenti (in pratica `&T` gode del tratto `Send`, quindi la lettura garantisce l'assenza di comportamenti non definiti).

N.B. Puntatori e riferimenti non hanno il tratto `Send`.

Un tipo composto (struct, array, ecc.) gode del tratto `Send` se tutti i suoi campi godono di tale tratto.

N.B. Se un tipo gode del tratto `Sync`, è lecito passarlo come riferimento non mutabile ad altri thread.

INFO: I tipi `Cell`, `RefCell` e `Rc` non godono del tratto `Sync` e non possono essere usati nel contesto multi-thread. Lo smart pointer **Arc** è un `Rc` che utilizza un'operazione atomica per incrementare il contatore interno delle copie, per cui può essere utilizzato su più thread.

In questo esempio vengono creati dei cloni di `Cell`: ognuno di essi, però, è trasferito ad un thread, per cui non sono utilizzati in contemporanea da thread diversi.

```
use std::cell::Cell;
use std::thread;

fn main() {
    // Crea un Cell contenente un intero
    let my_cell = Cell::new(42);

    // Clona il Cell per entrambi i thread
    let thread1_cell = my_cell.clone();
    let thread2_cell = my_cell.clone();

    // Thread 1: Incrementa il valore nel Cell
    let handle1 = thread::spawn(move || {
        thread1_cell.set(thread1_cell.get() + 1);
        println!("Thread1: Valore nel Cell: {}", thread1_cell.get());
    });

    // Thread 2: Legge il valore dal Cell
    let handle2 = thread::spawn(move || {
        println!("Thread2: Valore nel Cell: {}", thread2_cell.get()-1);
    });
    println!("Main Thread: Valore nel Cell: {}", my_cell.get());
    // Attendi che entrambi i thread terminino
    handle1.join().unwrap();
    handle2.join().unwrap();
}
```



Qui la `move` provoca, in realtà, una copia della variabile condivisa:

```
fn main() {
    // Creiamo una variabile condivisa tra più thread.
    let shared_value = 42;

    // Creiamo due thread che accedono alla variabile condivisa
    let handle1 = std::thread::spawn(move || {
        println!("Thread 1: Valore condiviso = {}", shared_value);
        let a = shared_value + 1;
        println!("Thread 1: Valore calcolato = {}", a);
    });

    let handle2 = std::thread::spawn(move || {
        println!("Thread 2: Valore condiviso = {}", shared_value);
        let b = shared_value + 10;
        println!("Thread 2: Valore calcolato = {}", b);
    });

    handle1.join().unwrap();
    handle2.join().unwrap();
}
```



```
use std::thread;

// Definiamo una struttura condivisa tra thread
#[derive(Clone, Copy)]
struct SharedData {
    value: u32,
}

fn main() {
    // Creiamo una variabile condivisa tra thread
    let mut shared_data = SharedData { value: 42 };

    // Creiamo un thread che accede alla variabile condivisa
    let handle = thread::spawn(move || {
        shared_data.value += 1;
        println!("Il valore condiviso è: {}", shared_data.value);
    });

    // Attendiamo il completamento del thread
    handle.join().unwrap();
    println!("Il valore condiviso è: {}", shared_data.value);
}
```



Per trasferire un dato che ha più possessori bisogna usare il tipo Arc.

```
use std::thread;
use std::sync::Arc;

fn main() {
    let data1 = Arc::new(1);
    let data2 = data1.clone();
    let data3 = data1.clone();
    println!("t0: {}", *data1);

    let jh1 = thread::spawn(move || {
        println!("t1: {}", *data2);
    });
    let jh2 = thread::spawn(move || {
        println!("t2: {}", *data3);
    });
    jh1.join().unwrap();
    jh2.join().unwrap();
}
```

Scoped Thread

La chiusura fornita ad un thread lanciato con la funzione scope accetta un parametro in grado di racchiudere l'intero ciclo di vita dei thread creati al suo interno. Tale parametro fornisce il metodo spawn. Terminata l'esecuzione della funzione lambda, la funzione scope non ritorna fin quando tutti i thread creati all'interno non sono terminati.

Questo comportamento permette al borrow checker di considerare corretto l'uso di riferimenti a variabili locali, dato che la loro durata sarà pari, almeno, a quella della funzione scope.

N.B. Il passaggio di variabili per riferimento tra funzione esterna e funzione lambda non è possibile nel caso di thread creati direttamente con spawn. Per questo si utilizzano gli scoped.

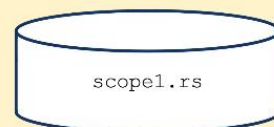
N.B. Non si utilizza move, perché non è necessario catturare la variabile locale. Il compilatore sa che il tempo di vita è limitato allo scope definito. Inoltre, non è necessario il metodo join, perché scope impedisce di proseguire fin quando i thread interni non sono terminati.

```
use std::thread;

fn main() {
    let numbers = vec![1, 2, 3];

    thread::scope(|s| {
        s.spawn(|| {
            println!("length: {}", numbers.len());
            // riferimento a variabile locale che non deve essere catturata con move
        });

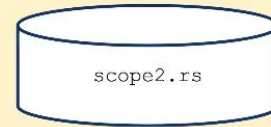
        s.spawn(|| {
            for n in &numbers {
                println!("{}", n);
            }
        });
    });
}
```



Le variabili libere intercettate dalle chiusure vengono utilizzate dai thread senza rendere impossibile il loro utilizzo una volta usciti dallo scope.

```
use std::thread;
fn main()
{
    let mut v = vec![1, 2, 3];
    let mut x = 0;
    thread::scope(|s| {
        s.spawn(|| { // è lecito creare un riferimento a v
            println!("length: {}", v.len());
        });
        s.spawn(|| { // anche qui viene catturato &v
            for n in &v {println!("{n}"); }
            x += v[0]+v[2]; // x è catturata come &mut
        });
    });

    // Solo quando entrambi i thread saranno terminati si proseguirà
    v.push(4); // non ci sono più riferimenti, si può modificare
    println!("Vettore: {:?}", v);
    println!("x: {:?}", x);
}
```



Modelli di concorrenza

Nel momento in cui si vogliono condividere delle risorse in scrittura, bisogna aggiungere delle sovrastrutture di controllo. Rust fornisce due modelli per la realizzazione di programmi concorrenti.

Il primo modello prevede una sincronizzazione degli accessi ad una struttura dati condivisa, a cui accedere in lettura e in scrittura. Il secondo, invece, esclude la condivisione di una struttura, permettendo uno scambio di informazione attraverso dei messaggi.

MUTEX

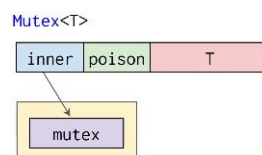
Per poter condividere dati modificabili tra thread differenti occorre un sistema che permetta di acquisire il permesso di modifica uno alla volta. Il **Mutex** (Mutual Exclusion Lock) è un oggetto del sistema operativo che offre i metodi **lock()** e **unlock()**, con i quali un thread può entrarne in possesso.

Quando il mutex viene richiesto da un thread ma è già occupato, l'invocazione viene messa in attesa.

INFO: L'unlock non deve essere esplicitato: la risorsa viene liberata automaticamente nel momento in cui lo smart pointer esce dal suo scope.

N.B. Il possesso del mutex non può essere condiviso, per cui se deve essere usato da più thread deve essere incapsulato in un Arc.

In Rust il mutex, in quanto smart pointer, incapsula il dato. Il metodo **lock()** restituisce un oggetto di tipo **LockResult<MutexGuard<T>>**.



Il campo **poison** è un flag e permette di stabilire se un'operazione eseguita sul dato ha causato un errore. In questo caso il mutex non è stato liberato, perché l'ultimo thread è stato terminato prima di farlo.

In questo esempio vengono generati 10 thread che utilizzano un dato contenuto in un mutex. Poiché ognuno di essi esegue un lock sul contenuto protetto, questo viene riservato finché non vengono concluse le operazioni del singolo thread, per poi essere liberato e reso disponibile agli altri thread.

```
use std::sync::Mutex;
use std::thread;

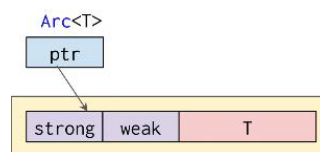
fn main() {
    let n = Mutex::new(0);
    thread::scope(|s| {
        for _ in 0..10 {
            s.spawn(|| {
                let mut guard = n.lock().unwrap();
                for _ in 0..100 {
                    *guard += 1;
                }

                println!("Alla fine del thread:{:?} n = {:?}",
                        thread::current().id(), guard);
            });
        }
    });
}
```



N.B. In questo caso non è stato necessario l'impiego di un Arc perché si è utilizzato lo scope().

L'Arc permette di avere più puntatori con possesso al mutex.



L'arc originale viene clonato, in modo tale da fornire un clone a ogni thread che deve utilizzarlo. Ognuno di essi punta sempre allo stesso Arc (e quindi allo stesso mutex), per cui i thread operano sullo stesso valore condiviso.

```
use std::sync::Mutex;
use std::sync::Arc;
use std::thread;

fn main() {
    // Creiamo un vettore condiviso protetto da un mutex
    let shared_data = Arc::new(Mutex::new(Vec::new()));

    // Cloniamo l'Arc per poterlo condividere tra più thread
    let shared_data_clone = Arc::clone(&shared_data);

    // Creiamo un thread per aggiungere dati al vettore condiviso
    let handle1 = thread::spawn(move || {
        let mut data = shared_data_clone.lock().unwrap();
        data.push(1);
    });

    // Cloniamo nuovamente l'Arc per un altro thread
    let shared_data_clone = Arc::clone(&shared_data);
    // Creiamo un secondo thread per aggiungere dati al vettore condiviso
    let handle2 = thread::spawn(move || {
        let mut data = shared_data_clone.lock().unwrap();
        data.push(2);
    });

    handle1.join().unwrap();
    handle2.join().unwrap();
    println!("{:?}", shared_data.lock().unwrap()); // Stampa il vettore condiviso
}
```



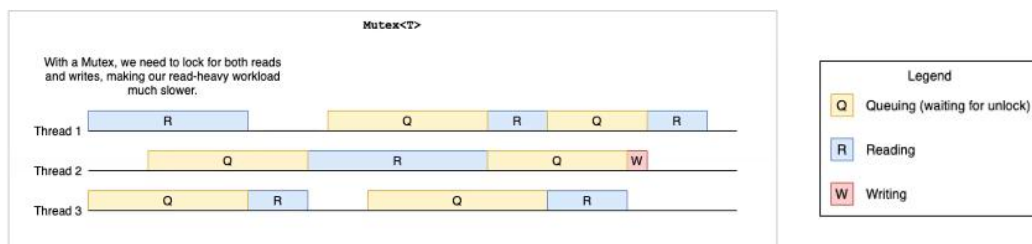
Questo esempio simula un errore in un thread: in questo caso il mutex risulta avvelenato, perché non liberato correttamente. La funzione di lock eseguita nel secondo thread lancia a sua volta un errore, perché il dato condiviso risulta corrotto.

```
use std::sync::{Arc, Mutex};
use std::thread;
use std::time::Duration;
fn main() {
    let data = Arc::new(Mutex::new(0));
    // Cloniamo l'arc per ogni thread in modo che ciascuno abbia un riferimento condiviso
    let data_clone1 = Arc::clone(&data);
    let data_clone2 = Arc::clone(&data);
    let incrementer = thread::spawn(move || { // Thread che incrementa il valore all'interno del Mutex
        let mut guard = data_clone1.lock().unwrap();
        *guard += 1;
        panic!("Oops, ho fatto un errore!"); // Simuliamo un errore nel thread (ad esempio, un panic)
    });
    let attempt_incrementer = thread::spawn(move || { // Thread che tenta di incrementare il valore dopo il panic
        // Ritardiamo l'esecuzione del thread di 1 secondo
        thread::sleep(Duration::from_secs(1));
        // Tentiamo di ottenere un blocco sul Mutex, ma il Mutex è nel
        // "poisoned state" a causa del panic del thread precedente
        let result = data_clone2.lock();
        match result {
            Ok(mut guard) => {
                *guard += 1;
                println!("Valore incrementato con successo dopo il poison: {}", *guard);
            }
            Err(poison_error) => {
                // Gestiamo l'errore derivante dal Mutex in stato "poisoned"
                println!("Errore: Il Mutex è stato avvelenato.");
                println!("Informazioni sull'errore: {:?}", poison_error);
            }
        }
    });
    incrementer.join().unwrap_err(); // Gestisce il panic del thread incrementer
    attempt_incrementer.join().unwrap();
    println!("Fine del main thread");
}
```

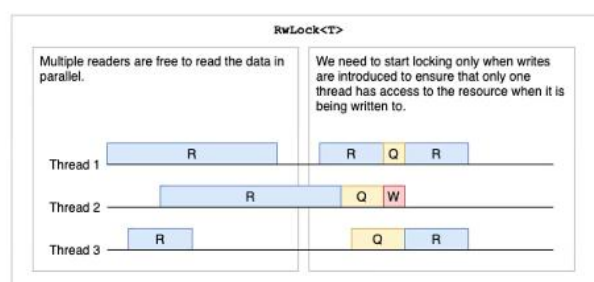


RwLock

L'utilizzo di un Mutex causa l'accesso esclusivo alla risorsa da parte dei thread. Questo è dannoso nel caso in cui la maggior parte degli accessi sono solo in lettura, perché impedisce che tali operazioni possano essere svolte in parallelo.



La struttura RwLock permette di effettuare una lock solo in caso di scrittura: solo in questo caso l'accesso al dato è esclusivo, mentre durante le letture può essere utilizzato in contemporanea da thread diversi.



I metodi forniti dallo smart pointer sono **read()** e **write()**, che restituiscono rispettivamente oggetti `LockResult<RwLockReadGuard>` e `LockResult<RwLockWriteGuard>`. Anche in questo caso, se il lock è avvelenato, il risultato ritorna un errore.

In questo esempio i thread accedono solo in lettura all'`Arc`, per cui l'accesso può avvenire in parallelo.

```
use std::sync::{Arc, RwLock};
use std::thread;

fn main() {
    // Creiamo una struttura dati condivisa protetta da un RwLock
    let data = Arc::new(RwLock::new(vec![1, 2, 3, 4, 5]));

    // Creiamo 10 thread che leggono dalla struttura dati condivisa
    let mut threads = vec![];

    for i in 0..10 {
        // Cloniamo l'arc per ogni thread in modo che ciascuno abbia un riferimento condiviso
        let data_clone = Arc::clone(&data);

        // Creiamo il thread
        let thread = thread::spawn(move || {
            // Otteniamo un blocco di lettura (non esclusivo) sul RwLock
            let guard = data_clone.read().unwrap();
            println!("Thread {}: {:?}", i, *guard);
        });

        threads.push(thread);
    }

    // Attendo che tutti i thread terminino
    for thread in threads {
        thread.join().unwrap();
    }
}
```



In questo caso invece la lock in scrittura può avvenire solo se nessun altro sta accedendo in lettura o scrittura al contenuto dell'`RwLock`.

```
use std::sync::{Arc, RwLock};
use std::thread;

fn main() {
    // Creiamo una struttura dati condivisa protetta da un RwLock
    let data = Arc::new(RwLock::new(vec![1, 2, 3]));

    // Cloniamo l'arc per ogni thread in modo che ciascuno abbia un riferimento condiviso
    let data_clone1 = Arc::clone(&data);
    let data_clone2 = Arc::clone(&data);

    // Thread che legge dalla struttura dati condivisa
    let reader = thread::spawn(move || {
        // Otteniamo un blocco di lettura (non esclusivo) sul RwLock
        let guard = data_clone1.read().unwrap();
        println!("Thread lettore: {:?}", guard);
    });

    // Thread che scrive sulla struttura dati condivisa
    let writer = thread::spawn(move || {
        // Otteniamo un blocco di scrittura (esclusivo) sul RwLock
        let mut guard = data_clone2.write().unwrap();
        guard.push(4);
        println!("Thread scrittore: {:?}", guard);
    });

    // Attendo che entrambi i thread terminino
    reader.join().unwrap();
    writer.join().unwrap();
}
```



Tipi atomici

Rust fornisce alcune strutture che permettono di sincronizzare gli aggiornamenti sui propri valori in modo indivisibile, tra thread diversi. Tra questi ci sono `AtomicBool`, `AtomicUsize`, `AtomicI32`.

Le operazioni disponibili sono la `load` (lettura), lo `store` (scrittura), operazioni aritmetiche come `fetch_add`, `fetch_sub`, operazioni logiche come `fetch_and`.

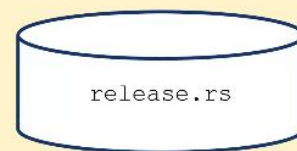
INFO: In alcuni contesti è sufficiente questo tipo di protezione.

N.B. Le operazioni di lettura e scrittura richiedono che sia specificato l'ordering, un tipo di dato che possiede le varianti `Relaxed`, `Release`, `Acquire`, `AcqRel`, `SeqCst`. Questi definiscono diversi modi con cui gestire l'ordinamento delle operazioni effettuate.

In questo esempio viene creato un thread produttore ed un consumatore. Il primo inserisce dei dati all'interno di un buffer, mentre il secondo li estrae consumandoli. Per sapere se il produttore ha finito di inserire i dati, viene impiegato l'uso di un booleano atomico. In questo caso, infatti, è sufficiente garantire che le operazioni di aggiornamento di questo flag siano consistenti e non lasciate a metà, senza dover ricorrere a strutture più complesse per assicurare la correttezza del dato.

```
use std::sync::{Arc, Mutex};
use std::sync::atomic::{AtomicBool, Ordering};
use std::thread;
const BUFFER_SIZE: usize = 10;

fn main() {
    let buffer = Arc::new(Mutex::new(Vec::with_capacity(BUFFER_SIZE)));
    let producer_finished = Arc::new(AtomicBool::new(false));
    let buffer_clone = Arc::clone(&buffer);
    let producer_finished_clone = Arc::clone(&producer_finished);
    let producer_handle = thread::spawn(move || {
        for i in 0..10 {
            thread::sleep(std::time::Duration::from_nanos(10));
            let mut buffer = buffer_clone.lock().unwrap();
            buffer.push(i); // Aggiungi un valore al buffer
            println!("Produttore: prodotto {}", i);
        }
        producer_finished_clone.store(true, Ordering::Release);
    });
    let buffer_clone = Arc::clone(&buffer);
    let consumer_handle = thread::spawn(move || {
        loop {
            let mut buffer = buffer_clone.lock().unwrap();
            let len = buffer.len();
            if len > 0 {
                let value = buffer.remove(0); // Se ci sono elementi nel buffer, consumane uno
                println!("Consumatore: consumato {}", value);
            } else if producer_finished.load(Ordering::Acquire) {
                break; // Se il produttore ha finito, esce dal ciclo
            }
            // Altrimenti, rilascia il lock e attendi finché ci sono elementi nel buffer o il produttore ha finito
        }
        println!("Tutti gli elementi sono stati consumati");
    });
    producer_handle.join().unwrap();
    consumer_handle.join().unwrap();
}
```



INFO: La variante **Release** si applica alle operazioni di scrittura e determina che le azioni non atomiche svolte prima di quella atomica non vengano riordinate dopo di essa. La variante **Acquire** si applica alle operazioni di lettura e determina che le azioni non atomiche svolte dopo l'operazione atomica non vengano riordinate prima di essa. L'**AcqRel** si applica ad operazioni che sono sia di scrittura che di lettura e garantisce le due precedenti. La `SeqCst` impone che i thread vedano tutte le operazioni sequenziali nello stesso ordine. La `Relaxed`, invece, non ha vincoli.

Il riordinamento è spesso effettuato dal compilatore, per motivi di ottimizzazione. Gli effetti negativi sono assenti nel caso in cui si lavori con un singolo thread, ma ciò non vale se i dati sono condivisi.

VARIABILI GLOBALI

Le variabili globali non possono essere modificate. Questa regola, tuttavia, ha una eccezione che è quella delle variabili atomiche.

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;
static COUNTER: AtomicUsize = AtomicUsize::new(0);

fn increment_counter() {
    let new_value = COUNTER.fetch_add(1, Ordering::Relaxed);
    println!("Thread ID: {:?}, New value of COUNTER: {}",
std::thread::current().id(), new_value);
}

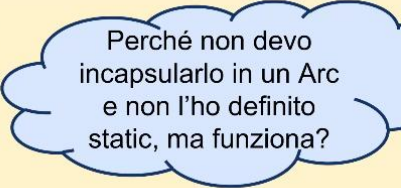
fn main() {
    let mut handles = vec![];
    // Creare 10 thread e farli eseguire la funzione increment_counter
    for _ in 0..10 {
        let handle = thread::spawn(increment_counter);
        handles.push(handle);
    }
    // Attendere la terminazione dei thread
    for handle in handles {
        handle.join().unwrap();
    }
    println!("Final value of COUNTER: {}", COUNTER.load(Ordering::Relaxed));
    // Output: Final value of COUNTER: 10
}
```



INFO: Le variabili globali vengono, per prassi, dichiarate con le lettere maiuscole. In caso contrario il compilatore restituisce un warning.

```
use std::time::Duration;
use std::sync::atomic::Ordering::Relaxed;
use std::thread;
use std::sync::atomic::AtomicUsize;

fn main() {
    let num_done = AtomicUsize::new(0);
    thread::scope(|s| {
        // A background thread to process all 100 items.
        s.spawn(|| {
            for i in 0..100 {
                println!("{}", i);
                num_done.store(i + 1, Relaxed);
            }
        });
        // The main thread shows status updates, every 10 nanoseconds.
        loop {
            let n = num_done.load(Relaxed);
            if n == 100 { break; }
            println!("Working.. {n}/100 done");
            thread::sleep(Duration::from_nanos(10));
        }
    });
    println!("Done!");
}
```



In questo caso la variabile viene condivisa senza essere globale nè essere inglobata in un Arc. Ciò è possibile grazie allo scope, che definisce un blocco preciso entro cui quella variabile può essere usata, per poi ritornare al main.

Weak

Così come per gli Rc, anche gli Arc impediscono la creazione di catene circolari. Per questo motivo è disponibile la struct Weak (in questo caso `std::sync::Weak<T>`, a differenza delle `std::rc::Weak<T>`) che offre le stesse funzionalità viste precedentemente. Fornisce il metodo `upgrade()`, che ritorna un `Option<Arc<T>>` e un `downgrade()`.

CONDITION VARIABLE

La condition variable è una condizione che deve essere verificata affinché due thread si sincronizzino. Vengono sfruttate quando un thread deve aspettare alcuni risultati intermedi, svolti da qualcun altro. Queste strutture dati permettono di bloccare l'esecuzione di un thread, così da evitare il consumo di CPU durante l'attesa.

INFO: Una strada alternativa potrebbe essere quella del polling, in cui il thread che aspetta controlla ripetutamente una variabile condivisa. Ciò comporta però uno spreco di capacità di calcolo e la latenza tra il momento in cui il dato è disponibile e quello in cui il thread si sblocca.

Il sistema operativo mette in stato di "not running" il thread in attesa, allo stesso modo di come accade per le operazioni di input/output, lettura sul disco, ecc. La funzione che mette in attesa un thread è **wait()**, mentre quella che serve a svegliarlo è la **notify()**. Quest'ultima deve essere eseguita dal thread che consegna l'elaborazione richiesta.

INFO: La funzione di **notify_one()** sveglia un thread in attesa, ma non è possibile sapere quale di preciso, non essendo qualcosa di deterministico, ma a discrezione del sistema operativo. Si consiglia, dunque, di usare **notify_all()**, per svegliare tutti i thread che aspettano la condizione in questione.

N.B. Una condition variable si abbina sempre ad un **mutex**, che garantisce la protezione del dato e la mutua esclusione.

L'uscita dalla wait provoca un lock sul mutex, per prendere possesso del risultato della funzione lanciata. Rust mette a disposizione la struttura **Condvar**, che facilita il collegamento con il dato del mutex.

```
use std::sync::{Arc, Mutex, Condvar};
use std::thread;
use std::time::Duration;

fn main() {
    // Create an Arc pair containing a Mutex and a Condvar
    let pair = Arc::new( (Mutex::new(false), Condvar::new()) );
    let pair2 = Arc::clone(&pair);

    // Inside our lock, spawn a new thread and wait for it to start
    thread::spawn(move || {
        let (lock, cvar) = &*pair2;
        let mut started = lock.lock().unwrap();
        thread::sleep(Duration::from_secs(5));
        *started = true; // We notify the Condvar that the value has changed
        cvar.notify_one();
    });

    // Wait for the thread to start up
    let (lock, cvar) = &*pair;
    let mut started = lock.lock().unwrap();

    println!("Waiting ...");
    started = cvar.wait(started).unwrap();
    println!("Thread started!");
}
```



N.B. Valorizzare la condition variable non è fondamentale: per risvegliare il thread basta la notify.

N.B. Può succedere che la notify arrivi prima di un wait: in questo caso, non essendoci ancora il thread in fase di not running, la notifica viene persa e con essa la possibilità, successiva, di potersi svegliare.

È buona norma verificare che l'operazione richiesta non sia stata ancora eseguita, prima di lanciare un wait.

```
use std::sync::{Arc, Mutex, Condvar};
use std::thread;
fn main() {
    const NUM_THREADS: usize = 5;
    let data = Arc::new((Mutex::new(0), Condvar::new()));
    let mut handles = vec![];
    for i in 0..NUM_THREADS {
        let data_clone = Arc::clone(&data);
        let handle = thread::spawn(move || {
            let (lock, cvar) = &*data_clone;
            let mut data_guard = lock.lock().unwrap();
            println!("Thread {} in attesa...", i);
            // Attendi fino a quando il dato non è diverso da 0
            if *data_guard == 0 { // dopo capiremo che e' meglio avere while come e' nella versione che trovate in rete
                data_guard = cvar.wait(data_guard).unwrap();
            }
            println!("Thread {} svegliato! Il dato è: {}", i, *data_guard);
        });
        handles.push(handle);
    }
    // Facciamo avanzare i thread dopo 2 secondi
    thread::sleep(std::time::Duration::from_secs(2));
    // Cambiamo il dato condiviso e svegliamo tutti i thread
    {
        let (lock, cvar) = &*data;
        let mut data_guard = lock.lock().unwrap();
        *data_guard = 42;
        cvar.notify_all();
    }
    for handle in handles {
        handle.join().unwrap();
    }
}
```



La wait si utilizza come metodo della condition variable e si passa come parametro la variabile di interesse (il mutex).

N.B. Un thread può essere risvegliato casualmente dalle **notifiche spurie**. Al ritorno del wait occorre comunque controllare la condizione attesa e per questo sono fornite alcune funzioni di attesa (**wait_while**) che prendono come argomento una condizione da verificare.

```
use std::{
    sync::{Arc, Condvar, Mutex},
    thread::{sleep,},
    time::Duration,
};
struct Counter {
    value: Mutex<u32>,
    condvar: Condvar,
}
fn main() {
    let counter = Arc::new(Counter {
        value: Mutex::new(0),
        condvar: Condvar::new(),
    });
    let counter_clone = counter.clone();
    let counting_thread = std::thread::spawn(move || loop {
        sleep(Duration::from_millis(100));
        let mut value = counter_clone.value.lock().unwrap();
        *value += 1;
        counter_clone.condvar.notify_all();
        if *value >= 15 {
            break;
        }
    });
    // Wait until the value more or equal to 15
    let mut value = counter.value.lock().unwrap();
    value = counter.condvar.wait_while(value, |val| *val < 15).unwrap();
    println!("Condition met. Value is now {}. ", *value);
    // Wait for counting thread to finish
    counting_thread.join().unwrap();
}
```



Al posto della wait_while può essere anche utilizzato un semplice while sulla condizione richiesta. Non è consigliato usare l'if, come nello scorso esempio.

Il metodo **wait_timeout** permette di dare un tempo massimo all'attesa, permettendo al thread di risvegliarsi anche senza che venga lanciato un notify. Il metodo restituisce un Result contenente la tupla (MutexGuard<'a, T>, WaitTimeoutResult). In ogni caso, il possesso del mutex ritorna al thread principale.

È possibile anche usare **wait_timeout_while**, il quale prende come parametro una condizione che se falsa porta il thread a riaddormentarsi, anche se c'è stato il timeout.

```
use std::sync::{Arc, Mutex, Condvar};
use std::thread;
use std::time::Duration;
struct SharedData {
    mutex: Mutex<bool>,
    cv: Condvar
}
impl SharedData {
    //Metodo costruttore
    pub fn new(condition: bool) -> Self {
        SharedData { mutex: Mutex::new(condition), cv: Condvar::new() }
    }
    pub fn change_and_notify(&self) {
        let mut data = self.mutex.lock().unwrap();
        *data = true;
        //Mandiamo la notifica alla condvar
        self.cv.notify_one();
    }
    pub fn looper(&self) {
        loop {
            //Il thread aspetta per una notifica. Nel caso siano passati 100 milisecondi smette di aspettare
            let lock: (MutexGuard<bool>, WaitTimeoutResult) = self.cv.wait_timeout(
                self.mutex.lock().unwrap(), Duration::from_millis(100)).unwrap();
            if *lock.0 == true {
                //Il thread ha ricevuto una notifica dato che il valore è stato cambiato, quindi esco
                print!("Il valore è cambiato quindi posso uscire dal ciclo correttamente ");
                if !lock.1.timed_out() {
                    println!("e non c'è stato time-out");
                }
                break
            }
            if lock.1.timed_out() {
                println!("E' scaduto il timer ma il valore non è cambiato. Ricomincio il ciclo");
            }
        }
    }
}

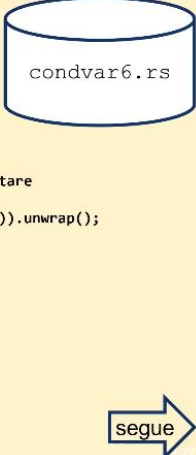
fn main() {
    let shared = Arc::new(SharedData::new(false));
    let shared2 = Arc::clone(&shared);

    let mut handles = vec![]; //vettore dei threads creati per poi fare le dovute join

    handles.push(thread::spawn(move || {
        shared2.looper();
    }));

    handles.push(thread::spawn(move || {
        //Aspetto prima di mandare la notifica
        thread::sleep(Duration::from_secs(1));
        shared.change_and_notify();
    }));

    //Join finali
    for handle in handles {
        handle.join().expect("Error");
    }
}
```




Modelli di concorrenza (scambio di messaggi su canale)

Quello che è stato visto finora è un modello che crea delle **strutture condivise** e protette dai mutex e dalle condition variable. Il principio, quindi, è che ci si sincronizza per comunicare.

Lo **scambio di messaggi**, invece, ha come scopo quello di comunicare per sincronizzarsi. Rust fornisce un **channel** che permette il trasferimento di messaggi (multiple producer, single consumer). L'informazione viene immessa all'interno del "tubo" e rimane lì fin quando il ricevitore non lo legge acquisendone il possesso.

INFO: Il canale è unbounded, quindi non ha dimensione massima, e mantiene l'ordine dei messaggi.

La trasmissione viene conclusa quando il sender è distrutto. Il ricevitore, in questo caso, riceve un errore. Se nel canale sono ancora presenti dei messaggi, questi possono comunque essere ancora letti. Se, al contrario, ad essere distrutto è il ricevitore allora i messaggi nel canale vengono persi.

La funzione **channel()** restituisce la tuple contenente le struct **Sender** e **Receiver**. I dati vengono mandati col metodo **send()** e ricevuto con **recv()**. Il sistema permette al trasmettitore di inviare un numero infinito di messaggi e al ricevitore di bloccarsi senza consumare cicli macchina. La **recv()** restituisce un **Result**.

N.B. Per lavorare con un sender su un thread, questo deve essere clonato.

```
use std::sync::mpsc::channel;
use std::thread;

fn main() {
    let (tx, rx) = channel();

    for _ in 0..3 {
        let tx = tx.clone();
        // cloned tx dropped within thread
        thread::spawn(move || tx.send("ok").unwrap());
    }

    // Drop the last sender to stop `rx` waiting for message.
    // The program will not complete if we comment this out.
    // **All** `tx` needs to be dropped for `rx` to have `Err`.
    drop(tx);

    // Unbounded receiver waiting for all senders to complete.
    while let Ok(msg) = rx.recv() {
        println!("{}", msg);
    }
}
```



N.B. I produttori devono essere tutti distrutti per non finire in un ciclo infinito. Il canale, infatti, viene eliminato solo quando vengono distrutti i sender.

INFO: Il ricevitore non conosce i trasmettitori.

Canali sincroni

Il canale può anche essere limitato. In questo caso si usa la funzione **sync_channel** che prende come parametro un bound usize. Quando il numero di messaggi nel canale raggiunge il limite, le chiamate **send** diventano bloccanti fin quando non viene liberato un posto.

INFO: Se il canale ha dimensione 0 si ottiene un **rendezvous**: in questo caso le letture devono sovrapporsi temporalmente alle scritture.

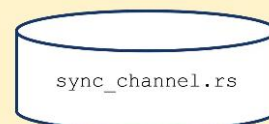
```
use std::sync::mpsc::sync_channel;
use std::thread;

fn main() {

    let (sender, receiver) = sync_channel(1);
    // this returns immediately

    sender.send(1).unwrap();
    thread::spawn(move || {
        // this will block until the previous message has been received
        sender.send(2).unwrap();
    });

    println!("{}", receiver.recv().unwrap());
    println!("{}", receiver.recv().unwrap());
}
```



N.B. Con il rendezvous la funzione di send viene “bloccata” fin quando non viene eseguita la recv.

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (sender, receiver) = mpsc::sync_channel(0);

    // Questo thread tenta di inviare un valore al canale, ma essendo la capacità 0,
    // si bloccherà fino a quando il valore non verrà letto dal receiver.
    thread::spawn(move || {
        sender.send("Sto trasmettendo un messaggio e ci aspettiamo
                    all'appuntamento").unwrap();

        println!("Valore inviato.");
    });

    // Il receiver legge il valore dal canale.
    let received_value = receiver.recv().unwrap();
    println!("Valore ricevuto: {}", received_value);
}
```



Barriera

Le barriere sono strumenti di sincronizzazione fra thread. Esse permettono ai thread di attendersi l'un l'altro in un determinato punto, attraverso la funzione **wait()**.

```
fn main() {
    let num_threads = 10;

    // Crea una barriera per sincronizzare num_threads thread
    let barrier = Arc::new(Barrier::new(num_threads));

    let mut handles = Vec::with_capacity(num_threads);

    for i in 0..num_threads {
        let barrier_clone = Arc::clone(&barrier);
        let handle = thread::spawn(move || {
            println!("Thread {} sta facendo un po' di lavoro", i);

            // Simula il lavoro con una sleep crescente per thread
            thread::sleep(std::time::Duration::from_secs((i+1).try_into().unwrap()));

            println!("Thread {} è arrivato alla barriera", i);
            // Attende che tutti i thread raggiungano la barriera
            barrier_clone.wait();

            println!("Thread {} ha superato la barriera", i);
        });
        handles.push(handle);
    }

    for handle in handles { // Attende che tutti i thread completino il loro lavoro
        handle.join().unwrap();
    }
}
```



Crossbeam

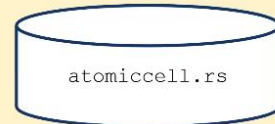
È una libreria che offre costrutti utili alla programmazione concorrente. In particolare fornisce dei costrutti atomici, come ad esempio **AtomicCell** che estende la mutabilità interna di Cell al contesto multithread, strutture dati concorrenti e canali multiple producer multiple consumer.

```
fn main() {
    let cell = Arc::new(AtomicCell::new(0));

    // Crea due thread che incrementano il valore nella cella
    let handles: Vec<JoinHandle<>> = (0..4).map(|_| {
        let cell = Arc::clone(&cell);
        std::thread::spawn(move || {
            for _ in 0..1000 {
                cell.fetch_add(1);
            }
        })
    }).collect();

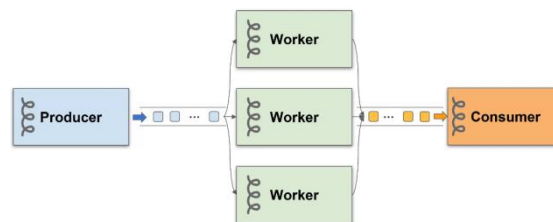
    // Attendi che entrambi i thread abbiano terminato
    for handle in handles {
        handle.join().unwrap();
    }

    // Stampa il valore finale nella cella
    println!("Final value: {}", cell.load());
}
```



Paradigma MPMC permette l'implementazione di 3 pattern concorrenti.

Il **Fan-out/Fan-in** permette di distribuire le attività a più thread indipendenti, per poi raccogliere i risultati prodotti in un singolo punto. Utilizza una coppia di canali per distribuire e raccogliere i dati.



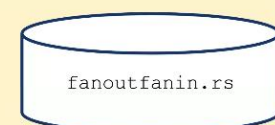
INFO: I canali limitati sono creati attraverso la funzione **bounded**. Esiste anche **unbounded**.

```
use std::thread;
use crossbeam_channel::{bounded, Receiver, Sender};

fn worker(id: usize, rx: Receiver<i32>, tx: Sender<String>) {
    while let Ok(value) = rx.recv() {
        tx.send(format!("W{} {}", id, value)).unwrap();
    }
}

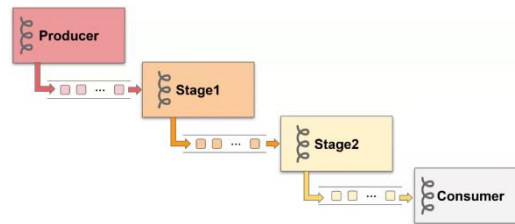
fn main() {
    let (tx_input, rx_input) = bounded<i32>(10);
    let (tx_output, rx_output) = bounded<String>(10);
    let mut worker_handles = Vec::new();
    for i in 0..3 {
        let rx = rx_input.clone();
        let tx = tx_output.clone();
        worker_handles.push(thread::spawn(move || worker(i, rx, tx)));
    }
    for i in 1..=10 {
        tx_input.send(i).unwrap();
    }
    drop(tx_input);

    while let Ok(result) = rx_output.recv() {
        println!("Received result: {}", result);
    }
    for handle in worker_handles { handle.join().unwrap(); }
}
```



N.B. I thread worker operano sia come receiver che come sender.

La **Pipeline** crea una serie di fasi di lavorazione, ognuna eseguita da un singolo thread. Utilizza un canale per inoltrare i semi-lavorati fra fasi successive.



```
use std::thread;
use crossbeam_channel::{bounded, Receiver, Sender};
fn stage_one(rx: Receiver<i32>, tx: Sender<String>) {
    while let Ok(value) = rx.recv() {
        tx.send(format!("S1({})", value)).unwrap();
    }
}
fn stage_two(rx: Receiver<String>, tx: Sender<String>) {
    while let Ok(value) = rx.recv() {
        tx.send(format!("S2( {} )", value)).unwrap();
    }
}
fn main() {
    let (tx_input, rx_input) = bounded::<i32>(10);
    let (tx_stage_one, rx_stage_one) = bounded::<String>(10);
    let (tx_output, rx_output) = bounded::<String>(10);

    let stage_one_handle = thread::spawn(move || stage_one(rx_input, tx_stage_one));
    let stage_two_handle = thread::spawn(move || stage_two(rx_stage_one, tx_output));

    for i in 1..=10 { tx_input.send(i).unwrap(); }
    drop(tx_input);

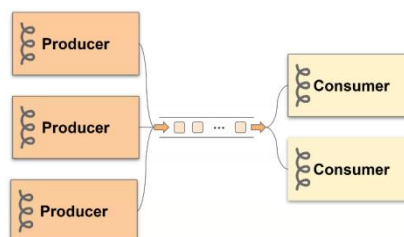
    while let Ok(result) = rx_output.recv() { println!("Received result: {}", result); }

    stage_one_handle.join().unwrap();
    stage_two_handle.join().unwrap();
}
```

pipeline.rs

N.B. I trasmettitori vengono distrutti uscendo dal thread, perché il possesso è stato fornito grazie alla move. Il drop è necessario, invece, su trasmettitore che è rimasto nel main.

Il **Producer/Consumer** consente ad uno o più thread produttori di generare valori da far elaborare al primo thread consumatore disponibile. Utilizza un solo canale di comunicazione.



```
use std::thread;
use crossbeam_channel::{bounded, Receiver, Sender};

fn producer(id: usize, tx: Sender<(usize,i32)>) {
    for i in 1..=5 { tx.send((id,i)).unwrap(); }
}

fn consumer(id: usize, rx: Receiver<(usize,i32)>) {
    while let Ok((sender_id, val)) = rx.recv() {
        println!("Consumer {} received {} from {}", id, val, sender_id);
    }
}

fn main() {
    let (tx, rx) = bounded::<(usize,i32)>(10);

    let mut handles = Vec::new();
    for i in 0..3 {
        let tx = tx.clone();
        handles.push(thread::spawn(move || producer(i, tx)));
    }
    for i in 0..2 {
        let rx = rx.clone();
        handles.push(thread::spawn(move || consumer(i, rx)));
    }
    drop(tx);
    for handle in handles { handle.join().unwrap(); }
}
```

producerconsumer.rs

After e Tick

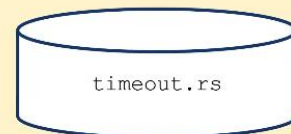
Creano l'estremo di ricezione che consegna un messaggio con un tempo indicato. Vengono usati per creare allarmi e timeout.

La funzione **after** crea un ricevitore che consegna un messaggio dopo un certo periodo. La funzione **tick** crea un ricevitore che consegna un messaggio con una data frequenza di trasmissione.

INFO: La macro **select!** fornita da crossbeam permette di ascoltare più canali contemporaneamente e di svolgere azioni in base a chi riceve il valore per primo.

```
use crossbeam_channel::select;
use crossbeam_channel;
use std::thread;
use std::time::{Duration, Instant};

fn main() {
    let (s, r) = channel::bounded(1);
    let after = channel::after(Duration::from_secs(5)); // Timeout dopo 5 secondi
    thread::spawn(move || {
        // Simula un'operazione di calcolo intensivo
        let start = Instant::now();
        while start.elapsed() < Duration::from_secs(8) {
            // Fa qualcosa di computazionalmente intenso
        }
        let _ = s.send("Operazione completata").unwrap();
    });
    select! {
        recv(r) -> msg => println!("{}", msg.unwrap()), // Riceve il messaggio
        recv(after) -> _ => println!("Timeout! Operazione non completata"), // Timeout
    }
    let (s, r) = channel::bounded(1);
    let after = channel::after(Duration::from_secs(5)); // Timeout dopo 5 secondi
    thread::spawn(move || {
        // Simula un'operazione di calcolo intensivo
        let start = Instant::now();
        while start.elapsed() < Duration::from_secs(1) {
            // Fa qualcosa di computazionalmente intenso
        }
        let _ = s.send("Operazione completata").unwrap();
    });
    select! {
        recv(r) -> msg => println!("{}", msg.unwrap()), // Riceve il messaggio
        recv(after) -> _ => println!("Timeout! Operazione non completata"), // Timeout
    }
}
```



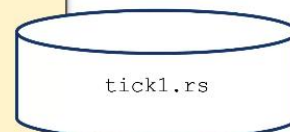
select! è un costrutto fornito dalla crate crossbeam che permette di ascoltare più canali contemporaneamente e di eseguire azioni basate su quale canale riceve un valore per primo.

Nell'esempio la prima select esegue solo le operazioni di `recv(after)`, perché è il canale su cui arriva prima un messaggio. Questo permette di aggiungere dei controlli basati su timeout (infatti il thread simula una lunga computazione, che fa superare il tempo del timeout).

```
use std::time::{Duration, Instant};
use crossbeam_channel::tick;

fn main()
{
    let start = Instant::now();
    let ticker = tick(Duration::from_millis(100));

    for _ in 0..5 {
        ticker.recv().unwrap();
        println!("Tempo trascorso: {:?}", start.elapsed());
    }
}
```



PROCESSI

Un processo costituisce l'unità base di esecuzione di un applicativo. Ognuno di essi ha uno spazio di indirizzamento all'interno del quale lavorano uno o più thread, e questo fornisce un meccanismo naturale di isolamento.

Rust utilizza la libreria `process` e la struct **Command** per offrire metodi e system call per la creazione di un nuovo processo. In particolare i metodi **arg()** e **args()** servono a passare al processo figlio uno o più argomenti, **output()** invece genera il processo e ne attende il **Result<Output>**.

```
pub struct Output {  
    pub status: ExitStatus,  
    pub stdout: Vec<u8>,  
    pub stderr: Vec<u8>,  
}
```

INFO: `ExitStatus` è una struttura che permette di aver informazioni su come è uscito il processo lanciato.

```
use std::process::Command;  
  
fn main() {  
    let output = if cfg!(target_os = "windows") {  
        Command::new("cmd")  
            .args(["/C", "echo hello"])  
            .output()  
            .expect("failed to execute process")  
    } else {  
        Command::new("sh")  
            .arg("-c")  
            .arg("echo hello")  
            .output()  
            .expect("failed to execute process")  
    };  
  
    println!("{:?}", output)  
    //Output { status: ExitStatus(unix_wait_status(0)), stdout: "hello\n", stderr: "" }  
}
```



I metodi **env()** ed **envs()** permettono di aggiungere o modificare le variabili d'ambiente del processo (che di default eredita dal processo padre). I metodi **env_remove()** e **env_clear()**, invece, permettono di eliminarle.

I flussi di ingresso e uscita possono essere regolati dai metodi **stdin()**, **stdout()** e **stderr()**. A questi è possibile passare una opzione: **inherit()** permette di ereditare il descrittore in uso dal processo padre, **piped()** crea una pipe monodirezionale padre-figlio, **null()** di ignorare il flusso.

```
use std::process::Stdio;  
use std::process::Command;  
  
fn main() {  
    let output = Command::new("ls")  
        .env("PATH", "/bin")  
        .stdout(Stdio::inherit())  
        .output()  
        .expect("ls command failed to start");  
}
```



In questo esempio viene aggiunta una variabile globale `PATH` ed ereditato lo stesso `stdout` del processo padre, quindi la stampa di `ls` viene fatta allo stesso modo. Se fosse stato inserito `null`, tutto il flusso in uscita sarebbe stato perso, venendo ridirezionato su `dev/null`.

Il metodo **status(&mut self)** permette di creare un processo, ma ritornando un **Result<ExitStatus>**. Non fornisce quindi lo stdout e lo stderr.

```
use std::process::Command;

fn main() {
    // Esegui il comando ls e ottieni il suo stato di uscita
    let child = Command::new("ls")
        .status()
        .expect("failed to execute process");

    // Stampa lo stato di uscita del processo
    if child.success() {
        println!("Process exited successfully with status: {}", child);
    } else {
        println!("Process exited with failure status: {}", child);
    }
}
```



Il metodo **spawn(&mut self)** crea un processo ma non ne aspetta la terminazione, a differenza dei due precedenti. Restituisce un **Result<Child>**.

INFO: Il metodo **output()** ha **stdin** e **stdout** di default a **piped()**, mentre **status()** e **spawn()** a **inherit()**.

La struttura **Child** offre diversi meccanismi per controllare il processo figlio. Il metodo **id()** restituisce il suo process id, il metodo **wait()** ne attende la terminazione, restituendo il codice di uscita. La funzione **wait_with_output()** chiude il flusso in ingresso, attende la terminazione e raccoglie quanto non ancora letto dei flussi di uscita ed errore nella struttura **Output**. Il metodo **kill()** ne forza la terminazione.

```
use std::process::Command;

fn main() {
    // Crea un'istanza di Command per il comando "echo" con l'argomento "Hello World"
    let mut process = Command::new("echo")
        .arg("Hello World")
        .spawn()
        .expect("Failed to spawn command");

    // Attendere che il processo figlio termini
    let exit_status = process.wait();

    // Verificare lo stato di uscita del processo figlio
    if exit_status.expect("Errore nel processo spawned").success() {
        println!("Child process has exited successfully.");
    }
}
```



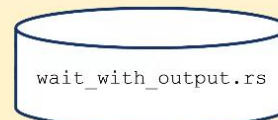
INFO: La **wait()** restituisce un **Result**, è possibile valutarne il successo attraverso **success()**.

```
use std::process::{Command, Stdio};

fn main() {
    let child_process = Command::new("ls")
        .arg("-l")
        .arg("-a")
        .stdout(Stdio::piped())
        .spawn()
        .expect("Impossibile avviare il processo 'ls'");

    let output = child_process.wait_with_output().expect("Impossibile ottenere l'output del processo");

    if output.status.success() {
        let stdout = String::from_utf8_lossy(&output.stdout);
        println!("Output del processo 'ls':\n{}", stdout);
    } else {
        let stderr = String::from_utf8_lossy(&output.stderr);
        println!("Errore durante l'esecuzione del processo 'ls': {}", stderr);
    }
}
```



INFO: La funzione **wait_with_output()** permette di acquisire l'output del processo, attraverso le proprietà **stdout** e **stderr**.

Quando vengono impostati gli stdin e stdout a piped, vengono creati dei canali di comunicazione tra il processo padre e quello figlio. Input e output vanno visti dal punto di vista del nuovo processo: attraverso process.stdin si può scrivere l'input da fornire al processo figlio, attraverso process.stdout se ne può leggere l'output (process è il valore di ritorno della Command::new()).

```
use std::io::prelude::*;
use std::process::{Command, Stdio};
fn main() {
    let process = match Command::new("rev")
        .stdin(Stdio::piped())
        .stdout(Stdio::piped())
        .spawn()
    {
        Err(err) => panic!("couldn't spawn rev: {}", err),
        Ok(process) => process,
    };
    match process.stdin.unwrap().write_all("isoc ovitrevid im non ehc innaaaa
onarE".as_bytes()) {
        Err(why) => panic!("couldn't write to stdin: {}", why),
        Ok(_) => println!("sent text to rev command"),
    }
    let mut child_output = String::new();
    match process.stdout.unwrap().read_to_string(&mut child_output) {
        Err(err) => panic!("couldn't read stdout: {}", err),
        Ok(_) => println!("Output from child process is:\n{}", child_output),
    }
}
```



La funzione **exit()** permette di terminare il processo corrente e tutti i suoi thread. Si può passare un parametro da fornire al sistema operativo (teoricamente 32, ma in pratica spesso solo gli 8 meto significativi) ed è responsabilità del programmatore assicurarsi che tutto sia stato liberato.

Anche la funzione **abort()** permette di terminare immediatamente il processo e i suoi thread, forzando però un codice di errore che viene interpretato come interruzione anomala.

La macro **panic!** causa la contrazione dello stack corrente, determinando la terminazione del processo solo se chiamata nel suo thread principale.

Orfani e zombie

Se un processo padre termina prima dei figli, questi ultimi diventano **orfani** e vengono riassegnati al processo init, con pid 1. Se, invece, un processo termina ma il padre non ha fatto una wait, il figlio diventa uno **zombie**. In questo caso la maggior parte delle risorse viene restituita al sistema operativo, tranne ID, stato di terminazione e alcuni contatori.

INFO: La fase di zombie può anche essere momentanea, nel caso in cui un processo finisce e solo successivamente viene invocata una wait dal padre.

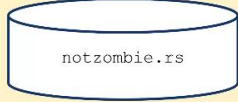
```
use std::process::{Command};
use std::thread;
use std::time::Duration;

fn main() {
    // Creazione di un processo figlio
    let mut child = Command::new("sleep")
        .arg("5")
        .spawn()
        .expect("failed to start child process");

    // Simulare un ritardo nel processo genitore
    println!("Waiting for the child process to finish...");
    thread::sleep(Duration::from_secs(7));

    // Attesa del processo figlio per evitare che diventi zombie
    let status = child.wait()
        .expect("failed to wait on child process");

    println!("Child process exited with status: {}", status);
}
```



InterProcess Communication

Il sistema operativo impedisce il trasferimento diretto di dati fra due processi, ma esistono alcuni meccanismi per superare questa barriera.

La difficoltà nello scambio di dati è dovuta alla rappresentazione interna, che può essere molto diversa e coinvolgere tipi di dati, come i puntatori, che non hanno senso al di fuori del proprio spazio di indirizzamento.

Una possibile soluzione utilizza dei formati intermedi (rappresentazione esterna) in linguaggi standard basati su testo (xml, json, ...) o binari (xdr, hdf, ...). Il passaggio a queste rappresentazioni è detta **serializzazione** (o **marshalling**).

Coda di messaggi

È una struttura dati che il sistema operativo mantiene al suo interno. Permette di raccogliere e accodare dei messaggi e di farli leggere, solitamente, ad un singolo destinatario. La comunicazione è asincrona, per cui il sistema operativo si fa carico di memorizzare i messaggi fin quando il destinatario non li avrà letti. La comunicazione, inoltre, è mono-direzionale.

INFO: La coda ha un nome univoco a livello di sistema operativo. Il destinatario definisce questo identificativo, così come il formato da utilizzare per i messaggi.

Pipe

Le pipe sono un altro strumento, che però può essere utilizzato quando c'è un legame stretto fra i processi coinvolti. La comunicazione è 1-1 e i messaggi non sono delimitati, per cui appaiono come un flusso di byte.

INFO: Per segmentare i messaggi, si può introdurre un prefisso che permetta di delinearne l'inizio (per esempio specificando la lunghezza).

```
use std::process::Stdio;
use std::process::Command;
fn main(){
    let echo_child = Command::new("echo")
        .arg("Ciao Mama")
        .stdout(Stdio::piped())
        .spawn()
        .expect("Failed to start echo process");

    let echo_out = echo_child.stdout.expect("Failed to open echo stdout");

    let sed_child = Command::new("sed")
        .arg("s/Mama/Mamma/")
        .stdin(Stdio::from(echo_out))
        .stdout(Stdio::piped())
        .spawn()
        .expect("Failed to start sed process");

    let output = sed_child.wait_with_output().expect("Failed to wait on sed");
    println!("{}", String::from_utf8_lossy(&output.stdout));
}
```

A diagram of a pipe file, represented as a cylinder with the text "pipe.rs" inside.

INFO: Il crate `serde` offre le funzionalità per serializzare e deserializzare buona parte delle strutture dati Rust, usando formati come json, toml, yaml.

```

use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let point = Point { x: 1, y: 2 };

    let serialized = serde_json::to_string(&point).unwrap();

    println!("{}", serialized); // {"x":1,"y":2}

    let deserialized: Point = serde_json::from_str(&serialized).unwrap();

    println!("{}", deserialized); // Point { x: 1, y: 2 }
}

```

Named Pipe

Sono anche chiamate FIFO e sono meno restrittive delle pipe. Offrono i seguenti vantaggi: esistono nel file system, possono essere utilizzati al di fuori di una parentela padre/figlio, esistono fino a quando non sono esplicitamente cancellati. Come le precedenti, sono unidirezionali.

Le named pipe sono create usando il comando `mkfifo` e devono essere aperte attraverso il metodo `open()`.

```

use std::process::Command;
use std::fs::OpenOptions;
use std::io::Write;
use std::path::Path;

fn main() {
    let pipe_path = "/tmp/my_pipe";

    Command::new("reader") // Avvia il processo lettore
        .env("PATH", "./target/debug/")
        .spawn()
        .expect("Failed to start child process");

    // Check if the named pipe exists
    if ! Path::new(pipe_path).exists() {
        eprintln!("Named pipe {} does not exist. Please create it first using mkfifo.", pipe_path);
        return;
    }

    // Open the named pipe for writing
    let mut pipe = match OpenOptions::new().write(true).open(pipe_path) {
        Ok(pipe) => pipe,
        Err(e) => {
            eprintln!("Failed to open named pipe: {}", e);
            return;
        }
    };

    let message = "Hello from Rust!\n";

    // Write a message to the named pipe
    match pipe.write_all(message.as_bytes()) {
        Ok(_) => println!("Message sent to the named pipe."),
        Err(e) => eprintln!("Failed to write to the named pipe: {}", e),
    }
}

```



```

use std::fs::OpenOptions;
use std::io::{BufRead, BufReader};
use std::path::Path;

fn main() {
    let pipe_path = "/tmp/my_pipe";

    // Check if the named pipe exists
    if !Path::new(pipe_path).exists() {
        eprintln!("Named pipe {} does not exist. Please create it first using mkfifo.", pipe_path);
        return;
    }

    // Open the named pipe for reading
    let pipe = match OpenOptions::new().read(true).open(pipe_path) {
        Ok(pipe) => pipe,
        Err(e) => {
            eprintln!("Failed to open named pipe: {}", e);
            return;
        }
    };

    // Create a buffered reader for the pipe
    let reader = BufReader::new(pipe);

    // Read from the pipe line by line
    for line in reader.lines() {
        match line {
            Ok(content) => println!("Received: {}", content),
            Err(e) => eprintln!("Failed to read from named pipe: {}", e),
        }
    }
}

```



Mail slot

Si tratta di una coda unidirezionale, dove il mittente viene identificato attraverso il metodo **MailslotClient** e il destinatario con **MailslotServer**.

```

use mail_slot::{MailslotServer, MailslotName};

fn main() {
    // Crea un nome per il mailslot (puoi cambiarlo a tuo piacimento)
    let name = MailslotName::local("naive");

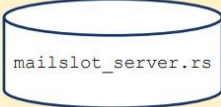
    // Crea un server mailslot
    let mut server = MailslotServer::new(&name).unwrap();

    println!("Server is running, waiting for messages...");

    'outer: loop {
        while let Some(msg) = server.get_next_unread().unwrap() {
            let msg_str = String::from_utf8(msg).unwrap();

            if msg_str == "STOP" {
                println!("Stop from client");
                break 'outer;
            }
            println!("message from client {}", msg_str);
        }
    }
}

```



```

use mail_slot::{MailslotName, MailslotClient};

fn main() {
    // Crea un nome per il mailslot (puoi cambiarlo a tuo piacimento)
    let name = MailslotName::local("naive");

    // Crea un client mailslot
    let mut client = MailslotClient::new(&name).unwrap();

    // Data to be sent
    let numbers = [42, 256, 1024];
    let text = "Hello, server!";

    // Create the message
    let message = format!("{}", {}, {}, {}, {}, numbers[0], numbers[1], numbers[2], text);

    // Write the message to the mail slot
    client.send_message(message.as_bytes()).unwrap();

    client.send_message(b"STOP").unwrap();
}

```

